



TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS

*An Adaptive Overcurrent Protection Scheme for Dual-Setting Directional
Recloser and Fuse Coordination in Unbalanced Distribution Networks
With Distributed Generation*

BY:

Jhala Nath Kafle

(081MSPSE009)

A PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE
REQUIREMENTS FOR THE MASTER'S DEGREE IN POWER SYSTEM
ENGINEERING

DEPARTMENT OF ELECTRICAL ENGINEERING

OCTOBER, 2026

PAGE OF APPROVAL

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS
DEPARTMENT OF ELECTRICAL ENGINEERING

The undersigned certify that they have read, and recommended to the Institute of Engineering for acceptance, a project report entitled “*An Adaptive Overcurrent Protection Scheme for Dual-Setting Directional Recloser and Fuse Coordination in Unbalanced Distribution Networks With Distributed Generation*” submitted by *Jhala Nath Kafle* in partial fulfilment of the requirements for the Master’s degree in Power System Engineering.

Er. Jeetendra Chaudhary
Associate Professor & Head of Department
Department of Electrical Engineering
Pulchowk Campus, Lalitpur
Institute of Engineering, Tribhuvan University
Kathmandu, Nepal

Er. Akhileshwar Kumar Mishra
Assistant Professor & Coordinator (M.Sc.)
Department of Electrical Engineering
Pulchowk Campus, Lalitpur
Institute of Engineering, Tribhuvan University
Kathmandu, Nepal

Date: _____

COPYRIGHT

The author has agreed that the Library, Department of Electrical Engineering, Pulchowk Campus, Institute of Engineering may make this report freely available for inspection. Moreover, the author has agreed that permission for extensive copying of this project report for scholarly purpose may be granted by the Head of the Department wherein the project report was done. It is understood that recognition will be given to the author of this report and to the Department of Electrical Engineering, Pulchowk Campus, Institute of Engineering in any use of the material of this project report. Copying or publication or the other use of this report for financial gain without approval of the Department of Electrical Engineering, Pulchowk Campus, Institute of Engineering and the author's written permission is prohibited.

Request for permission to copy or to make any other use of the material in this report in whole or in part should be addressed to:

Head
Department of Electrical Engineering
Pulchowk Campus, Institute of Engineering
Lalitpur, Nepal

ACKNOWLEDGEMENTS

I express my sincere gratitude to *Er. Jeetendra Chaudhary*, Head of the Department, and to the faculty of the Department of Electrical Engineering, Pulchowk Campus, Institute of Engineering, for the guidance, the review comments and the facilities that made this project possible, and to *Er. Akhileshwar Kumar Mishra*, coordinator of the M.Sc. programme in Power System Engineering, for the encouragement throughout the work.

I also thank my colleagues of the M.Sc. Power System Engineering programme for the discussions on distribution protection and on the use of DIGSILENT PowerFactory. I am especially thankful to my brother, *Er. Subash Khanal*, and to *Er. Sagar Shrestha* for their constant support throughout this work, and to my family for their patience and encouragement.

Jhala Nath Kafle
081MSPSE009

ABSTRACT

The recloser–fuse coordination of distribution networks with distributed generation (DG) can be lost when conventional overcurrent protection is used, because the DG changes the magnitude and the direction of the fault current seen by the protective devices. This project studies a protection coordination strategy based on a dual-setting directional recloser (DSDR) for the fuse-saving scheme of an unbalanced distribution network. The mid-line recloser is given independent forward and reverse settings, so that it coordinates with the lateral fuses for faults on both sides of it. The strategy is implemented on the IEEE 13-node test feeder in DIgSILENT PowerFactory with a synchronous DG, the characteristics of commercial recloser relays and fuses, unbalanced load-flow and short-circuit studies, and an electromagnetic-transient simulation of the reclosing sequence. The results show that the DG makes the fuses operate before the fast operation of a conventional recloser, increasingly so as the DG penetration rises. With the DG connected, a conventional single-setting mid-line recloser keeps the coordination in 24 of the 39 studied fault cells. The dual-setting directional recloser alone raises this to 25 cells, and together with a revision of the fuse sizes to 38 of 39 cells under the zero-margin criterion; the remaining cell, an LG fault at node 692, is limited by the delayed dial range of the recloser.

Keywords: distributed generation, dual-setting directional recloser, fuse saving, recloser–fuse coordination, IEEE 13-node feeder, DIgSILENT PowerFactory.

TABLE OF CONTENTS

PAGE OF APPROVAL	II
COPYRIGHT	III
ACKNOWLEDGEMENTS	IV
ABSTRACT	V
TABLE OF CONTENTS	VI
LIST OF FIGURES	IX
LIST OF TABLES	X
LIST OF SYMBOLS AND ABBREVIATIONS	XII
CHAPTER ONE: INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives	1
1.3.1 General Objective	1
1.3.2 Specific Objectives	2
1.4 Scope of the Study	2
1.5 Significance of the Study	2
1.6 Limitations of the Study	3
1.7 Organization of the Report	3
CHAPTER TWO: LITERATURE REVIEW	4
2.1 Recloser–Fuse Protection of Distribution Feeders	4
2.2 Impact of Distributed Generation and Directional Adaptive Protection	4
2.3 Relay and Fuse Characteristics	4
2.4 Related Research and Data Used	4
2.5 Method Followed and Research Gap	5
CHAPTER THREE: METHODOLOGY	6
3.1 Research Methodology	6
3.2 Model Setup in DIGSILENT PowerFactory	7
3.3 Load Flow Analysis	7
3.4 Recloser Pickup Current	7
3.5 Short-Circuit Analysis	7
3.6 Mathematical Formulation of the DSDR	7
3.6.1 Forward Fast and Delayed Operating Time	7
3.6.2 Forward Pickup Current and Constraints	8
3.6.3 Fuse Characteristic and Coefficient	8

3.6.4	Series Fuses	8
3.6.5	Reverse Operating Times, Pickup and Constraints	9
3.7	Device Characteristics Used in the Model	9
3.7.1	R1: GE IAC77B801A	9
3.7.2	R2: GE/Alstom CDG34	10
3.7.3	Fuses	10
3.8	Coordination Criteria	10
3.9	Design Procedure	10
CHAPTER FOUR: SYSTEM MODELING AND SIMULATION SETUP		12
4.1	DIgSILENT PowerFactory Environment	12
4.2	IEEE 13-Node Feeder Model	12
4.3	Source and Feeder	13
4.4	Distributed Generation Model	13
4.5	Recloser Model	14
4.6	Fuse Model	14
4.7	Fault Model	15
4.8	Measurement and Data Acquisition	16
4.9	Simulation Cases	16
CHAPTER FIVE: RESULTS OBTAINED		17
5.1	Load-Flow Results	17
5.2	Branch Currents and Fault Levels	18
5.3	Recloser Pickup Currents and Settings	18
5.4	Short-Circuit Results With and Without DG	18
5.5	Fuse Coefficients	19
5.6	Conventional Coordination Without DG	20
5.7	Coordination With DG and a Conventional R2	21
5.8	DSDR Settings and Fuse Revision	23
5.9	Coordination With the DSDR	24
5.10	Operating Times With the DSDR	26
5.11	Single Setting Against Dual Setting on Selected Faults	27
5.12	Time-Sequence Check	30
5.13	Time-Domain Verification	31
5.14	Verification With the Relay and Fuse Models of PowerFactory	33
5.15	Sensitivity of the Pickups With DG	33
5.16	Effect of the DG Penetration Level	33
5.17	Results Summary	37
CHAPTER SIX: DISCUSSION		38
6.1	Effect of DG on Fault Current	38
6.2	Effect of Fault Current Direction	38
6.3	Performance of the Conventional Protection	38
6.4	Performance of the DSDR	38

6.5	Limits Reached by the Devices	38
6.6	Engineering Implication	39
CHAPTER SEVEN: CONCLUSION AND FUTURE WORK		40
7.1	Conclusion	40
7.2	Major Findings	40
7.3	Contributions of the Present Work	41
7.4	Future Work	41
REFERENCES		42
APPENDIX A: IMPLEMENTATION FORMULAS		43
APPENDIX B: COMPARISON WITH THE REFERENCE PAPER		44
APPENDIX C: PYTHON SCRIPTS		52
APPENDIX D: COMPLETE COPY OF THE REFERENCE PAPER		91

LIST OF FIGURES

Figure 3.1	Flowchart of the recloser–fuse coordination method with the DSDR.	6
Figure 4.1	IEEE 13-node feeder with the reclosers R1 and R2, the fuses and the DG at node 692.	12
Figure 5.1	LG fault at 611: R2 forward, fuses F684 and F671-2; CTI = +143 ms.	21
Figure 5.2	LLG fault at the middle of 692–675 through 1 Ω ; CTI = +127 ms. .	21
Figure 5.3	Coordination status with DG, without the DSDR.	22
Figure 5.4	LL fault at 646 through 1 Ω : R1 with F646 and F632; CTI = +200 ms.	22
Figure 5.5	LL fault at 645 through 1.5 Ω , conventional R2.	23
Figure 5.6	Three-phase fault at 10 per cent of line 632–633, conventional R2; CTI = –83 ms, the fuse melts first.	23
Figure 5.7	The two setting groups of R2: reverse (DG contribution, faults towards 632) and forward (grid contribution, faults beyond 671), each with one fault of the model.	24
Figure 5.8	Coordination status with DG and the DSDR.	25
Figure 5.9	Bolted LL fault at 646: R2 reverse setting with F646 and F632; CTI = +328 ms.	26
Figure 5.10	Bolted LL fault at 646: R1 with F646 and F632; CTI = +226 ms. . .	26
Figure 5.11	Three-phase fault at 10 per cent of line 632–633: single setting (left, CTI = –22 ms) and dual setting (right, CTI = +16 ms).	28
Figure 5.12	LG fault at 646: the single-setting R2 does not pick up the reverse current (left); the reverse group does (right, CTI = +661 ms).	29
Figure 5.13	Time-sequence check of all 39 cells, single against dual setting. . .	31
Figure 5.14	Time-domain current at node 632 for an LL (a–c) fault at 684 through 0.2 Ω : without DG (top) and with DG (bottom).	32
Figure 5.15	Percentage variation of the CTI with DG penetration (outer ring 671, inner ring 633).	34
Figure 5.16	Currents through the fuse (grid and DG share), the recloser (grid share) and from the DG, and the resulting CTI, against DG penetration.	34
Figure 5.17	Time–current characteristics at all penetration levels: fuse operating points (squares) and recloser operating points (circles).	35
Figure 5.18	Node 633, fuse F633 and R1, at each penetration level, with the short-circuit currents, operating times and CTI.	35
Figure B.1	CTI against DG penetration: this study (left) and the values read from the paper’s Fig. 7 (right).	51

LIST OF TABLES

Table 4.1	Parameters of the synchronous DG in the PowerFactory model. . . .	13
Table 4.2	Recloser settings. For R2 the fast plug is half and the delayed plug equal to $1.25I_{nom}$, because the CDG curve starts at twice the plug setting.	14
Table 4.3	Fuses and their sizes (A055C, E-rated). Bold: changed by the fuse revision with the DG.	15
Table 4.4	Simulation cases.	16
Table 5.1	Bus voltages from the unbalanced load flow, without and with the DG.	17
Table 5.2	Load current at the reclosers (largest phase; + is forward, away from the grid).	17
Table 5.3	Rated, minimum and maximum fault current of the protected branches without DG. XFM1 HV side: 4.16 kV currents for a fault on the 0.48 kV side.	18
Table 5.4	Maximum fault current and recloser currents without and with the DG (+ forward, – reverse).	19
Table 5.5	Fuse coefficients, eq. (3.7), $a_i = -1.8$. Size and t_{MMT} : fuse of the design without DG at the same current.	20
Table 5.6	Coordination status with DG and a conventional R2. ✓ exists, × does not exist, – not applicable.	22
Table 5.7	Coordination status with DG and the revised fuses: single-setting R2 against dual-setting R2.	25
Table 5.8	Recloser operating times with the DSDR, DG in service.	27
Table 5.9	Fuse times with the DSDR, DG in service. Margin = t_{MMT} minus the last fast trip (R2 for faults below R2; the later of R1 and R2 reverse above it). Nodes 632, 671 and 680 are on the main feeder and have no fuse.	27
Table 5.10	Nine faults with the DG in service: single-setting against dual-setting R2, same fuses.	28
Table 5.11	Three-phase fault at 10 per cent of line 632–633: currents, settings and operating times.	29
Table 5.12	LG fault at 646 (phase c): currents, settings and operating times. . .	30
Table 5.13	Reclosing sequence of the EMT simulation.	32
Table 5.14	CTI against DG penetration, conventional protection.	33
Table 5.15	Largest bolted fault current at each node (A) against DG penetration.	36
Table 5.16	Load flow, pickups by eq. (3.3) and lowest fault current through each recloser against DG penetration. *: below the start of the recloser's curve with the fixed settings (not detected).	36
Table 5.17	Summary of the coordination results.	37
Table B.1	Comparison of the results with the reference paper.	44
Table B.2	DG parameters: reference paper (Table I) against the PowerFactory model.	46

Table B.3	Branch currents: model against Table II of the paper. XFM1 HV side: the model values are 4.16 kV currents for a fault on the 0.48 kV side, so this row is not comparable.	47
Table B.4	Maximum fault current, DG out: IEEE benchmark, this study and the paper's Table II. Deviations are relative to the benchmark.	48
Table B.5	Fuse coefficients against Table III of the paper. " b_i rev.": index i counted from the source, which reproduces the paper's values in R1's zone.	48
Table B.6	Recloser operating times against Table IV of the paper.	49
Table B.7	Fuse times against Table IV of the paper.	49
Table B.8	Coordination status with DG and a conventional R2: model against the paper's Fig. 14.	50
Table B.9	Reclosing sequence of the EMT simulation against the paper's Fig. 10.	50
Table B.10	CTI shares against DG penetration: this study and the paper's Fig. 7.	51
Table C.1	Python scripts of the study.	52

LIST OF SYMBOLS AND ABBREVIATIONS

Symbols

I_f	Fault current seen by a protection device
I_p	Pickup current of a recloser
I_s	Plug (current) setting of the CDG relay
I_{nom}	Rated (load) current
M	Multiple of pickup, I_f/I_p
TDS, TMS	Time-dial / time-multiplier setting
t_F, t_D	Fast and delayed operating time of a recloser
t_{MMT}, t_{TCT}	Minimum-melting and total-clearing time of a fuse
A, B, n	Constants of the inverse-time curve
a_i, b_i	Coefficients of the fuse characteristic
k, l, m	Index of the recloser, of a downstream and of an upstream fault location
z, i	Number of series fuses in the fault path; position of a fuse in it
Z_f	Fault impedance
X_d, X'_d, X''_d	Direct-axis synchronous, transient and subtransient reactance
X_q, X'_q, X''_q	Quadrature-axis reactances
R_a	Armature resistance

Abbreviations

CT	Current Transformer
CTI	Coordination Time Interval
DG	Distributed Generation
DSDR	Dual-Setting Directional Recloser
EMT	Electromagnetic Transient (simulation)
LG	Line-to-Ground fault
LL	Line-to-Line fault
LLG	Line-to-Line-to-Ground fault
LLL	Three-Phase fault
MMT	Minimum Melting Time
OLF	Overload Factor
PD	Protection Device
PF	DIgSILENT PowerFactory
R1	Feeder-Head Recloser
R2	Mid-Line Recloser (the DSDR)
RMS	Root Mean Square
SBDG	Synchronous-Based Distributed Generator
TCC	Time–Current Characteristic
TCT	Total Clearing Time
TDS	Time Dial Setting
TMS	Time Multiplier Setting

CHAPTER ONE: INTRODUCTION

1.1 Background

Overcurrent protection of radial distribution networks is traditionally coordinated on the assumption that the fault current flows from the substation towards the loads. A recloser upstream of a fuse first operates on a fast characteristic, to clear a temporary fault before the fuse melts, and then on a delayed characteristic, to let the fuse clear a permanent fault. This *fuse-saving* arrangement only works while the fuse curve lies between the two recloser curves over the whole range of fault currents.

Distributed generation changes this. A DG downstream of a recloser feeds current towards a fault upstream of it, so the recloser carries current in the reverse direction, and a fuse between the two sources carries the sum of both contributions while each recloser carries only one. The current seen by a protection device then depends on the fault location, the topology and the DG, and a single fixed setting is no longer suitable for both directions.

The dual-setting directional recloser (DSDR) addresses this by giving the mid-line recloser separate pickup and time-dial settings for the forward and for the reverse direction. This project implements the DSDR method of [1] on the IEEE 13-node feeder in DIgSILENT PowerFactory and evaluates it.

1.2 Problem Statement

The problem addressed is the loss of recloser–fuse coordination caused by the change of fault-current magnitude and direction when a DG is connected. A conventional scheme that is coordinated without DG may lose the fuse-saving property in the zone between the grid and the DG. The project investigates whether the DSDR method restores the coordination in an independently built model that uses the manufacturer-based time–current characteristics of the specified commercial protection devices.

1.3 Objectives

1.3.1 General Objective

To build and validate a DIgSILENT PowerFactory implementation of the DSDR protection-coordination method on the IEEE 13-node test feeder, with and without distributed generation.

1.3.2 Specific Objectives

1. Build and verify the IEEE 13-node feeder with its recloser and fuse protection.
2. Perform unbalanced load-flow and short-circuit studies for all fault types and phase combinations.
3. Calculate the recloser pickup currents, time dials and the fuse coefficients with the method's equations.
4. Establish the conventional recloser–fuse coordination without DG.
5. Connect the 4.05 MVA synchronous DG at node 692 and quantify the change of the fault currents and of their direction at the reclosers.
6. Classify the coordination with DG for a conventional recloser R2.
7. Design the forward and reverse settings of R2 as a DSDR, classify the resulting coordination, and determine the corresponding operating times.
8. Verify the reclosing sequence in the time domain.
9. Study how the coordination degrades as the DG penetration rises from 0 to 100 per cent.

1.4 Scope of the Study

The study covers the IEEE 13-node feeder: unbalanced load flow, LG, LL, LLG and LLL faults at twelve fault locations, pickup and time-dial calculation, fuse coordination, the DSDR forward and reverse settings, the coordination classification with and without the DSDR, and one EMT simulation. The IEEE 34-node feeder is outside the completed work.

1.5 Significance of the Study

The study gives a reproducible implementation of a protection-coordination method for feeders with DG in a tool that utilities use. It shows what the method achieves when the real relay dials and fuse sizes are respected, and it separates the contribution of the dual setting from the contribution of the fuse revision.

1.6 Limitations of the Study

- The method does not uniquely specify the R2 settings, the fuse sizes, the source impedance or the fault type of each operating-time case; these choices were therefore defined in this study, and every choice and its basis is listed in Section 4.8.
- The library model of the CDG34 relay is not directional. The two setting groups are modelled as separate relay units and the direction is assigned from the fault location.
- Coordination is judged with a zero-margin criterion: the fuse must not start to melt before the fast trip.
- The DG penetration study (Section 5.16) uses one DG location (692) and keeps the protection of the design without DG at every level.
- The IEEE 34-node feeder has not been modelled.

1.7 Organization of the Report

Chapter One introduces the problem and the objectives. Chapter Two reviews the literature on the protection concepts and the method followed. Chapter Three gives the method and its equations. Chapter Four describes the PowerFactory model, the DG, the protection devices and the simulation cases. Chapter Five presents the results. Chapter Six discusses the findings and their limits. Chapter Seven gives the conclusions and the future work. The comparison with the reference paper is in Appendix B.

CHAPTER TWO: LITERATURE REVIEW

2.1 Recloser–Fuse Protection of Distribution Feeders

Distribution feeders are protected by reclosers and fuses with inverse-time characteristics: the operating time falls as the multiple of the pickup current rises, so the device nearest the fault acts first and the upstream devices act as backup [2]. The pickup fixes where the characteristic starts and the time dial scales the operating time; real relays offer only discrete taps and a limited dial range. In the fuse-saving scheme the recloser’s fast curve must lie below the minimum-melting curve of the fuse, so that a temporary fault is cleared without blowing the fuse, and its delayed curve above the total-clearing curve, so that the fuse isolates a permanent fault. Fuses in series must be selective: the fuse nearer the fault clears before the upstream fuse starts to melt.

2.2 Impact of Distributed Generation and Directional Adaptive Protection

A DG raises the current through the fuses and can reverse the current through a recloser. The recloser and the fuse then no longer carry the same current, and a scheme that was coordinated without DG may let the fuse melt before the fast trip [3]. A directional element distinguishes the direction of the fault current, and adaptive protection changes the settings with the state of the network. The dual-setting directional recloser (DSDR) combines both: it holds a forward and a reverse setting group and selects one from the direction of the current, without communication [1].

2.3 Relay and Fuse Characteristics

Inverse-time relay curves are given by standard equations [2] or by the manufacturer’s own equation or table; the GE IAC and the CDG series follow their manufacturer’s curves, in which the time is the time dial multiplied by a function of $M = I_f/I_p$ [4]. A fuse is described by its minimum-melting time (MMT), when the element starts to melt, and its total-clearing time (TCT), when the arc is extinguished.

2.4 Related Research and Data Used

A classification technique decides for each fault whether recloser–fuse coordination with DG exists [3]. A benchmark overcurrent protection scheme for the IEEE 13-node feeder [5], with device types and fuse sizes, is given in [6]; the published short-circuit cur-

rents of the IEEE test feeders [7] are used here to validate the fault levels, and the relay curves are those of the manufacturers' manuals [4]. Because the feeder is unbalanced, all calculations are made per phase and keep only the phases that exist at each node.

2.5 Method Followed and Research Gap

The study follows the adaptive overcurrent protection scheme with a DSDR of [1]: its equations for the forward and reverse operating times, pickups, fuse coefficients and setting constraints, and its flowchart for the design sequence. The reference reports that the DSDR restores coordination in every case, but it does not publish the DSDR settings, and its fuse curves differ from its own fuse equation. The gap addressed here is an independent implementation in PowerFactory with device-specific relay curves, dial ranges and fuse sizes, which shows how much of the improvement comes from the dual setting and where the method reaches the limits of the devices.

CHAPTER THREE: METHODOLOGY

3.1 Research Methodology

The work follows the flowchart of the reference method [1], shown in Figure 3.1 as it is implemented. It has three stages: (A) the conventional scheme without DG, (B) the same settings with the DG connected, and (C) R2 as a dual-setting directional recloser with the fuse revision of the method.

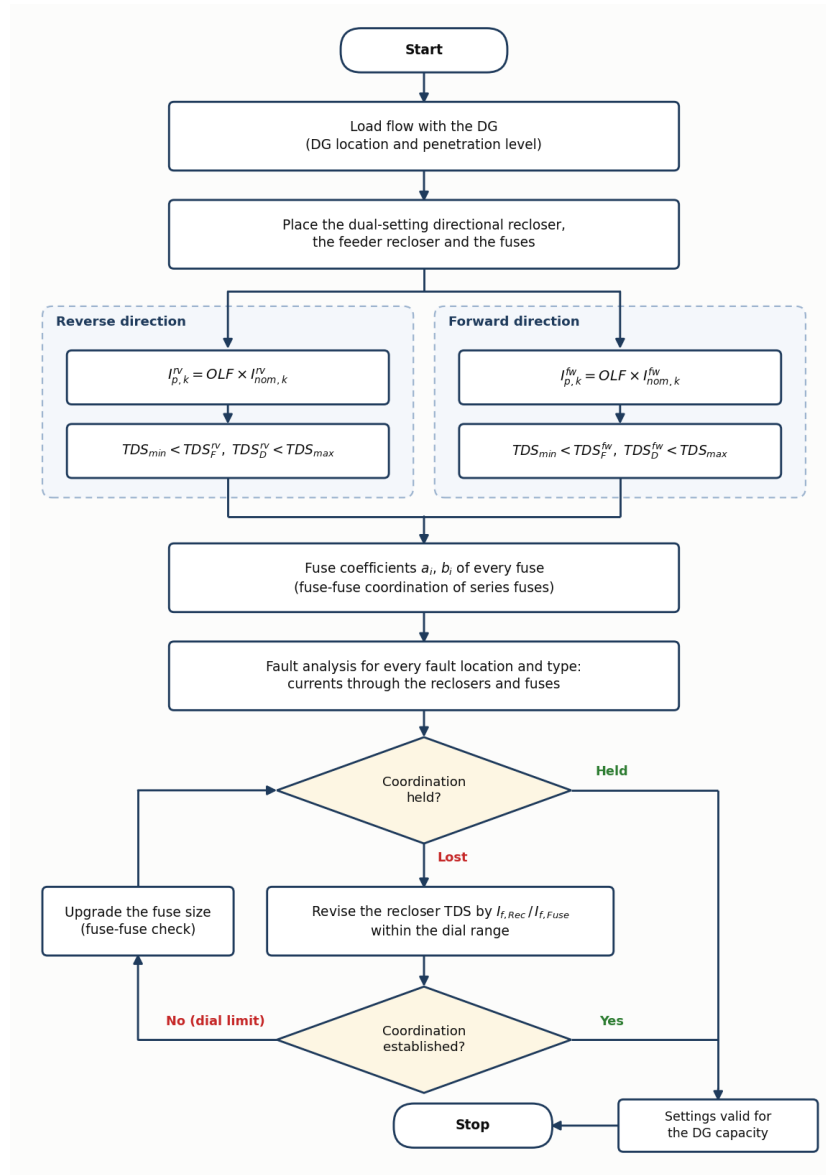


Figure 3.1 Flowchart of the recloser-fuse coordination method with the DSDR.

3.2 Model Setup in DIgSILENT PowerFactory

The study is carried out in DIgSILENT PowerFactory 2021 SP2 [8]. The feeder is the IEEE 13-node model of the DIgSILENT example library; its lines, loads and regulator taps were checked against the IEEE data. The study case that includes the 115/4.16 kV, 5 MVA substation transformer is used, because it reproduces the IEEE short-circuit benchmark [7].

3.3 Load Flow Analysis

The load flow is the unbalanced three-phase AC load flow of PowerFactory. It gives the rated current of every protected branch (largest phase), the bus voltages per phase, and the current and its direction at the two reclosers with and without the DG.

3.4 Recloser Pickup Current

The method uses an overload factor of 1.25 [1]. The pickup is 1.25 times the rated current of the protected branch, eq. (3.3), and is then set on the nearest available tap of the relay.

3.5 Short-Circuit Analysis

Short circuits are calculated with the *complete method* of PowerFactory, which starts from the pre-fault load flow. LG, LL, LLG and LLL faults are applied at every node for every phase combination that exists there, with the DG out and in. The maximum fault current of a branch is a bolted fault at its nearest node; the minimum is an LG fault through 3Ω at its farthest node [1]. The current through R1, R2, every fuse and every branch of Table 5.3 is stored for each fault.

3.6 Mathematical Formulation of the DSDR

The equations below are those of the reference method [1].

3.6.1 Forward Fast and Delayed Operating Time

$$t_{k,l,F}^{fw} = TDS_{k,F}^{fw} \left[\frac{A_k^{fw}}{\left(I_{f,l}/I_{p,k}^{fw} \right)^{n_k^{fw}} - 1} + B_k^{fw} \right] \quad (3.1)$$

$$t_{k,l,D}^{fw} = TDS_{k,D}^{fw} \left[\frac{A_k^{fw}}{\left(I_{f,l} / I_{p,k}^{fw} \right)^{n_k^{fw}} - 1} + B_k^{fw} \right] \quad (3.2)$$

The fast and the delayed operation of recloser k for a fault at location l use the same curve with two time dials. $I_{f,l}$ is the fault current through the recloser and I_p its pickup.

3.6.2 Forward Pickup Current and Constraints

$$I_{p,k}^{fw} = OLF \times I_{nom,k}^{fw} \quad (3.3)$$

$$TDS_{k,min} < TDS_{k,F}^{fw}, TDS_{k,D}^{fw} < TDS_{k,max} \quad (3.4)$$

$$I_{p,k,min} < I_{p,k}^{fw} < I_{p,k,max} \quad (3.5)$$

The pickup lies above the load current and below the minimum fault current of the zone; the dials must stay inside the range of the device.

3.6.3 Fuse Characteristic and Coefficient

$$\log t_{fuse,i} = a_i \log I_{f,i} + b_i \quad (3.6)$$

$$b_i = \log \left[\frac{t_{k,F}^{fw} + t_{k,D}^{fw}}{2} \right] - a_i \log I_{f,i} \quad (3.7)$$

with $a_i = -1.8$. Equation (3.7) places a single fuse midway between the fast and the delayed time at the calibration current.

3.6.4 Series Fuses

$$t_{TCT,i} \leq 0.75 t_{MMT,i+1} \quad (3.8)$$

$$b_i = \log \left[t_{k,F}^{fw} + \frac{i \left(t_{k,D}^{fw} - t_{k,F}^{fw} \right)}{z + 1} \right] - a_i \log I_{f,i} \quad (3.9)$$

where z is the number of series fuses in the fault path and $i = 1$ is the fuse closest to the fault. Equation (3.9) divides the window between the two recloser times among the series fuses.

3.6.5 Reverse Operating Times, Pickup and Constraints

$$t_{k,m,F}^{rv} = TDS_{k,F}^{rv} \left[\frac{A_k^{rv}}{\left(I_{f,m}/I_{p,k}^{rv} \right)^{n_k^{rv}} - 1} + B_k^{rv} \right] \quad (3.10)$$

$$t_{k,m,D}^{rv} = TDS_{k,D}^{rv} \left[\frac{A_k^{rv}}{\left(I_{f,m}/I_{p,k}^{rv} \right)^{n_k^{rv}} - 1} + B_k^{rv} \right] \quad (3.11)$$

$$I_{p,k}^{rv} = OLF \times I_{nom,k}^{rv} \quad (3.12)$$

$$TDS_{k,min} < TDS_{k,F}^{rv}, TDS_{k,D}^{rv} < TDS_{k,max} \quad (3.13)$$

$$I_{p,k,min} < I_{p,k}^{rv} < I_{p,k,max} \quad (3.14)$$

Here m is a fault location upstream of the recloser. The reverse current is the contribution of the DG, and the reverse pickup is based on the reverse load current that flows when the DG exports towards the substation.

3.7 Device Characteristics Used in the Model

The model uses the time–current curves of the PowerFactory library types of the protection devices.

3.7.1 R1: GE IAC77B801A

The operating time of R1 follows the GE IAC extremely inverse characteristic. The manufacturer gives it as $t = M_{TDS} [A + B/(M - C) + D/(M - C)^2 + E/(M - C)^3]$ with $A = 0.004$, $B = 0.638$, $C = 0.620$, $D = 1.787$ and $E = 0.246$ [4]. With the constants of the PowerFactory library type (the same values to four digits) the curve is

$$t = TDS \left[0.004 + \frac{0.6379}{M - 0.62} + \frac{1.7872}{(M - 0.62)^2} + \frac{0.2461}{(M - 0.62)^3} \right], \quad M = \frac{I_f}{I_p}, \quad (3.15)$$

valid from $M = 1.5$ to $M = 40$. For 4219 A, for example, it gives 0.096 s (fast, TDS 0.5) and 1.926 s (delayed, TDS 10).

3.7.2 R2: GE/Alstom CDG34

The CDG extremely inverse characteristic is a manufacturer's table of time against $M = I_f/I_s$ for time multipliers from 0.1 to 1.0. It starts at twice the plug setting and is interpolated on log-log axes:

$$t = t_1 \left(\frac{t_2}{t_1} \right)^f, \quad f = \frac{\ln(M/M_1)}{\ln(M_2/M_1)}. \quad (3.16)$$

The table is not defined below a multiplier of 0.1, so 0.1 is the fastest setting of this relay. The dial of R1 is called the time dial setting (TDS), as in the IAC77B801A documentation, whereas the CDG34 documentation calls the dial of R2 the time multiplier setting (TMS). Both scale the operating time of the curve in the same way, and the general equations of Section 3.6 write TDS for either.

3.7.3 Fuses

The fuses are Gould-Shawmut A055C (5.5 kV, E-rated) with their minimum-melting and total-clearing curves. The fuse times are read from these curves at the current through the fuse with the same interpolation as eq. (3.16). The straight line of eq. (3.6) is used only to compute the coefficients of Table 5.5.

3.8 Coordination Criteria

A fault cell is classified as *coordination exists* when all of the following hold:

$$t_{MMT}(I_{fuse}) > t_F \quad \text{for every recloser that feeds the fault through the fuse,} \quad (3.17)$$

$$t_{TCT}(I_{fuse}) < t_D \quad \text{for the same reclosers,} \quad (3.18)$$

together with the series-fuse condition of eq. (3.8) and a margin of at least 10 cycles between the delayed curves of R2 and R1. The coordination time interval is $CTI = t_{MMT} - t_F$, the time by which the fuse starts to melt after the latest fast trip; a cell is held when $CTI > 0$ (the zero-margin criterion). A cell (node and fault type) is held only if every phase combination of that fault type is held. The fuse and each recloser are evaluated at their own currents, because with DG they differ.

3.9 Design Procedure

The flowchart of Figure 3.1 is applied in these steps.

1. Run the load flow without DG and set the recloser pickups from the rated currents.
2. Run the short-circuit study for the minimum and maximum fault currents of each section.
3. Set the dials: R1 at TDS 0.5 and 10; R2 fast at its lowest dial, delayed 10 cycles below R1.
4. Calculate the fuse coefficients and check the installed fuses, including the series fuses.
5. Classify the coordination without DG and revise the fuses until every case is held.
6. Connect the DG with the settings unchanged and classify again.
7. Set the reverse pickup and the reverse dials of R2 as a DSDR.
8. Revise the dials and, where a dial limit is reached, the fuse sizes until the cases are held.
9. Classify the coordination with the DSDR and determine the operating times.
10. Verify with PowerFactory's own relay and fuse models and the EMT simulation.

CHAPTER FOUR: SYSTEM MODELING AND SIMULATION SETUP

4.1 DIgSILENT PowerFactory Environment

The model is the project *IEEE13 DSDR Fuse Coordination* in PowerFactory 2021 SP2. Load flows use the unbalanced AC method, short circuits the complete method, and the time-domain study the EMT simulation with a step of 0.1 ms. The relays and fuses are modelled with the PowerFactory library types of the actual devices.

4.2 IEEE 13-Node Feeder Model

Figure 4.1 shows the feeder with its protection devices. The feeder has the voltage regulator at the substation, overhead and cable sections, the 4.16/0.48 kV transformer XFM-1, spot and distributed loads, and two capacitors. The middle part of the distributed load on line 632–671 is placed behind a short lateral (node DL) so that a fault there lies below its fuse.

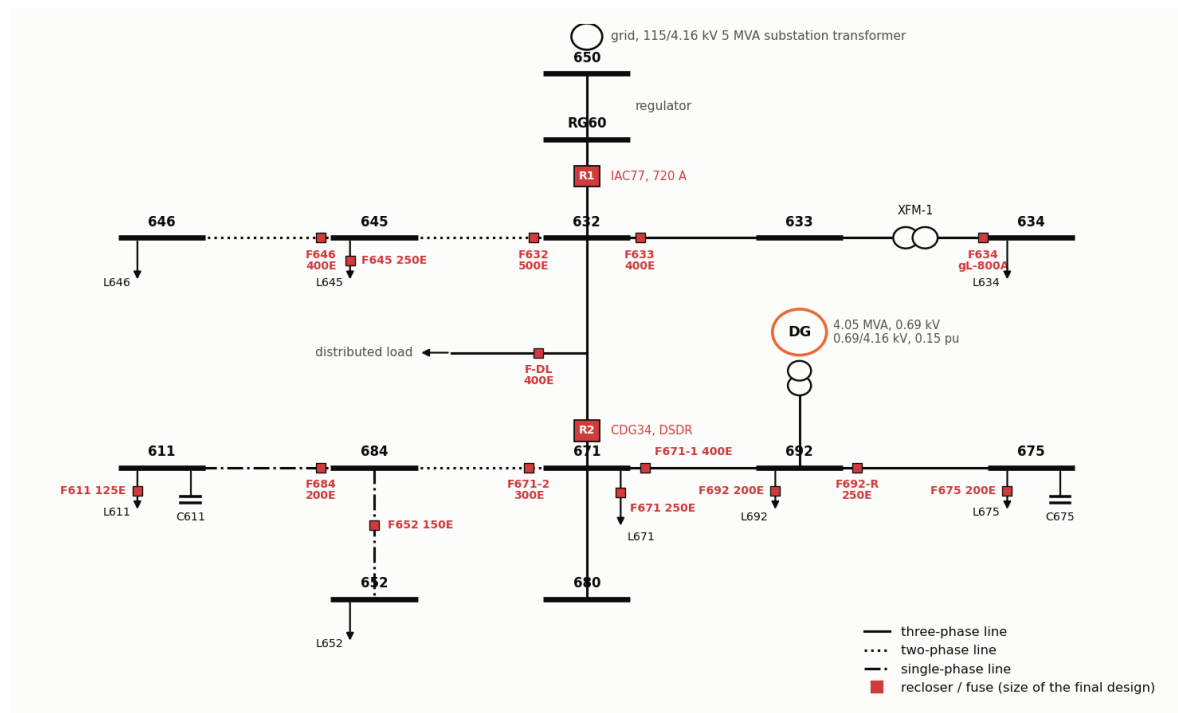


Figure 4.1 IEEE 13-node feeder with the reclosers R1 and R2, the fuses and the DG at node 692.

4.3 Source and Feeder

The source is an external grid at 115 kV behind the 5 MVA, 115/4.16 kV substation transformer ($u_k = 8.06\text{percent}$). With this source the fault currents agree with the IEEE benchmark [7] within about 2 per cent at every node. The alternative study case with a stiff 4.16 kV source overstates the fault levels by 20 to 65 per cent and is not used. The regulator taps are fixed at +10, +8 and +11.

4.4 Distributed Generation Model

The DG is a 4.05 MVA, 0.69 kV synchronous machine connected to node 692 through a 0.69/4.16 kV transformer with a leakage reactance of 0.15 p.u. It is modelled with the standard synchronous-machine model of PowerFactory, round rotor, with the short-circuit data of Table 4.1 (values read back from the model). In the load flow the machine delivers 3.24 MW and controls its terminal voltage to 1.0 p.u.

Parameter	Symbol	Value	Unit
Leakage reactance	X_l	0.05	pu
Stator resistance	R_a	0.0014	pu
d-axis synchronous reactance	X_d	1.4	pu
d-axis transient reactance	X'_d	0.231	pu
d-axis subtransient reactance	X''_d	0.118	pu
d-axis transient open-circuit time constant	T'_{d0}	5.5	s
d-axis subtransient open-circuit time constant	T''_{d0}	0.05	s
q-axis synchronous reactance	X_q	1.372	pu
q-axis transient reactance	X'_q	0.8	pu
q-axis subtransient reactance	X''_q	0.118	pu
q-axis transient open-circuit time constant	T'_{q0}	1.25	s
q-axis subtransient open-circuit time constant	T''_{q0}	0.19	s
Mechanical starting time	$M = 2H$	1.5	s
DG rating	S_n	4.05	MVA
DG voltage	U_n	0.69	kV
Step-up transformer, HV side	U_{HV}	4.16	kV
Step-up transformer leakage reactance	x_T	0.15	pu

Table 4.1 Parameters of the synchronous DG in the PowerFactory model.

4.5 Recloser Model

R1 is at the feeder head (RG60–632). R2 is on line 632–671 at the 671 end; it is the DSDR, because with the DG at 692 it carries forward current for faults at 671 and beyond and reverse current for faults towards 632. Table 4.2 gives the settings. The forward and the reverse group of R2 are four relay units of the CDG34 library type on two current transformers.

Recloser	Relay, CT	$1.25 I_{nom}$	Setting	Dial fast / delayed
R1 forward	GE IAC77B801A, 900/5	734.6 A	tap 4 A = 720 A	TDS 0.5 / 10
R2 forward	CDG34, 1000/5	587.5 A	plug 300 A / 600 A	TMS 0.1 / 1.0
R2 reverse	CDG34, 500/5	315.3 A	plug 150 A / 300 A	TMS 0.1 / 1.0

Table 4.2 Recloser settings. For R2 the fast plug is half and the delayed plug equal to $1.25 I_{nom}$, because the CDG curve starts at twice the plug setting.

4.6 Fuse Model

Table 4.3 lists the fifteen fuses. The starting size is the size drawn in the figures of the reference paper where a figure shows the fuse (the fuse bands were digitised and matched to the A055C library curves within 2 per cent), otherwise the size of the benchmark scheme [6]. The last two columns are the sizes after the fuse revision of the method without DG and with the DSDR.

Fuse	Location	Start	No DG	DSDR	Source of the starting size
F632	lateral 632–645, at 632	400E	400E	500E	Ref. [1], Figs. 12, 15, 16
F633	lateral 632–633, at 632	250E	250E	400E	Benchmark scheme [6], Table V
F634	XFM-1, 0.48 kV side	gL-800A	gL-800A	gL-800A	Benchmark scheme [6], Table V
F645	load at 645	250E	250E	250E	Benchmark scheme [6], Table V
F646	line 645–646, at 645	300E	300E	400E	Ref. [1], Figs. 11, 15, 16
F-DL	distributed-load lateral on 632–671	250E	250E	400E	Benchmark scheme [6], Table V
F671	load at 671	250E	250E	250E	Benchmark scheme [6], Table V
F671-1	671–692 (switch), at 671	300E	300E	400E	Ref. [1], Fig. 9
F692	load at 692	200E	200E	200E	Benchmark scheme [6], Table V
F692-R	cable 692–675, at 692	250E	200E	250E	Ref. [1], Fig. 9
F675	load at 675	200E	200E	200E	Benchmark scheme [6], Table V
F671-2	lateral 671–684, at 671	300E	300E	300E	Ref. [1], Fig. 8
F684	line 684–611, at 684	200E	200E	200E	Ref. [1], Fig. 8
F611	load at 611	125E	125E	125E	Benchmark scheme [6], Table V
F652	cable 684–652, at 684	150E	150E	150E	Benchmark scheme [6], Table V

Table 4.3 Fuses and their sizes (A055C, E-rated). Bold: changed by the fuse revision with the DG.

4.7 Fault Model

Faults are applied at the nodes 632, 633, 634, 645, 646, DL, 671, 692, 675, 680, 684, 611 and 652. Only the fault types that the phases of a node allow are applied, for example LL and LLG on phases b–c at 645 and 646, and LG only at 611 and 652. This gives 39 node/fault-type cells. Selected faults through a fault impedance are added for the time–current figures.

4.8 Measurement and Data Acquisition

For every fault the phase currents through R1, R2 and each fuse are read from the short-circuit result. The operating current of a device is its largest phase current. The forward direction is away from the grid; R2 sits at the 671 end of its line, where PowerFactory counts the current positive into the line, so its values are multiplied by -1 . The following modelling choices were made:

- the source: the IEEE benchmark substation transformer, with the complete short-circuit method;
- R1: tap 4 A on a 900/5 current transformer, i.e. a pickup of 720 A;
- R2: all settings of Table 4.2, derived from eqs. (3.3) and (3.12);
- the operating-time cases of Table 5.8: bolted LLL faults, LL at two-phase nodes, LG at single-phase nodes;
- the EMT study: a reclosing dead time of 0.2 s and a breaker time of three cycles.

4.9 Simulation Cases

Case	DG	Faults	Purpose
Load flow	out / in	–	Rated currents, voltages, direction at R1 and R2
Maximum faults	out / in	All nodes, LG/LL/LLG/LLL, all phase combinations, bolted	Classification, Tables II–IV
Minimum faults	out / in	LG through $3\ \Omega$ at all nodes	$I_{f,min}$, pickup check
Figure faults	in	Selected faults, bolted and through a fault impedance	Time–current figures
EMT	out / in	LL a–c at 684 through $0.2\ \Omega$	Reclosing sequence
Penetration	0–100 %	LLL at 633, LL at 684, all nodes	CTI, fault levels and pickups against DG size

Table 4.4 Simulation cases.

CHAPTER FIVE: RESULTS OBTAINED

5.1 Load-Flow Results

Table 5.1 gives the bus voltages with and without the DG. Without DG the lowest voltage is 0.971 p.u. (phase c at 611). The DG raises the voltages of the loaded phases a and c at node 671 and beyond by about 0.03 to 0.05 p.u.

Bus	Without DG (p.u.)			With DG (p.u.)		
	A	B	C	A	B	C
650	0.9981	1.0170	0.9979	1.0084	1.0200	1.0057
RG60	1.0605	1.0679	1.0665	1.0715	1.0710	1.0748
632	1.0187	1.0616	1.0142	1.0482	1.0640	1.0361
633	1.0158	1.0588	1.0124	1.0454	1.0611	1.0344
634	0.9918	1.0406	0.9936	1.0221	1.0430	1.0161
645	–	1.0540	1.0106	–	1.0563	1.0325
646	–	1.0522	1.0085	–	1.0546	1.0304
DL	1.0028	1.0666	0.9939	1.0407	1.0685	1.0221
671	0.9877	1.0722	0.9753	1.0334	1.0738	1.0094
692	0.9877	1.0722	0.9753	1.0334	1.0738	1.0094
675	0.9809	1.0767	0.9729	1.0271	1.0781	1.0072
680	0.9877	1.0722	0.9753	1.0334	1.0738	1.0094
684	0.9859	–	0.9732	1.0315	–	1.0074
652	0.9811	–	–	1.0265	–	–
611	–	–	0.9712	–	–	1.0055

Table 5.1 Bus voltages from the unbalanced load flow, without and with the DG.

Table 5.2 shows the load current at the two reclosers. With the DG the current through R2 reverses: 252.2 A flows from 671 towards 632. This reverse load current is the basis of the reverse pickup, eq. (3.12).

Device	Without DG (A)	Direction	With DG (A)	Direction
R1	+587.7	forward	+284.9	forward
R2	+470.0	forward	–252.2	reverse

Table 5.2 Load current at the reclosers (largest phase; + is forward, away from the grid).

5.2 Branch Currents and Fault Levels

Table 5.3 gives the rated current of every protected branch from the unbalanced load flow, the minimum fault current (LG through $3\ \Omega$ at the farthest node of the branch's section) and the maximum fault current (bolted, nearest node). The largest fault current at the feeder head is 4.73 kA; the fault levels agree with the IEEE 13-node short-circuit benchmark within about 2 per cent.

Branch	I_{nom} (A)	$I_{f,min}$ (kA)	$I_{f,max}$ (kA)
RG60–632	587.7	1.07	4.73
632–633	81.0	0.76	4.1
632–645	143.1	0.73	3.42
632–671	470.0	0.88	3.39
XFM1-HV side	81.0	0.08	1.81
XFM1-LV side	706.4	0.71	15.67
645–646	66.1	0.73	3.11
671–692	229.7	0.74	3.39
671–684	71.1	0.66	2.66
671–680	0.0	0.64	2.93
692–675	205.9	0.74	3.15
684–611	71.1	0.68	1.86
684–652	63.0	0.66	1.87

Table 5.3 Rated, minimum and maximum fault current of the protected branches without DG. XFM1 HV side: 4.16 kV currents for a fault on the 0.48 kV side.

5.3 Recloser Pickup Currents and Settings

With the rated currents of Table 5.3, eq. (3.3) gives 734.6 A for R1 and 587.5 A for R2 forward. The settings actually applied are those of Table 4.2. Without DG both pickups are below the lowest minimum fault current of their zones (1074 A and 879 A).

5.4 Short-Circuit Results With and Without DG

Table 5.4 gives the largest fault current at each node and the current through the two reclosers. The DG raises the fault current at every node, by 38 per cent at 632 and by 64 per cent at 671. For faults at 632, 633, 645, 646 and DL the current through R2 becomes negative: R2 then carries only the DG contribution, 1.1 to 2.0 kA, in the reverse direction, while R1 still carries the grid contribution.

Bus	Fault	$I_{f,max}$ at the fault (A)		Through R1 (A)		Through R2 (A)	
		no DG	DG	no DG	DG	no DG	DG
632	LLL ABC	4735	6526	+4735	+4734	+0	−1813
633	LLL ABC	4102	5426	+4162	+3970	+66	−1500
634	LLL ABC	15671	17608	+2034	+1611	+298	−618
645	LLG BC	3418	4341	+3528	+3330	+478	−1262
646	LLG BC	3112	3854	+3192	+3000	+475	−1135
671	LLL ABC	3390	5564	+3413	+3412	+3390	+3391
692	LLL ABC	3390	5564	+3413	+3412	+3390	+3391
675	LLL ABC	3151	5022	+3202	+3090	+3174	+3050
680	LLL ABC	2930	4460	+3006	+2789	+2976	+2757
684	LLG AC	2655	4107	+2836	+2669	+2744	+2651
652	LG A	1874	2239	+2094	+1795	+2059	+1761
611	LG C	1857	2236	+2084	+1814	+1991	+1718
DL	LLL ABC	3946	5902	+3959	+3954	+1	−1972

Table 5.4 Maximum fault current and recloser currents without and with the DG (+ forward, − reverse).

5.5 Fuse Coefficients

Table 5.5 gives the fuse coefficients of eq. (3.9), with $i = 1$ for the fuse closest to the fault. The last columns give the installed fuse and its melting time at the same current; it is much shorter than the time the equation asks for, because real fuse curves are steeper than the slope -1.8 .

Fuse	I_f (A)	i/z	t_F (s)	t_D (s)	t_{fuse} (s)	b_i	Size	t_{MMT} (s)
F632	3418	2/2	0.133	2.670	1.824	6.62	400E	0.512
F633	4102	1/1	0.100	2.008	1.054	6.53	250E	0.102
F634	15671	1/2	0.439	8.772	3.217	8.06	gL-800A	0.021
F645	3418	1/2	0.133	2.670	0.979	6.35	250E	0.158
F646	3112	1/2	0.156	3.115	1.142	6.35	300E	0.273
F-DL	3946	1/1	0.106	2.130	1.118	6.52	250E	0.112
F671	3390	1/1	0.054	1.601	0.827	6.27	250E	0.161
F671-1	3390	3/3	0.054	1.601	1.214	6.44	300E	0.218
F692	3390	1/2	0.054	1.601	0.569	6.11	200E	0.094
F692-R	3151	2/3	0.057	1.829	0.943	6.27	200E	0.112
F675	3151	1/3	0.057	1.829	0.500	6.00	200E	0.112
F671-2	2655	3/3	0.069	2.572	1.946	6.45	300E	0.428
F684	1857	2/3	0.113	5.778	2.946	6.35	200E	0.465
F611	1857	1/3	0.113	5.778	1.530	6.07	125E	0.120
F652	1874	1/2	0.112	5.660	1.961	6.18	150E	0.207

Table 5.5 Fuse coefficients, eq. (3.7), $a_i = -1.8$. Size and t_{MMT} : fuse of the design without DG at the same current.

5.6 Conventional Coordination Without DG

With the starting fuse sizes 35 of the 39 cells are coordinated. The four lost cells are the faults at 675, where the series fuses F692-R (250E) and F671-1 (300E) are not selective: F692-R does not clear the fault early enough before F671-1 starts to melt. The fuse revision of the method changes F692-R to 200E, after which all 39 cells are coordinated. Figures 5.1 and 5.2 show two cases: R2's fast curve lies below the fuses and its delayed curve above them. In every time–current plot of this chapter the dotted lines carry each operating point to the time axis with its operating time. The arrow at the fuse current gives the coordination time interval $CTI = t_{MMT} - t_F$ (green when positive, red when the fuse melts first) and the margin $t_D - t_{TCT}$ by which the fuse clears before the delayed trip.

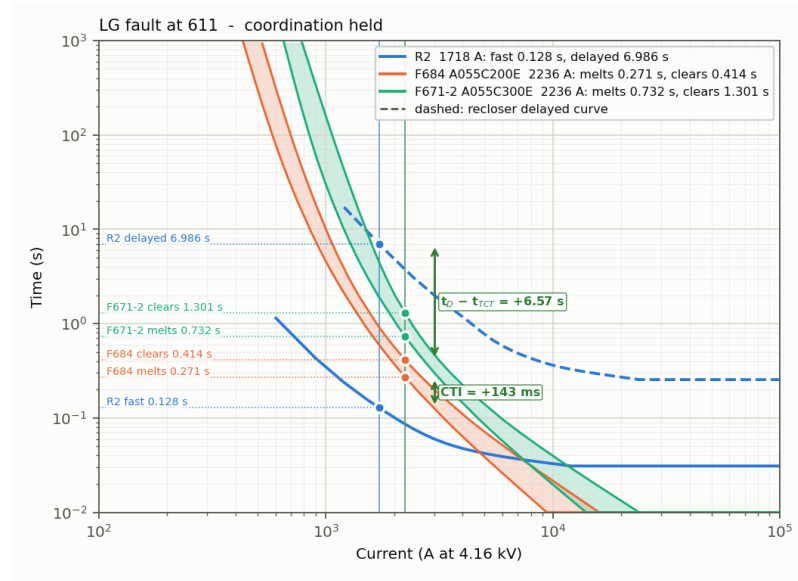


Figure 5.1 LG fault at 611: R2 forward, fuses F684 and F671-2; CTI = +143 ms.

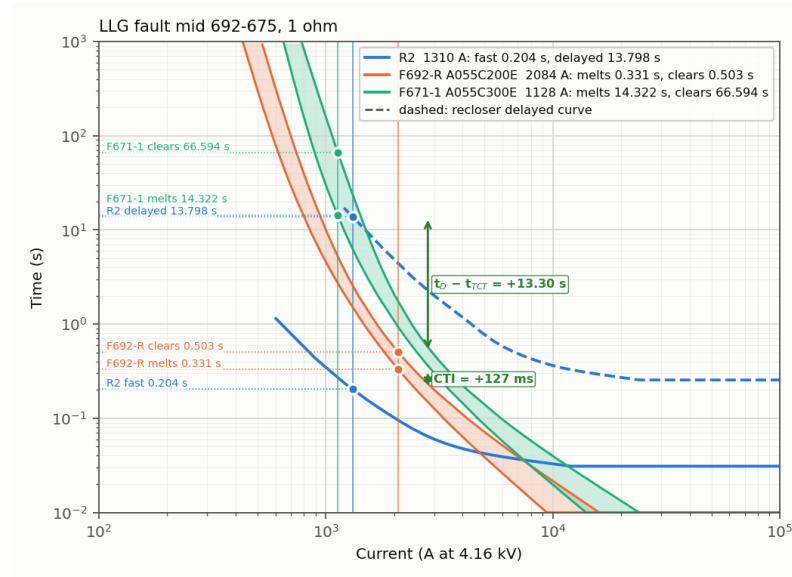


Figure 5.2 LLG fault at the middle of 692–675 through 1 Ω ; CTI = +127 ms.

5.7 Coordination With DG and a Conventional R2

With the DG connected and the settings unchanged, coordination is held in 24 of 39 cells (Table 5.6, Figure 5.3). The lost cells are at 633, 645, 646, DL and 675: the fuse now carries the grid and the DG current and melts before the fast trip, and for some LG faults the single-setting R2 does not pick up the reverse DG current at all. At 633 and DL the fuses have almost no margin even without DG, so every fault type is lost there.

Fault	632	633	645	646	DL	671	692	675	680	684	611	652
LG	✓	×	×	×	×	✓	✓	✓	✓	✓	✓	✓
LL	✓	×	✓	×	×	✓	✓	×	✓	✓	—	—
LLG	✓	×	✓	×	×	✓	✓	×	✓	✓	—	—
LLL	✓	×	—	—	×	✓	✓	×	✓	—	—	—

Table 5.6 Coordination status with DG and a conventional R2. ✓ exists, × does not exist, — not applicable.

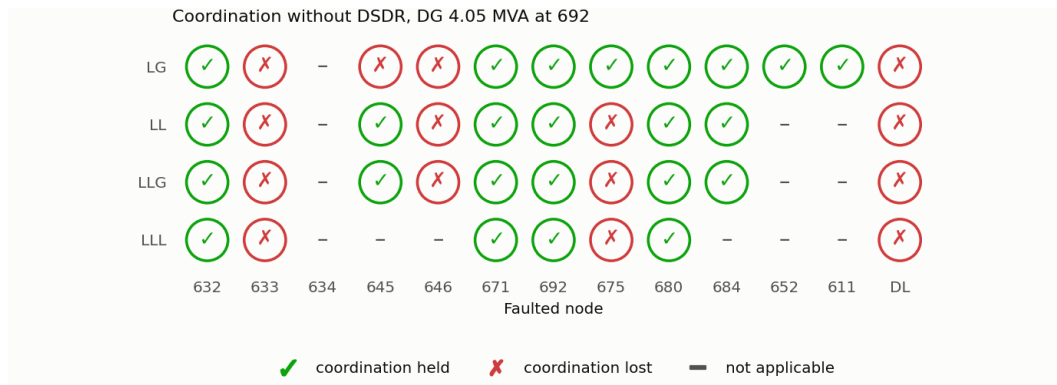


Figure 5.3 Coordination status with DG, without the DSDR.

Figures 5.4 to 5.6 show three cases with the conventional R2.

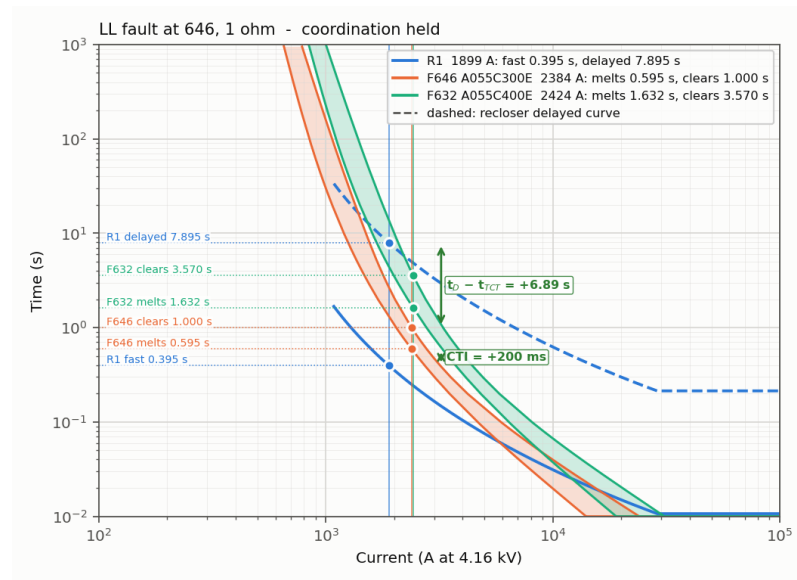


Figure 5.4 LL fault at 646 through 1 Ω: R1 with F646 and F632; CTI = +200 ms.

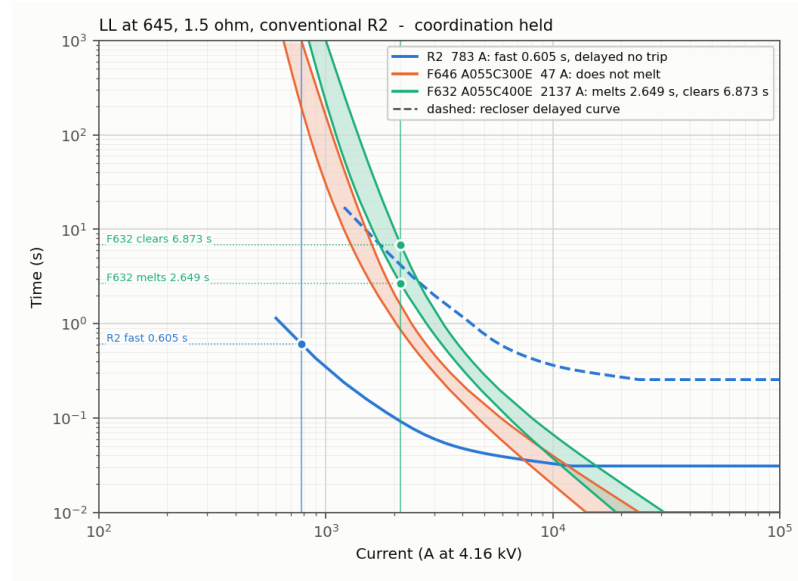


Figure 5.5 LL fault at 645 through 1.5Ω , conventional R2.

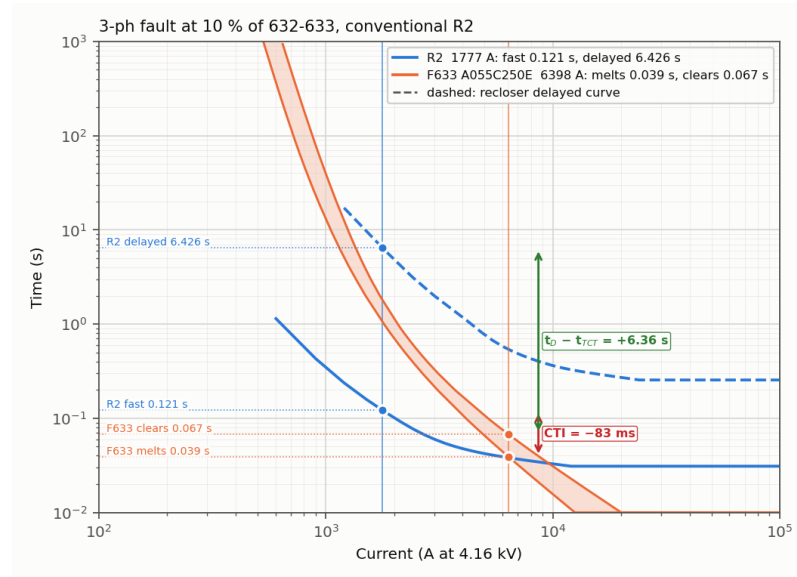


Figure 5.6 Three-phase fault at 10 per cent of line 632–633, conventional R2; CTI = -83 ms, the fuse melts first.

5.8 DSDR Settings and Fuse Revision

The reverse load current of R2 is 252.2 A, so eq. (3.12) gives a reverse pickup of 315.3 A. The reverse group is set on a 500/5 current transformer with plugs of 150 A (fast) and 300 A (delayed) and the same time multipliers 0.1 and 1.0 (Figure 5.7). Step 8 of the method (revising the dials) could not improve any lost case, because R1 is already at both ends of the IAC dial range (0.5 and 10) and R2 at both ends of the CDG table (0.1 and

1.0). Step 9 (revising the fuses) was therefore needed: F632 to 500E; F633, F646, F-DL and F671-1 to 400E; F692-R to 250E (Table 4.3).

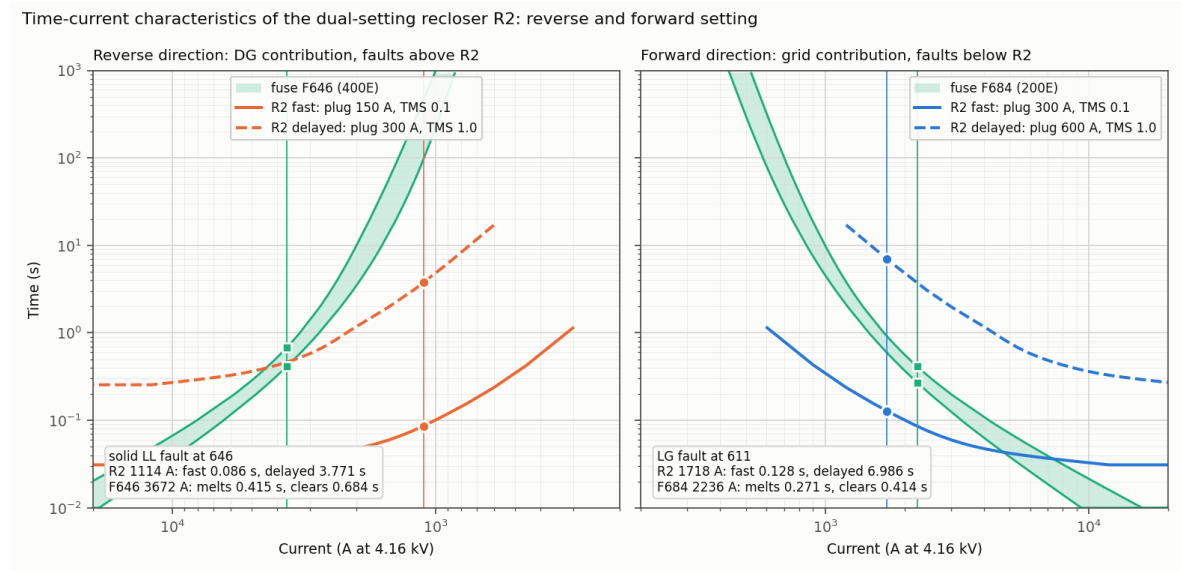


Figure 5.7 The two setting groups of R2: reverse (DG contribution, faults towards 632) and forward (grid contribution, faults beyond 671), each with one fault of the model.

5.9 Coordination With the DSDR

With R2 as a DSDR and the revised fuses, coordination is held in 38 of 39 cells (Table 5.7, Figure 5.8). With the same revised fuses but a single-setting R2 only 31 cells are held. The dual setting itself therefore restores seven cells: all four fault types at 633, and the LG faults at 645, 646 and DL.

The cell that remains lost is the LG fault at 692. F671-1 must be 400E to stay selective with F692-R, but it then clears after R2's delayed trip (7.6 s against 4.0 s), and R2's delayed dial is already at its maximum.

Fault, R2	632	633	645	646	DL	671	692	675	680	684	611	652
LG, single setting	✓	×	×	×	×	✓	×	✓	✓	✓	✓	✓
LG, dual setting	✓	✓	✓	✓	✓	✓	×	✓	✓	✓	✓	✓
LL, single setting	✓	×	✓	✓	✓	✓	✓	✓	✓	✓	–	–
LL, dual setting	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	–	–
LLG, single setting	✓	×	✓	✓	✓	✓	✓	✓	✓	✓	–	–
LLG, dual setting	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	–	–
LLL, single setting	✓	×	–	–	✓	✓	✓	✓	✓	–	–	–
LLL, dual setting	✓	✓	–	–	✓	✓	✓	✓	✓	–	–	–

Table 5.7 Coordination status with DG and the revised fuses: single-setting R2 against dual-setting R2.

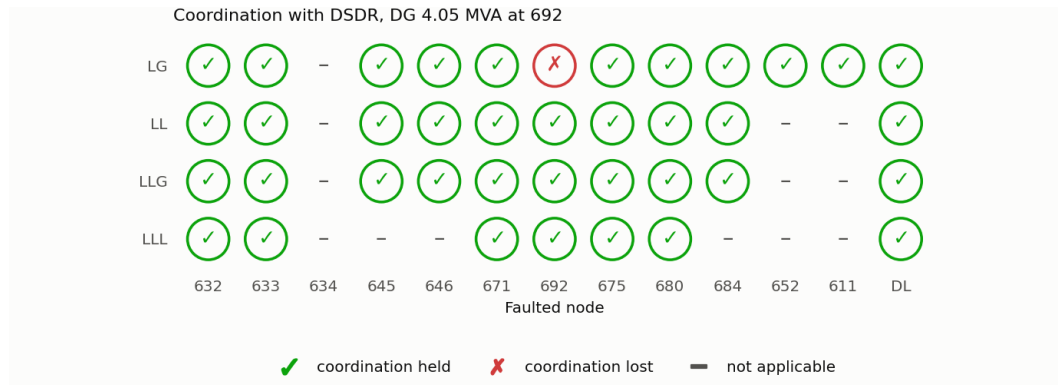


Figure 5.8 Coordination status with DG and the DSDR.

Figures 5.9 and 5.10 show a bolted LL fault at 646. The fuses carry 3.67 kA. R2 carries 1.11 kA in reverse and trips on its reverse fast curve in 0.086 s, before F646 melts (0.415 s).

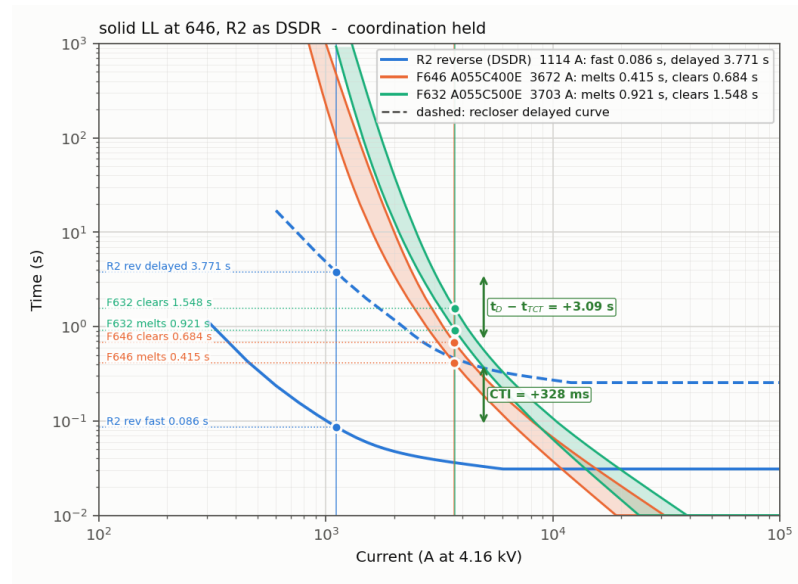


Figure 5.9 Bolted LL fault at 646: R2 reverse setting with F646 and F632; CTI = +328 ms.

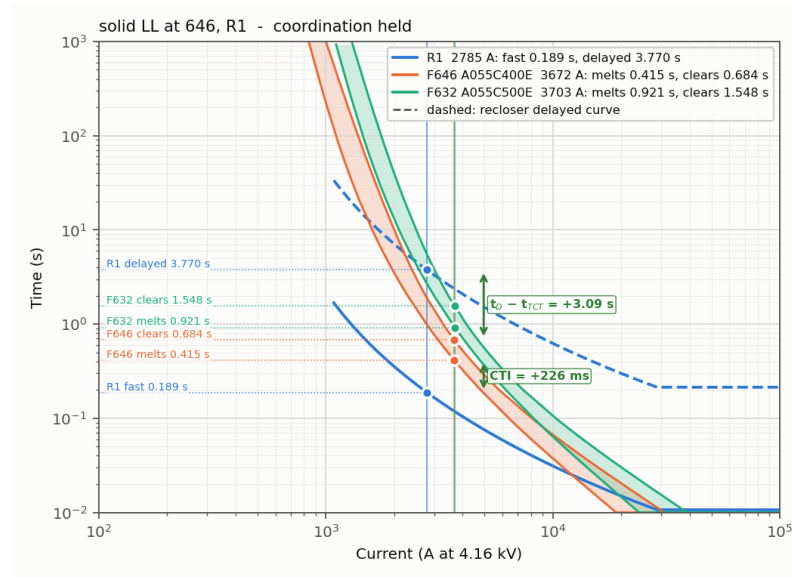


Figure 5.10 Bolted LL fault at 646: R1 with F646 and F632; CTI = +226 ms.

5.10 Operating Times With the DSDR

Tables 5.8 and 5.9 give the operating times with the DSDR. R1 operates forward for every node. R2 operates in reverse for the nodes upstream of it and forward for the nodes downstream. The fuse time is the minimum-melting time of the fuse nearest to the fault at the current through that fuse. In all nine rows with a fuse it is longer than the last fast trip; the smallest margins are 6 ms at 675 and 15 ms at DL and 652.

Node	Fault	I_{R1} (A)	R1 fast (s)	R1 delayed (s)	I_{R2} (A)	Dir.	R2 fast (s)	R2 delayed (s)
632	LLL	4734	0.081	1.627	1813	rev	0.051	1.416
633	LLL	3970	0.106	2.111	1500	rev	0.060	2.001
645	LL	3124	0.155	3.095	1232	rev	0.075	3.002
646	LL	2785	0.189	3.770	1114	rev	0.086	3.771
DL	LLL	3954	0.106	2.123	1972	rev	0.048	1.213
671	LLL	3412	0.134	2.677	3391	fwd	0.054	1.600
692	LLL	3412	0.134	2.677	3391	fwd	0.054	1.600
675	LLL	3090	0.158	3.152	3050	fwd	0.059	1.941
684	LL	2543	0.222	4.437	2543	fwd	0.072	2.813
680	LLL	2789	0.188	3.761	2757	fwd	0.066	2.381
611	LG	1814	0.435	8.709	1718	fwd	0.128	6.986
652	LG	1795	0.446	8.919	1761	fwd	0.123	6.570

Table 5.8 Recloser operating times with the DSDR, DG in service.

Node	Fault	Fuse	Size	I_{fuse} (A)	t_{MMT} (s)	t_{TCT} (s)	Last fast trip (s)	Margin (s)	Status
633	LLL	F633	400E	5426	0.149	0.235	0.106	+0.043	held
645	LL	F632	500E	4191	0.613	0.970	0.155	+0.458	held
646	LL	F646	400E	3672	0.415	0.684	0.189	+0.226	held
DL	LLL	F-DL	400E	5902	0.121	0.193	0.106	+0.015	held
692	LLL	F671-1	400E	3391	0.525	0.891	0.054	+0.471	held
675	LLL	F692-R	250E	5022	0.065	0.106	0.059	+0.006	held
684	LL	F671-2	300E	3963	0.148	0.230	0.072	+0.076	held
611	LG	F684	200E	2236	0.271	0.414	0.128	+0.143	held
652	LG	F652	150E	2239	0.138	0.193	0.123	+0.015	held

Table 5.9 Fuse times with the DSDR, DG in service. Margin = t_{MMT} minus the last fast trip (R2 for faults below R2; the later of R1 and R2 reverse above it). Nodes 632, 671 and 680 are on the main feeder and have no fuse.

5.11 Single Setting Against Dual Setting on Selected Faults

Table 5.10 compares the two settings of R2 on nine faults with the same (revised) fuses: the six faults of the figures above and three cells of the classification. Figures 5.11 and 5.12 show two of the diagrams.

#	Figure	Fault	Z_f (Ω)	I_{R1} (A)	I_{R2} (A), direction	R1 fast (s)	R2 fast (s)		Fuse: melts (s)	Fuse saving	
							single	dual		single	dual
01	Fig. 8	LG fault at 611	0	1814	1718 (fwd)	0.435	0.128	0.128	F684 200E: 0.271	held	held
02	Fig. 9	LLG fault at the middle of 692-675	1	1445	1310 (fwd)	0.742	0.204	0.204	F692-R 250E: 0.618	held	held
03	Fig. 11	LL fault at 646	1	1899	847 (rev)	0.395	0.500	0.131	F646 400E: 1.734	lost	lost
04	Fig. 12	LL fault at 645	1.5	1694	783 (rev)	0.507	0.605	0.149	F632 500E: 9.129	lost	lost
05	Fig. 13	Three-phase fault at 10 % of 632-633	0	4645	1777 (rev)	0.084	0.121	0.052	F633 400E: 0.100	lost	held
06	Figs. 15, 16	LL fault at 646	0	2785	1114 (rev)	0.189	0.278	0.086	F646 400E: 0.415	held	held
07	Fig. 14 / 17 cell	LG fault at 646 (phase c)	0	2359	469 (rev)	0.255	no trip	0.396	F646 400E: 1.057	lost	held
08	Fig. 14 / 17 cell	LG fault at 633 (phase b)	0	2874	644 (rev)	0.178	0.974	0.210	F633 400E: 0.502	lost	held
09	Fig. 14 / 17 cell	LG fault at 692 (phase a)	0	2197	2182 (fwd)	0.293	0.089	0.089	F671-1 400E: 2.842	lost	lost

Table 5.10 Nine faults with the DG in service: single-setting against dual-setting R2, same fuses.

In cases 5, 7 and 8 the dual setting restores the fuse saving: the reverse group picks up a DG current that the forward group does not see (case 7: 469 A) or sees only slowly. Cases 3, 4 and 9 are lost with both settings; they are limits of the fuse revision and of the dial range, not of the direction. Cases 3 and 4 are faults through impedance: the larger fuses that are needed for the bolted faults are too slow for these lower currents. Cases 5 and 7 are shown in detail in Figures 5.11 and 5.12; Tables 5.11 and 5.12 list their currents, settings and operating times.

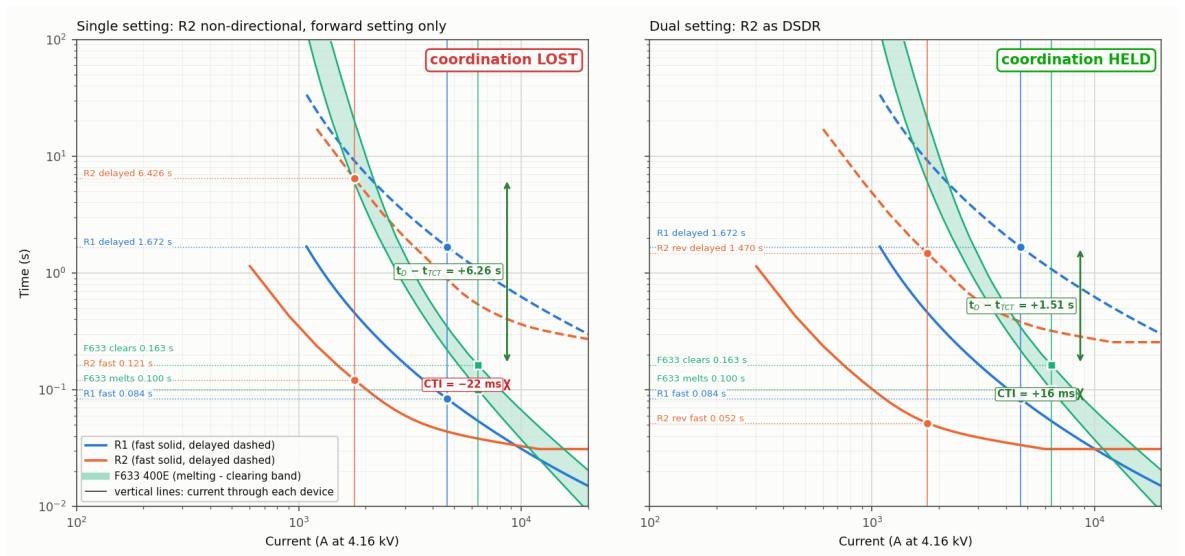


Figure 5.11 Three-phase fault at 10 per cent of line 632–633: single setting (left, CTI = –22 ms) and dual setting (right, CTI = +16 ms).

Quantity	Single setting	Dual setting (DSDR)
Fault	Three-phase fault at 10 % of 632-633, LLL, phases a-b-c	
Fault impedance Z_f	0 Ω (bolted)	
Fault-path current (F633)	6398 A	
Current through R1, direction	4645 A, forward (RG60 $\rightarrow$ 632), grid contribution	
Current through R2, direction	1777 A, REVERSE (671 $\rightarrow$ 632), DG contribution only	
R2 setting applied (fast / delayed)	FORWARD: plug 300/600 A, starts 600/1200 A, TMS 0.1/1.0	REVERSE: plug 150/300 A, starts 300/600 A, TMS 0.1/1.0
R1 fast / delayed	0.084 s / 1.672 s	0.084 s / 1.672 s
R2 fast / delayed	0.121 s / 6.426 s	0.052 s / 1.470 s
Primary fuse melts / clears	F633 400E: 0.100 s / 0.163 s	F633 400E: 0.100 s / 0.163 s
Backup fuse melts	-	-
Fuse saving	LOST: F633 melts (0.100 s) before R2 fast (0.121 s)	HELD

Table 5.11 Three-phase fault at 10 per cent of line 632–633: currents, settings and operating times.

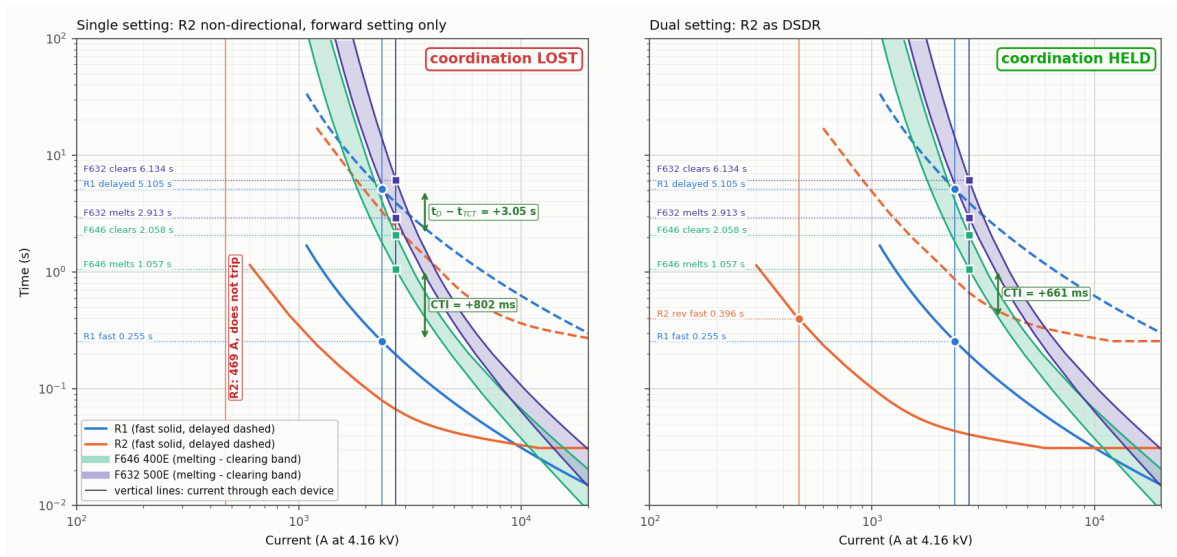


Figure 5.12 LG fault at 646: the single-setting R2 does not pick up the reverse current (left); the reverse group does (right, CTI = +661 ms).

Quantity	Single setting	Dual setting (DSDR)
Fault	LG fault at 646 (phase c), LG, phases C	
Fault impedance Z_f	0 Ω (bolted)	
Fault-path current (F646)	2732 A	
Current through R1, direction	2359 A, forward (RG60 $\rightarrow$ 632), grid contribution	
Current through R2, direction	469 A, REVERSE (671 $\rightarrow$ 632), DG contribution only	
R2 setting applied (fast / delayed)	FORWARD: plug 300/600 A, starts 600/1200 A, TMS 0.1/1.0	REVERSE: plug 150/300 A, starts 300/600 A, TMS 0.1/1.0
R1 fast / delayed	0.255 s / 5.105 s	0.255 s / 5.105 s
R2 fast / delayed	no trip / no trip	0.396 s / no trip
Primary fuse melts / clears	F646 400E: 1.057 s / 2.058 s	F646 400E: 1.057 s / 2.058 s
Backup fuse melts	F632 500E: 2.913 s	F632 500E: 2.913 s
Fuse saving	LOST: R2 does not pick up (469 A)	HELD

Table 5.12 LG fault at 646 (phase c): currents, settings and operating times.

5.12 Time-Sequence Check

The classification above uses the strict rule that every recloser must trip before the fuse starts to melt. A time-sequence check follows each fault in time instead: the fuse heats with the current that still flows after each recloser opens. For the 18 cells upstream of R2 the strict rule gives 11 cells held with the single setting and 18 with the dual setting; the time sequence gives 15 and 18 (Figure 5.13). The three cells that the DSDR restores in the time sequence as well are the LG faults at 633, 645 and 646, where the single-setting R2 never trips on a reverse current of about 0.5 to 0.6 kA. For the 21 cells downstream of R2 the DG at 692 feeds the fault directly, with no recloser in its path, and the result is the same for both settings.

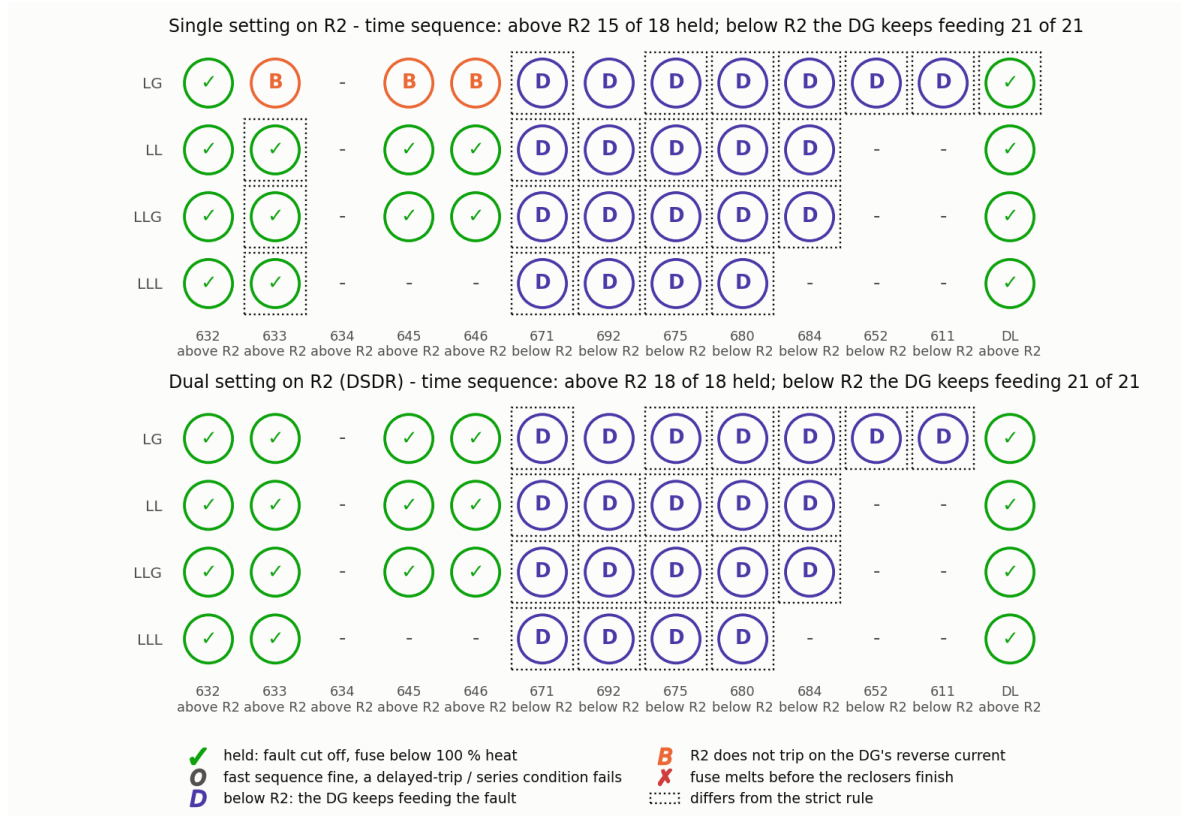


Figure 5.13 Time-sequence check of all 39 cells, single against dual setting.

5.13 Time-Domain Verification

Figure 5.14 is the EMT simulation of an LL (a–c) fault at 684 through 0.2Ω with the conventional settings. The recloser trips follow R2's fast curve applied to the simulated current, and the fuse F671-2 (300E) melts when the accumulated heating $\sum \Delta t / t_{MMT}(I)$ reaches one. Without DG the two fast shots use 45 per cent of the melting heat, the fuse survives both and then clears the permanent fault. With the DG the fuse melts at the end of the second fast shot, because the DG keeps feeding the fault while R2 is open (Table 5.13). The feeder current at node 632 is 557 / 588 A RMS (phases a / c) before the fault and 2362 / 2719 A during it, with peaks of 3338 / 3842 A.

Current at node 632, LL a-c fault at 684 through 0.2 ohm, PowerFactory EMT

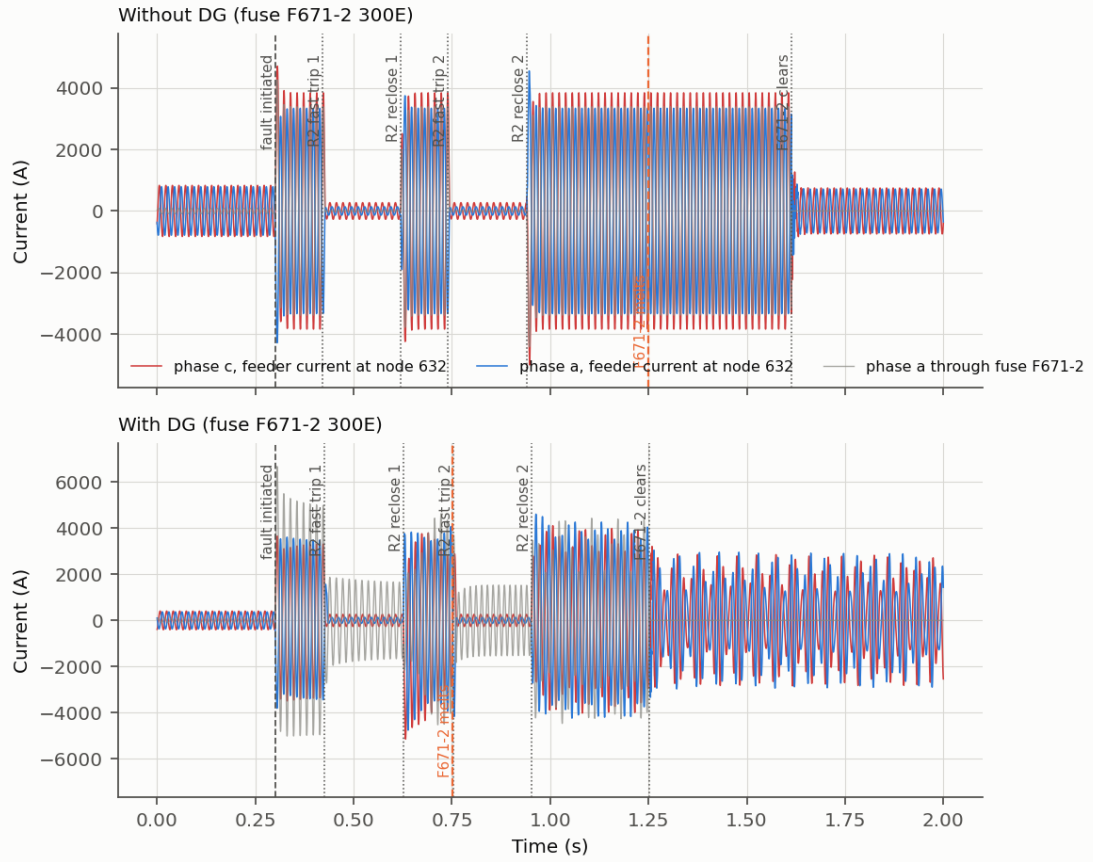


Figure 5.14 Time-domain current at node 632 for an LL (a–c) fault at 684 through 0.2Ω : without DG (top) and with DG (bottom).

Event (time in s)	DG out	DG in
Fault initiated	0.30	0.30
R2 fast trip 1	0.420	0.426
R2 reclose 1	0.620	0.626
R2 fast trip 2	0.740	0.753
R2 reclose 2	0.940	0.953
Fuse F671-2 melts	1.248	0.752
Fuse operation / clears	1.613	1.253
Fuse heat after the fast shots	45 %	105 %

Table 5.13 Reclosing sequence of the EMT simulation.

5.14 Verification With the Relay and Fuse Models of PowerFactory

The operating times of Chapter Five are calculated from the time–current curves of the devices. As a check, the settings were entered into the relay and fuse elements of the model, and PowerFactory itself calculated the tripping time of every device for every fault, for the conventional and for the DSDR settings.

5.15 Sensitivity of the Pickups With DG

The pickups are a further limit of the settings. With the DG, the lowest LG fault current through $3\ \Omega$ is 577 A at R1 and 368 A at R2, below the pickups of 734.6 A and 587.5 A, because the DG supplies part of the fault.

5.16 Effect of the DG Penetration Level

To see how the conventional coordination degrades as the DG grows, seven separate PowerFactory projects were made, copies of the model with the DG at 692 set to 0, 10, 25, 37, 50, 75 and 100 per cent of 4.05 MVA. The machine, its step-up transformer and its active and reactive power are scaled together, so the per-unit data of Table 4.1 stay the same; at 0 per cent the DG is out of service. Every model keeps the conventional protection of the design without DG (single-setting R2, fuses of the design without DG), and all operating times are PowerFactory’s own relay and fuse calculations. Two fuse–recloser pairs are followed: fuse F633 against R1 for a three-phase fault at 633 (R1’s zone), and the lateral fuse F671-2 at node 671 against R2 for a line-to-line fault at 684 (R2’s zone). The coordination time interval is

$$CTI = t_{MMT}(\text{fuse}) - t_F(\text{recloser}), \quad (5.1)$$

and, for the doughnut chart of Figure 5.15, each level’s share of the ring, $CTI\% = 100 \times CTI / \sum |CTI|$ over the seven levels, so that each ring adds up to 100 per cent.

DG (%)	MVA	Node 633: F633 and R1 (LLL)				Node 671: F671-2 and R2 (LL at 684)			
		I fuse / R1 (A)	t_{MMT} / t_F (s)	CTI (ms)	ring %	I fuse / R2 (A)	t_{MMT} / t_F (s)	CTI (ms)	ring %
0	0.00	4102 / 4162	0.102 / 0.098	+4	+2.1	2574 / 2711	0.470 / 0.067	+403	+26.0
10	0.41	4256 / 4142	0.094 / 0.099	–5	–3.1	2715 / 2681	0.401 / 0.068	+333	+21.5
25	1.01	4478 / 4111	0.083 / 0.100	–17	–9.6	2926 / 2636	0.323 / 0.069	+254	+16.4
37	1.50	4648 / 4088	0.077 / 0.101	–24	–13.9	3094 / 2598	0.277 / 0.070	+207	+13.4
50	2.02	4824 / 4062	0.071 / 0.102	–31	–17.9	3276 / 2557	0.237 / 0.072	+166	+10.7
75	3.04	5140 / 4015	0.062 / 0.104	–42	–24.2	3623 / 2511	0.184 / 0.073	+110	+7.1
100	4.05	5426 / 3970	0.055 / 0.106	–51	–29.2	3963 / 2543	0.147 / 0.072	+75	+4.9

Table 5.14 CTI against DG penetration, conventional protection.

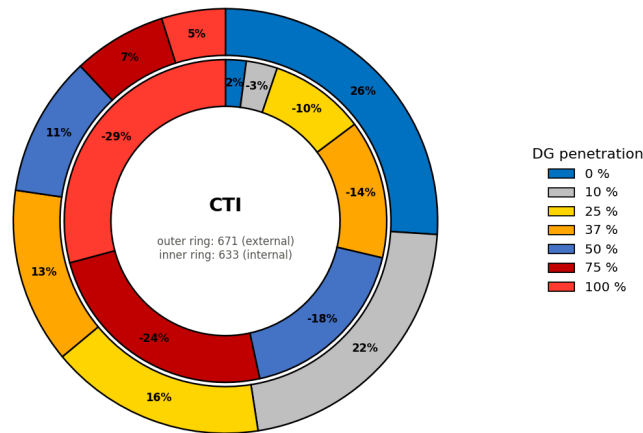


Figure 5.15 Percentage variation of the CTI with DG penetration (outer ring 671, inner ring 633).

At node 633 the fuse current rises from 4102 A to 5426 A while R1's current falls from 4162 A to 3970 A; the CTI falls from +4 ms to -51 ms. At node 671 the fuse current rises from 2574 A to 3963 A while R2's current stays at about 2543 A; the CTI falls from +403 ms to +75 ms. The mechanism is the same at both nodes (Figure 5.16): the fuse lies between the fault and both sources and carries the grid share and the DG share, while the recloser carries only the grid share, which even falls slightly because the DG holds up the voltage of the network during the fault. A fuse curve is steep, so the extra current shortens its melting time much more than the lower current lengthens the recloser's time, and the CTI decreases. At 633 the margin is only a few milliseconds without DG and is lost from about 4 per cent penetration; at 671 the margin is large and stays positive up to 100 per cent.

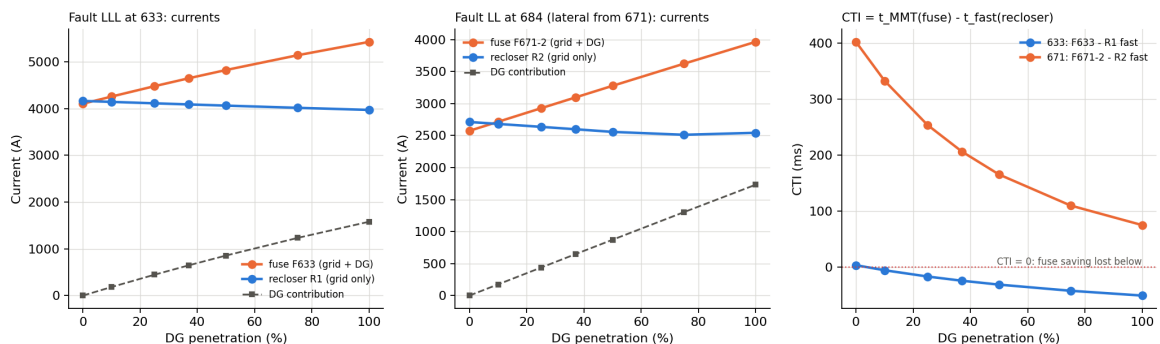


Figure 5.16 Currents through the fuse (grid and DG share), the recloser (grid share) and from the DG, and the resulting CTI, against DG penetration.

Figure 5.17 shows the same on the time–current characteristics: the curves are fixed because the settings are not changed, the recloser’s operating point stays where it is, and the fuse’s operating point moves to a higher current and a shorter time with every step. Figure 5.18 gives node 633 level by level, with the vertical short-circuit lines of the fuse and the recloser currents, the operating times and the CTI.

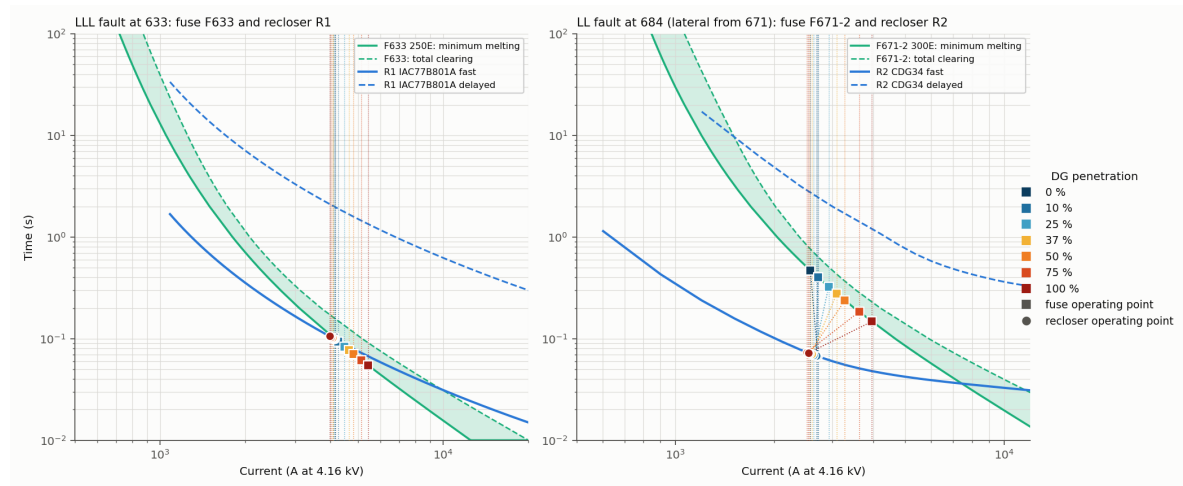


Figure 5.17 Time–current characteristics at all penetration levels: fuse operating points (squares) and recloser operating points (circles).

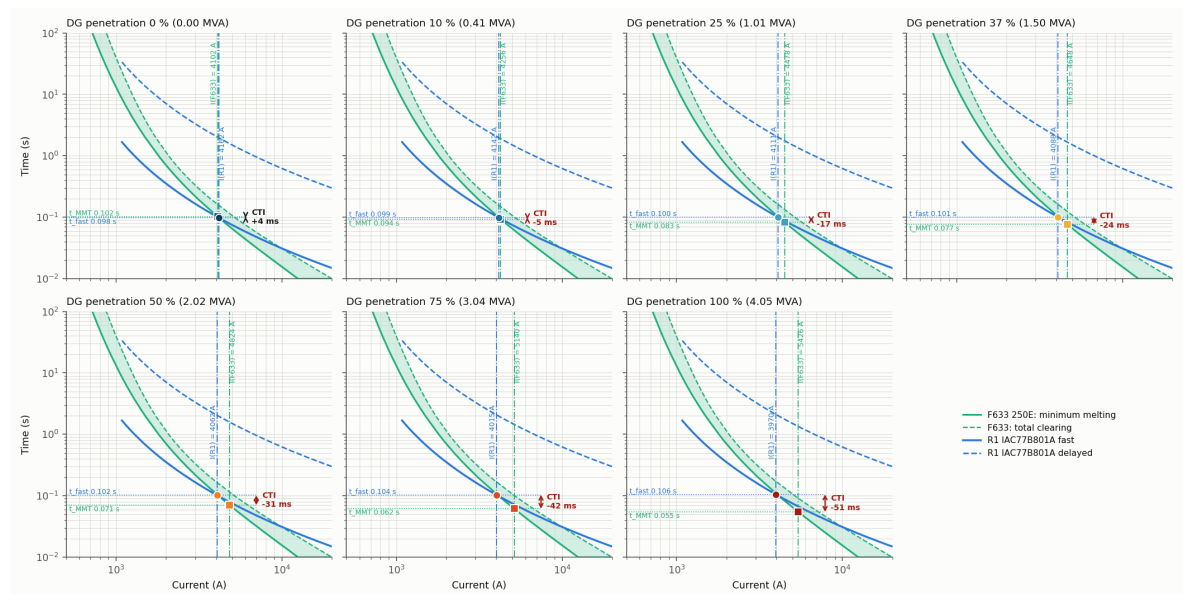


Figure 5.18 Node 633, fuse F633 and R1, at each penetration level, with the short-circuit currents, operating times and CTI.

The fault level rises at every node (Table 5.15), most near the DG (671, 692, 675: about +60 per cent) and least behind the 4.16/0.48 kV transformer (634) and on the single-phase laterals. The current through the reclosers does not rise with it.

Node	Fault	0 %	25 %	50 %	75 %	100 %	Change
632	LLL ABC	4735	5227	5690	6123	6526	+38 %
633	LLL ABC	4102	4478	4824	5140	5426	+32 %
634	LLL ABC	15671	16307	16828	17257	17608	+12 %
645	LLG BC	3418	3685	3932	4149	4341	+27 %
646	LLG BC	3112	3332	3528	3701	3854	+24 %
671	LLL ABC	3390	3919	4461	5010	5564	+64 %
692	LLL ABC	3390	3919	4461	5010	5564	+64 %
675	LLL ABC	3151	3612	4087	4557	5022	+59 %
680	LLL ABC	2930	3322	3711	4091	4460	+52 %
684	LLG AC	2655	3027	3395	3756	4107	+55 %
652	LG A	1874	1983	2080	2164	2239	+19 %
611	LG C	1857	1973	2074	2161	2236	+20 %
DL	LLL ABC	3946	4454	4952	5436	5902	+50 %

Table 5.15 Largest bolted fault current at each node (A) against DG penetration.

The load flow changes as well (Table 5.16). The DG supplies part of the load locally, so the load current through both reclosers falls, the voltages rise, and the power through R2 reverses between 75 and 100 per cent. By eq. (3.3) the pickups would have to fall with the penetration; with the fixed settings (R1 720 A, its curve starting at 1080 A; R2 fast curve starting at 600 A) the reclosers no longer detect the weakest faults (LG through 3 Ω) in their zones, R2 from just above 50 per cent.

DG (%)	DG P (MW)	R1 load (A)	R2 load (A)	R2 flow	$V_{671,a}$ (p.u.)	R1 1.25 I_{nom} (A)	R2 1.25 I_{nom} (A)	R1 lowest 3 Ω fault (A)	R2 lowest 3 Ω fault (A)
0	0.00	587.7	470.0	forward	0.988	734.6	587.5	1074*	878
10	0.32	543.1	430.1	forward	0.994	678.9	537.6	1018*	821
25	0.81	480.7	372.2	forward	1.002	600.9	465.2	937*	738
37	1.20	434.8	328.6	forward	1.009	543.5	410.8	874*	673
50	1.62	389.9	285.3	forward	1.015	487.4	356.6	809*	606
75	2.43	321.2	222.4	forward	1.025	401.5	278.0	689*	483*
100	3.24	284.9	252.2	reverse	1.033	356.1	315.2	577*	368*

Table 5.16 Load flow, pickups by eq. (3.3) and lowest fault current through each recloser against DG penetration. *: below the start of the recloser's curve with the fixed settings (not detected).

The penetration study therefore shows the two ways in which fixed settings fail as the DG grows: the fuse saving is lost because the fuse speeds up while the recloser does not, and the sensitivity is lost because the currents through the reclosers fall. Both are reasons for settings that adapt to the DG, which is what the dual-setting directional recloser provides.

5.17 Results Summary

Stage	Cells held (of 39)
A. No DG, starting fuse sizes	35
A. No DG, after the fuse revision (F692-R to 200E)	39
B. DG connected, conventional R2	24
B. DG connected, no-DG fuses, R2 as DSDR	25
C. DG connected, revised fuses, conventional R2	31
C. DG connected, revised fuses, R2 as DSDR	38
C. The same settings and fuses with the DG out of service	37

Table 5.17 Summary of the coordination results.

The dual setting alone, with the fuses of the design without DG, restores one cell (24 to 25). Together with the fuse revision it restores fourteen (24 to 38), seven of which need the dual setting.

CHAPTER SIX: DISCUSSION

6.1 Effect of DG on Fault Current

The DG raises the fault current at every node, most near its connection point (671 and 692: from 3.39 to 5.56 kA). The fuses carry this higher current and melt faster, while R1 carries about the same grid current as before. The margin between the fuse and the fast trip, which is small even without DG, disappears.

6.2 Effect of Fault Current Direction

For faults at 632, 633, 645, 646 and DL the current through R2 is reverse and consists only of the DG contribution. Its magnitude (0.5 to 2.0 kA) is much lower than the forward fault currents for which the forward group is set. A forward group with plugs of 300 and 600 A responds slowly to these currents, or not at all for LG faults.

6.3 Performance of the Conventional Protection

With the conventional single setting the scheme keeps coordination at the nodes on the main feeder and loses it on the fused laterals of R1's zone and at 675. The loss depends on the location and on the fault type; it is not a uniform failure of the feeder.

6.4 Performance of the DSDR

The reverse group, with half the plug settings, trips for every reverse fault before the fuse melts once the fuses are revised. The result of 38 cells against 31 with the same fuses isolates its contribution. Without the fuse revision the improvement is one cell only: at 633 and DL the fuses of the design without DG melt in 0.05 to 0.16 s with the DG, before R1's fast trip (0.11 to 0.18 s), and no setting of R2 can change R1's time. The DSDR is therefore necessary but not sufficient.

6.5 Limits Reached by the Devices

R1's dials are the two extremes of the IAC range, and the CDG table ends at 0.1 and 1.0. The method's dial revision has no room. The fuse revision creates a conflict between the two states of the network: the larger fuses that are needed with the DG are too slow for two cells without it, and for the faults through impedance. Fuse sizes cannot adapt to the

DG state. For faults downstream of R2 the fuse saving relies on the DG's own protection disconnecting it during the dead time, which is not part of the method.

6.6 Engineering Implication

Adaptive directional protection is justified where a DG is downstream of a mid-line recloser. To apply it, the engineer must check the dial range of the actual relay, revise the fuse sizes for the DG state, verify the pickup against the reduced minimum fault current with DG, and provide for the DG infeed to faults beyond the recloser.

CHAPTER SEVEN: CONCLUSION AND FUTURE WORK

7.1 Conclusion

The DSDR method of [1] was implemented on the IEEE 13-node feeder in DIgSI-LENT PowerFactory with the manufacturer-based time–current characteristics of the protection devices. The feeder model agrees with the IEEE short-circuit benchmark within about 2 per cent. Without DG, the revised fuse scheme achieves coordination in all 39 cells, compared with 35 of 39 cells with the starting fuse sizes. With the 4.05 MVA DG at node 692 and a conventional R2, coordination is held in 24 cells. The dual setting alone, with the fuses of the design without DG, raises this to 25 cells; the fuse revision alone, with a single-setting R2, to 31; and the two together to 38 of 39 cells. The combined DSDR and fuse-revision strategy therefore substantially improves the coordination in the zone between the grid and the DG, achieving 38 of 39 coordinated fault cells under the zero-margin criterion, and seven of the restored cells need the dual setting. The remaining LG fault at node 692 is limited by the maximum available R2 delayed dial.

The faults downstream of R2 with the DG in service remain outside what the dual setting can correct.

7.2 Major Findings

1. Fault levels at the IEEE 13-node benchmark (feeder head 4.73 kA).
2. R1: pickup 720 A, TDS 0.5 / 10, GE IAC extremely inverse curve.
3. R2 forward: plugs 300 / 600 A, TMS 0.1 / 1.0; R2 reverse: plugs 150 / 300 A, TMS 0.1 / 1.0; reverse pickup by eq. (3.12) 315.3 A.
4. With DG the load current at R2 reverses (252 A) and R2 carries 1.1 to 2.0 kA in reverse for upstream faults.
5. Coordination (of 39 cells): 35 without DG with the starting fuses and 39 after the fuse revision; 24 with DG and a conventional R2; 25 with the DSDR alone; 31 with the fuse revision alone; 38 with the DSDR and the fuse revision, under the zero-margin criterion.
6. The dual setting restores seven cells with the revised fuses and one cell without the revision.
7. The LG fault at 692 cannot be coordinated within the delayed dial range of R2.

8. With rising DG penetration and fixed settings the CTI falls at both studied nodes (633: from about +4 to −51 ms; 671: from +403 to +75 ms), the fault level rises by up to 64 per cent, and R2 stops detecting the weakest fault of its zone above about 50 per cent.
9. In the EMT simulation the fuse survives the two fast shots without DG and melts at 1.25 s; with DG it melts during the second fast shot.

7.3 Contributions of the Present Work

- A reproducible PowerFactory implementation of the method: one command rebuilds the model and all results.
- Separation of the effect of the dial setting from the effect of the fuse revision.
- Identification of the limits of the method: dial range, DG-in against DG-out fuse sizes, DG infeed downstream of the recloser, and sensitivity of the pickups with DG.
- A time-domain verification and a verification with the relay models of the tool.

7.4 Future Work

Future work may include the extension of the study to the IEEE 34-node test feeder, an explicit directional element for R2, a recloser with a wider dial range or with user-defined curves, a setting-group change triggered by the DG state, the disconnection logic of the DG during the reclosing dead time, the sensitivity to fault impedance, inverter-based generation, and hardware-in-the-loop tests.

REFERENCES

- [1] M. Yousaf, A. Jalilian, K. M. Muttaqi, and D. Sutanto, “An adaptive overcurrent protection scheme for dual-setting directional recloser and fuse coordination in unbalanced distribution networks with distributed generation,” *IEEE Transactions on Industry Applications*, vol. 58, no. 2, pp. 1831–1842, Mar./Apr. 2022, doi: 10.1109/TIA.2022.3146095.
- [2] G. Benmouyal *et al.*, “IEEE standard inverse-time characteristic equations for overcurrent relays,” *IEEE Transactions on Power Delivery*, vol. 14, no. 3, pp. 868–872, Jul. 1999.
- [3] A. Naiem, Y. Hegazy, A. Abdelaziz, and M. Elsharkawy, “A classification technique for recloser-fuse coordination in distribution systems with distributed generation,” *IEEE Transactions on Power Delivery*, vol. 27, no. 1, pp. 176–185, Jan. 2012.
- [4] GE Multilin, *735/737 Feeder Protection Relay – Instruction Manual*, manual P/N 1601-0048-DK (GEK-106291F), Markham, ON, Canada, 2010, Ch. 5: Overcurrent Curves (IAC curves).
- [5] W. H. Kersting, “Radial distribution test feeders,” *IEEE Transactions on Power Systems*, vol. 6, no. 3, pp. 975–985, Aug. 1991.
- [6] M. Yousaf, K. M. Muttaqi, and D. Sutanto, “Overcurrent protection scheme for the IEEE 13-node benchmark test feeder with improved selectivity,” in *Proc. IEEE Power & Energy Society General Meeting*, 2020, doi: 10.1109/PESGM41954.2020.9281529.
- [7] W. H. Kersting and G. Shirek, “Short circuit analysis of IEEE test feeders,” in *Proc. IEEE PES Transmission and Distribution Conference and Exposition*, 2012.
- [8] DIgSILENT GmbH, *PowerFactory 2021 User Manual*, Gomaringen, Germany, 2021.

APPENDIX A: IMPLEMENTATION FORMULAS

The operating times are evaluated with the following expressions, where $M = I_f/I_p$ for R1 and $M = I_f/I_s$ for R2.

$$\text{R1 (IAC): } t = TDS \left[0.004 + \frac{0.6379}{M - 0.62} + \frac{1.7872}{(M - 0.62)^2} + \frac{0.2461}{(M - 0.62)^3} \right] \quad (\text{A.1})$$

$$\text{R2 (CDG table): } t = t_1 \left(\frac{t_2}{t_1} \right)^f, \quad f = \frac{\ln(M/M_1)}{\ln(M_2/M_1)} \quad (\text{A.2})$$

$$\text{Fuse (curve points): } t = t_1 \left(\frac{t_2}{t_1} \right)^f, \quad f = \frac{\ln(I/I_1)}{\ln(I_2/I_1)} \quad (\text{A.3})$$

$$\text{Fuse saving: } t_{MMT}(I_{fuse}) > t_F, \quad t_{TCT}(I_{fuse}) < t_D \quad (\text{A.4})$$

$$\text{Series fuses: } t_{TCT,i} \leq 0.75 t_{MMT,i+1} \quad (\text{A.5})$$

$$\text{Fuse heating (EMT): } \sum_j \frac{\Delta t_j}{t_{MMT}(I_j)} = 1 \Rightarrow \text{the fuse melts} \quad (\text{A.6})$$

Worked example, node 633, three-phase fault, DG in service. R1 carries 3970 A: $M = 3970/720 = 5.514$, $M - 0.62 = 4.894$, bracket $= 0.004 + 0.1303 + 0.0746 + 0.0021 = 0.2111$, so $t_F = 0.5 \times 0.2111 = 0.106$ s and $t_D = 10 \times 0.2111 = 2.111$ s. R2 carries 1500 A in reverse: fast, $M = 1500/150 = 10.0$, table 0.060 s; delayed, $M = 1500/300 = 5.0$, table 2.0 s. The fuse F633 (400E) carries 5426 A, the sum of both contributions: its curve points are 4800 A at 0.2 s and 6392 A at 0.1 s, $f = \ln(5426/4800)/\ln(6392/4800) = 0.428$, $t_{MMT} = 0.2 \times 0.5^{0.428} = 0.149$ s. The fuse outlasts the last fast trip (0.106 s) by 0.043 s.

APPENDIX B: COMPARISON WITH THE REFERENCE PAPER

This appendix compares the results of Chapter Five with the values printed in the reference paper [1] or read from its figures.

Basis of Comparison

The comparison uses only values that the paper prints or that can be read from its figures. The model was not tuned to match the paper; differences are stated and explained from the evidence in the model.

Aspect	This project	Reference paper	Assessment
Rated branch currents	within 2 %	Table II	Very good
Fault levels	IEEE benchmark, feeder head 4.73 kA	5.41 kA, some branches higher	Lower
R1 worked example (4219 A)	0.096 / 1.926 s	0.097 / 1.932 s	Very good
Recloser curves	IAC77B801A and CDG34 library curves	Same devices	Same
Fuse coefficients, series fuses of R1's zone	within 0.05 with reversed index	Table III	Index reversed in the paper
DG model	Dynamic synchronous machine, Table I data	Synchronous machine	Same
Coordination with DG, conventional R2	24 of 39 held; 29 cells agree	30 of 39 held (Fig. 14)	Good
Coordination with the DSDR	38 of 39 held	39 of 39 held (Fig. 17)	Good; one cell at the dial limit
Fuse current, LL at 646	3.67 kA	3.64 kA (Fig. 15)	Very good
R2 reverse current, LL at 646	1.11 kA	1.54 kA	Lower
Table IV, R1 times	12 to 78 % slower (652: 34 % faster)	Table IV	Follows the fault levels
Table IV, R2 and fuse times	assumed R2 settings, revised fuses	settings not published	Not comparable in detail
EMT sequence, fuse operation	melts 1.25 s, clears 1.61 s	1.21 s (Fig. 10)	Close if 1.21 s is melting
IEEE 34-node feeder	not modelled	validated	Remaining work

Table B.1 Comparison of the results with the reference paper.

Base-Case Comparison

The rated currents test the feeder model independently of the protection and of the DG. Their agreement within 2 per cent shows that the topology, the loads and the regulator are implemented as in the paper.

Fault-Current Magnitude Difference

The maximum fault currents of the feeder branches are 4 to 60 per cent below the paper's Table II. The model was validated against the published IEEE short-circuit results instead, which it meets within about 2 per cent. The paper's values cannot all be correct: 6.73 kA on 632–633 and 7.83 kA on 692–675 exceed its own 5.41 kA at the feeder head. The two worked cases of the paper illustrate the remaining difference: R2 carries 1718 A for the LG fault at 611 (paper: 1974 A) and 1114 A in reverse for the LL fault at 646 (paper: 1542 A).

Fuse and Coordination Comparison

The paper's Table III could be reproduced only with the series index reversed. The fuse times in the paper's figures do not follow its fuse equation: for F646 at 3.6 kA the figure shows about 0.18 s, the equation with Table III gives 1.85 s. The figures match A055C library fuses, and the sizes change between figures (F646 is 200E in Fig. 12 and 300E in Figs. 11 and 16). The model therefore uses library fuse curves throughout.

Operating-Time Comparison (Table IV)

The R1 equation and settings reproduce the paper's worked example, so the difference of the R1 times comes from the fault currents. In every row of the paper the delayed time of R1 is 20 times the fast time, the ratio of the dials 10 and 0.5, which confirms the dials. For R2 the paper prints no settings and the ratio of its two times is not constant, so the R2 times and the fuse times of the model are the result of the stated method and cannot be matched to the paper row by row.

Central Finding Comparison

The central finding of the paper is reproduced: the DG makes a conventional scheme lose coordination in the zone between the grid and the DG, and a dual-setting directional R2

restores it there. The model adds two qualifications. The dual setting needs the fuse revision to be effective; and it does not help for faults downstream of R2, which the DG feeds directly.

Tables of the comparison

Parameter	Symbol	Paper	Model	Unit
Leakage reactance	X_l	0.05	0.05	pu
Stator resistance	R_a	0.0014	0.0014	pu
d-axis synchronous reactance	X_d	1.4	1.4	pu
d-axis transient reactance	X'_d	0.231	0.231	pu
d-axis subtransient reactance	X''_d	0.118	0.118	pu
d-axis transient open-circuit time constant	T'_{d0}	5.5	5.5	s
d-axis subtransient open-circuit time constant	T''_{d0}	0.05	0.05	s
q-axis synchronous reactance	X_q	1.372	1.372	pu
q-axis transient reactance	X'_q	0.8	0.8	pu
q-axis subtransient reactance	X''_q	0.118	0.118	pu
q-axis transient open-circuit time constant	T'_{q0}	1.25	1.25	s
q-axis subtransient open-circuit time constant	T''_{q0}	0.19	0.19	s
Mechanical starting time	$M = 2H$	1.5	1.5	s
DG rating	S_n	4.05	4.05	MVA
DG voltage	U_n	0.69	0.69	kV
Step-up transformer, HV side	U_{HV}	4.16	4.16	kV
Step-up transformer leakage reactance	x_T	0.15	0.15	pu

Table B.2 DG parameters: reference paper (Table I) against the PowerFactory model.

Branch	I_{nom} (A)			$I_{f,min}$ (kA)		$I_{f,max}$ (kA)		
	model	paper	dev. %	model	paper	model	paper	dev. %
RG60–632	587.7	587.1	+0.1	1.07	1.24	4.73	5.41	–13
632–633	81.0	81.1	–0.1	0.76	0.72	4.1	6.73	–39
632–645	143.1	143.3	–0.1	0.73	0.93	3.42	4.25	–20
632–671	470.0	478.2	–1.7	0.88	1.08	3.39	4.52	–25
XFM1-HV side	81.0	81.1	–0.1	0.08	0.73	1.81	6.48	–72
XFM1-LV side	706.4	704.7	+0.2	0.71	0.59	15.67	18.76	–16
645–646	66.1	64.9	+1.8	0.73	0.84	3.11	3.64	–15
671–692	229.7	229.3	+0.2	0.74	0.78	3.39	3.69	–8
671–684	71.1	70.9	+0.3	0.66	0.69	2.66	3.47	–23
671–680	0.0	0.0	–	0.64	0.59	2.93	6.25	–53
692–675	205.9	205.9	+0.0	0.74	0.71	3.15	7.83	–60
684–611	71.1	70.8	+0.4	0.68	0.63	1.86	1.93	–4
684–652	63.0	63.1	–0.2	0.66	0.66	1.87	2.67	–30

Table B.3 Branch currents: model against Table II of the paper. XFM1 HV side: the model values are 4.16 kV currents for a fault on the 0.48 kV side, so this row is not comparable.

The maximum fault currents of the model are lower than those of the paper's Table II. To decide which values are right, both are compared in Table B.4 with the IEEE 13-node short-circuit benchmark [7], as quoted by the same authors in [6]. The model agrees with the benchmark within 2 per cent at 10 of the 11 nodes (largest difference 4.7 per cent), while the paper's values lie 4 to 151 per cent above it. Two of the paper's values, 6.73 kA on 632–633 and 7.83 kA on 692–675, even exceed its own 5.41 kA at the feeder head, which is not possible in a radial feeder without DG. The lower fault levels of this study are therefore those of the IEEE benchmark, not an error of the model.

Node	IEEE benchmark	This study		Reference paper	
	(kA)	If,max (kA)	dev. %	If,max of branch (kA)	dev. %
632	4.80	4.74	−1.3	5.41	+12.7
633	4.15	4.10	−1.1	6.73	+62.2
645	3.41	3.42	+0.2	4.25	+24.6
646	3.05	3.11	+2.0	3.64	+19.3
671	3.35	3.39	+1.2	4.52	+34.9
692	3.35	3.39	+1.2	3.69	+10.1
675	3.12	3.15	+1.0	7.83	+151.0
680	2.91	2.93	+0.7	6.25	+114.8
684	2.62	2.66	+1.3	3.47	+32.4
652	1.79	1.87	+4.7	2.67	+49.2
611	1.85	1.86	+0.4	1.93	+4.3

Table B.4 Maximum fault current, DG out: IEEE benchmark, this study and the paper’s Table II. Deviations are relative to the benchmark.

Fuse	I_f (A)	i/z	t_F (s)	t_D (s)	t_{fuse} (s)	b_i	b_i rev.	paper	Size	t_{MMT} (s)
F632	3418	2/2	0.133	2.670	1.824	6.62	6.35	6.38	400E	0.512
F633	4102	1/1	0.100	2.008	1.054	6.53	6.53	6.83	250E	0.102
F634	15671	1/2	0.439	8.772	3.217	8.06	8.33	8.32	gL-800A	0.021
F645	3418	1/2	0.133	2.670	0.979	6.35	6.62	6.65	250E	0.158
F646	3112	1/2	0.156	3.115	1.142	6.35	6.62	6.67	300E	0.273
F-DL	3946	1/1	0.106	2.130	1.118	6.52	6.52	6.54	250E	0.112
F671	3390	1/1	0.054	1.601	0.827	6.27	6.27	6.08	250E	0.161
F671-1	3390	3/3	0.054	1.601	1.214	6.44	6.00	5.85	300E	0.218
F692	3390	1/2	0.054	1.601	0.569	6.11	6.39	6.13	200E	0.094
F692-R	3151	2/3	0.057	1.829	0.943	6.27	6.27	6.14	200E	0.112
F675	3151	1/3	0.057	1.829	0.500	6.00	6.44	6.19	200E	0.112
F671-2	2655	3/3	0.069	2.572	1.946	6.45	6.01	5.87	300E	0.428
F684	1857	2/3	0.113	5.778	2.946	6.35	6.35	6.15	200E	0.465
F611	1857	1/3	0.113	5.778	1.530	6.07	6.52	6.19	125E	0.120
F652	1874	1/2	0.112	5.660	1.961	6.18	6.47	6.17	150E	0.207

Table B.5 Fuse coefficients against Table III of the paper. “ b_i rev.”: index i counted from the source, which reproduces the paper’s values in R1’s zone.

Node	Fault	I_{R1} (A)	R1 fast (s) model / paper	R1 delayed (s) model / paper	I_{R2} (A)	Dir.	R2 fast (s) model / paper	R2 delayed (s) model / paper
632	LLL	4734	0.081 / 0.070	1.627 / 1.398	1813	rev	0.051 / 0.045	1.416 / 0.514
633	LLL	3970	0.106 / 0.094	2.111 / 1.880	1500	rev	0.060 / 0.052	2.001 / 0.706
645	LL	3124	0.155 / 0.105	3.095 / 2.095	1232	rev	0.075 / 0.087	3.002 / 1.659
646	LL	2785	0.189 / 0.128	3.770 / 2.555	1114	rev	0.086 / 0.103	3.771 / 2.046
DL	LLL	3954	0.106 / 0.087	2.123 / 1.733	1972	rev	0.048 / 0.076	1.213 / 1.402
671	LLL	3412	0.134 / 0.105	2.677 / 2.091	3391	fwd	0.054 / 0.068	1.600 / 1.113
692	LLL	3412	0.134 / 0.115	2.677 / 2.307	3391	fwd	0.054 / 0.072	1.600 / 1.258
675	LLL	3090	0.158 / 0.121	3.152 / 2.414	3050	fwd	0.059 / 0.074	1.941 / 1.332
684	LL	2543	0.222 / 0.125	4.437 / 2.507	2543	fwd	0.072 / 0.075	2.813 / 1.394
680	LLL	2789	0.188 / 0.137	3.761 / 2.747	2757	fwd	0.066 / 0.079	2.381 / 1.555
611	LG	1814	0.435 / 0.326	8.709 / 6.523	1718	fwd	0.128 / 0.146	6.986 / 4.385
652	LG	1795	0.446 / 0.676	8.919 / 13.525	1761	fwd	0.123 / 0.143	6.570 / 9.370

Table B.6 Recloser operating times against Table IV of the paper.

Node	Fault	Fuse	Size	I_{fuse} (A)	t_{MMT} (s)		t_{TCT} (s)	Last fast trip (s)	Margin (s)	Status
633	LLL	F633	400E	5426	0.149	0.163	0.235	0.106	+0.043	held
645	LL	F632	500E	4191	0.613	0.276	0.970	0.155	+0.458	held
646	LL	F646	400E	3672	0.415	0.181	0.684	0.189	+0.226	held
DL	LLL	F-DL	400E	5902	0.121	0.248	0.193	0.106	+0.015	held
692	LLL	F671-1	400E	3391	0.525	0.175	0.891	0.054	+0.471	held
675	LLL	F692-R	250E	5022	0.065	0.185	0.106	0.059	+0.006	held
684	LL	F671-2	300E	3963	0.148	0.205	0.230	0.072	+0.076	held
611	LG	F684	200E	2236	0.271	0.417	0.414	0.128	+0.143	held
652	LG	F652	150E	2239	0.138	0.233	0.193	0.123	+0.015	held

Table B.7 Fuse times against Table IV of the paper.

Fault	632	633	645	646	DL	671	692	675	680	684	611	652
LG (model)	✓	×	×	×	×	✓	✓	✓	✓	✓	✓	✓
LG (paper)	✓	✓	×	✓	✓	✓	✓	✓	✓	✓	✓	✓
LL (model)	✓	×	✓	×	×	✓	✓	×	✓	✓	–	–
LL (paper)	✓	✓	×	×	✓	✓	✓	✓	✓	✓	–	–
LLG (model)	✓	×	✓	×	×	✓	✓	×	✓	✓	–	–
LLG (paper)	✓	×	×	×	✓	✓	✓	×	✓	✓	–	–
LLL (model)	✓	×	–	–	×	✓	✓	×	✓	–	–	–
LLL (paper)	✓	×	–	–	✓	✓	✓	×	✓	–	–	–

Table B.8 Coordination status with DG and a conventional R2: model against the paper's Fig. 14.

Event (time in s)	Paper (read off Fig. 10)	Model, DG out	Model, DG in
Fault initiated	0.30	0.30	0.30
R2 fast trip 1	≈0.46	0.420	0.426
R2 reclose 1	≈0.65	0.620	0.626
R2 fast trip 2	≈0.80	0.740	0.753
R2 reclose 2	≈1.00	0.940	0.953
Fuse F671-2 melts	–	1.248	0.752
Fuse operation / clears	1.21	1.613	1.253
Fuse heat after the fast shots	–	45 %	105 %

Table B.9 Reclosing sequence of the EMT simulation against the paper's Fig. 10.

DG penetration (Fig. 7 of the paper)

The paper's Fig. 7 gives the CTI at nodes 633 and 671 as shares of a doughnut chart. The trend is the same in both: the CTI falls monotonically as the penetration rises. At 671 the shares of the first four levels are close to the paper's. The levels at which the CTI changes sign differ: in the paper both nodes cross zero at about 40–50 per cent; in this study 633 crosses at about 4 per cent and 671 does not cross up to 100 per cent. The crossing depends on the starting margin, which depends on fuse sizes and settings that the paper does not publish.

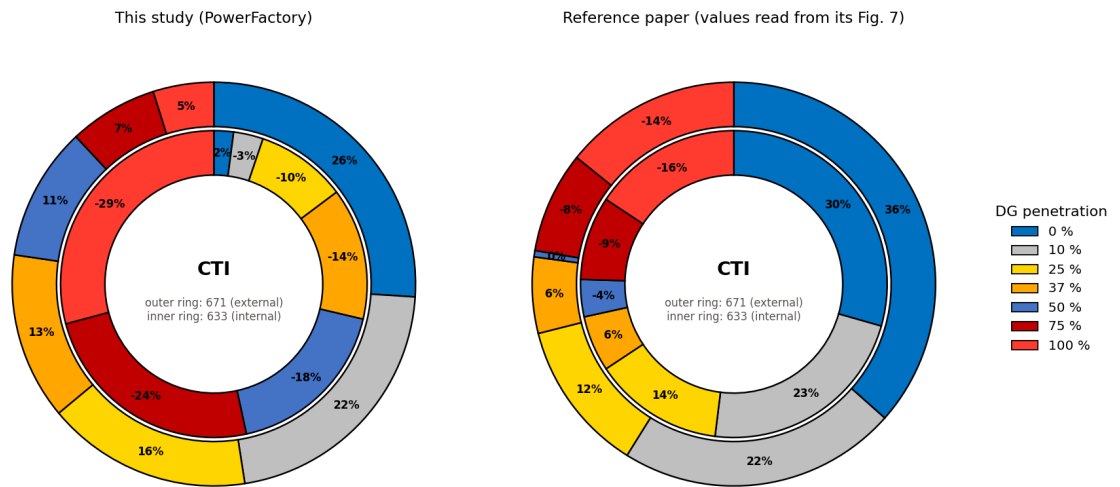


Figure B.1 CTI against DG penetration: this study (left) and the values read from the paper's Fig. 7 (right).

DG (%)	Node 633 (ring %)		Node 671 (ring %)	
	this study	paper	this study	paper
0	+2.1	+30	+26.0	+36
10	-3.1	+23	+21.5	+22
25	-9.6	+14	+16.4	+12
37	-13.9	+6	+13.4	+6
50	-17.9	-4	+10.7	+0
75	-24.2	-9	+7.1	-8
100	-29.2	-16	+4.9	-14

Table B.10 CTI shares against DG penetration: this study and the paper's Fig. 7.

APPENDIX C: PYTHON SCRIPTS

The model, the studies and the evaluation of the coordination were run in DlgSILENT PowerFactory through its Python interface. This appendix lists the scripts in the order in which they are run. Table C.1 gives the purpose of each script.

Script	Purpose
show_progress.py	Script of the PowerFactory script object Final(1): runs the load flows, the short-circuit database and the coordination study, and prints the report in the Output Window
pf_setup.py	Connection to PowerFactory and helper functions
protection_data.py	Feeder, device and fault data
curves.py	Relay and fuse time–current curves
step1_build_model.py	Builds the protection devices and the DG in the model
step2_studies.py	Unbalanced load flow and short-circuit studies
step3_design_and_evaluate.py	Settings, fuse coefficients and classification of the coordination
step4_apply_settings.py	Writes the settings into the relay and fuse elements of the model
step5_fig10_emt.py	EMT simulation of the reclosing sequence

Table C.1 Python scripts of the study.

C.1 show_progress.py

Script of the PowerFactory script object Final(1): runs the load flows, the short-circuit database and the coordination study, and prints the report in the Output Window.

```

1  """
2  Script behind the PowerFactory script object "Final(1)". The report appears in the Output Window.
3
4  First (always)      load flow on all buses and branches, "From node -> To node", DG disconnected
5                      and connected                                loadflow_all_buses.py
6
7  Stage 1 (RUN_DATABASE) load-flow and short-circuit database, DG disconnected and connected, with
                        the
8                      current through R1 and R2 and its direction, If,max, If,min and a validation
9                      checklist                                database_study.py
10                     -> results/database/ (no relay or fuse setting is touched)
11
12  Stage 2 (only when RUN_COORDINATION = True, i.e. after the database has been checked)
13                     design by the paper's method: Tables II, III, IV, Figs. 14, 17;
14                     settings written into the model and PowerFactory's trip times checked;

```

```

15         Tables I-IV (Table IV with the fuse t_MMT) -> results/database/    make_tables.py
16                                     step2_studies.py, step3..., step4...
17
18 While it runs, PowerFactory fills the Output Window with its own messages (a fault on a phase a
19 bus does not have is reported as "Short-circuit calculation not possible"; that is normal). At the
20 end the window is cleared and only the report is printed. The report is also saved as
21 results/Show_Progress_output.txt.
22
23 Replaces the old Show_Progress.py, which needs relays named "R1" and "R2"; in this model they are
24 "R1 Fast", "R1 Delayed", "R2 Fast", "R2 Delayed", "R2 Rev Fast", "R2 Rev Delayed".
25
26 Run: Python Script (ComPython) -> Execute. The model is left in the study case
27 "Study with Substation Transformer".
28 """
29
30 import contextlib
31 import csv
32 import gc
33 import io
34 import os
35 import runpy
36 import subprocess
37 import sys
38 import traceback
39
40 DIR = r"C:\Users\Rabin\Desktop\Digsilent_project - TWIST AND TURN\DSDR_Study"
41 if not os.path.isdir(DIR):
42     DIR = os.path.dirname(os.path.abspath(__file__))
43 RES = os.path.join(DIR, "results")
44 OUT_TXT = os.path.join(RES, "Show_Progress_output.txt")
45 RUN_DATABASE = True          # False: only the load flow on all buses and branches
46 RUN_COORDINATION = True     # stage 2 after the database; switched on 2026-10-01 (database
                              # accepted)
47 OWN = ("pf_setup", "protection_data", "curves", "step3_design_and_evaluate")
48
49
50 def forget():
51     """PowerFactory keeps its Python session between runs: drop this project's modules (and with
52     them the Application object of an earlier run) so every run starts clean."""
53     for m in OWN:
54         sys.modules.pop(m, None)
55     gc.collect()
56
57
58 forget()
59 if DIR not in sys.path:
60     sys.path.insert(0, DIR)
61 import pf_setup
62
63 app = pf_setup.get_app()
64 report = []
65 pf_setup.out = lambda msg="": report.append(str(msg))          # the steps' messages go into the report
66
67
68 def banner(text):
69     report.extend(["", "#" * 100, "# " + text, "#" * 100])
70
71

```

```

72 def step(script, *args):
73     """Run one step of the pipeline; what it prints goes into the report."""
74     argv = getattr(sys, "argv", None)
75     sys.argv = [script] + list(args)
76     buf = io.StringIO()
77     try:
78         with contextlib.redirect_stdout(buf):
79             runpy.run_path(os.path.join(DIR, script), run_name="__main__")
80     finally:
81         report.extend(buf.getvalue().splitlines())
82         if argv is None:
83             del sys.argv
84         else:
85             sys.argv = argv
86     sys.modules.pop("step3_design_and_evaluate", None) # step 4 must read the new results
87
88
89 def external(script):
90     """Run a step that needs no PowerFactory with the normal python.exe: PowerFactory's built-in
91     interpreter is some 50 times slower on pure calculation (step 3: 13 minutes instead of 15 s)."""
92     exe = os.path.join(os.path.dirname(os.path.dirname(os.__file__)), "python.exe")
93     if not os.path.isfile(exe):
94         return step(script)
95     env = dict(os.environ, PYTHONIOENCODING="utf-8")
96     p = subprocess.run([exe, os.path.join(DIR, script)], cwd=DIR, env=env, stdout=subprocess.PIPE,
97                       stderr=subprocess.STDOUT, creationflags=getattr(subprocess, "CREATE_NO_WINDOW", 0))
98     report.extend(p.stdout.decode("utf-8", "replace").splitlines())
99     if p.returncode:
100         raise RuntimeError("%s failed (exit code %d)" % (script, p.returncode))
101
102
103 def grid(title, column):
104     """Fig. 14 / 17 as text: one symbol per faulted node and fault type."""
105     rows = list(csv.DictReader(open(os.path.join(RES, "Fig14_Fig17_classification.csv"))))
106     nodes = []
107     for r in rows:
108         if r["node"] not in nodes:
109             nodes.append(r["node"])
110     sym = {"held": "v", "lost": "x", "n/a": "-"}
111     report.extend(["", " " + title, " (v) exists (x) doesn't exist (-) not applicable"])
112     for ft in ("LG", "LL", "LLG", "LLL"):
113         cells = {r["node"]: sym[r[column]] for r in rows if r["fault"] == ft}
114         report.append(" %-4s |" % ft + "".join(" (%s)" % cells[n] for n in nodes))
115     report.append(" + " + "-" * (6 * len(nodes)) + "> faulted node")
116     report.append(" " + "".join(" %-5s" % n for n in nodes))
117
118
119 def run():
120     app.PrintPlain("Running. PowerFactory's messages below are replaced by the report at the end.")
121     app.EchoOff()
122     banner("LOAD FLOW ON ALL BUSES AND BRANCHES (From node -> To node; no setting is changed)")
123     step("loadflow_all_buses.py")
124     if not RUN_DATABASE:
125         report.extend(["", "Short-circuit database and relay coordination not run: RUN_DATABASE = False in show_progress.py."])
126     return
127     banner("STAGE 1 - LOAD-FLOW AND SHORT-CIRCUIT DATABASE (no relay or fuse setting is changed)")

```

```

128     step("database_study.py")
129     if not RUN_COORDINATION:
130         report.extend(["", "Relay coordination (pickup, fast / delayed operation, TDS, TCC, CTI) has
            NOT been run:",
131             "RUN_COORDINATION = False in show_progress.py. Set it to True once the
            database is accepted."])
132         return
133     banner("STAGE 2 - RELAY COORDINATION")
134     banner("1. LOAD FLOWS AND SHORT CIRCUITS (PowerFactory, complete method, unbalanced)")
135     step("step2_studies.py")
136     banner("2. DESIGN BY THE PAPER'S METHOD - Tables II, III, IV; Figs. 14, 17")
137     external("step3_design_and_evaluate.py")
138     banner("FIG. 14 AND FIG. 17 - recloser-fuse coordination, DG 4.05 MVA at 692")
139     grid("Fig. 14: without the DSDR", "without DSDR (Fig14)")
140     grid("Fig. 17: with the DSDR", "with DSDR (Fig17)")
141     banner("3. DSDR SETTINGS IN THE MODEL - PowerFactory's trip times against the calculation")
142     step("step4_apply_settings.py", "dsdr")
143     banner("4. TABLES I-IV OF THE PAPER (Table IV with the fuse t_MMT and t_TCT) -> results/
            database/")
144     external("make_tables.py")
145     external("make_figures_1_to_6.py")
146     report.extend(["", "Figures: " + os.path.join(RES, "figures") + " (Fig01 ... Fig17)",
147         "Tables: " + os.path.join(RES, "database") + " (Table_I ... Table_IV csv,
            Tables_I_to_IV.txt)"])
148
149
150     try:
151         try:
152             run()
153         except Exception:
154             report.extend(["", "SCRIPT STOPPED WITH AN ERROR:" + [" " + s for s in traceback.
                format_exc().splitlines()])
155             raise
156         finally:
157             try:
158                 app.EchoOn()
159                 app.GetOutputWindow().Clear()
160             except Exception:
161                 pass
162             for line in report:
163                 app.PrintPlain(line)
164             with open(OUT_TXT, "w", encoding="utf-8") as fh:
165                 fh.write("\n".join(report) + "\n")
166             app.PrintPlain("Report saved: " + OUT_TXT)
167     finally:
168         del app
169         forget()

```

C.2 pf_setup.py

Connection to PowerFactory and helper functions.

```

1 """
2 Shared PowerFactory helpers for the replication of
3

```

```

4   M. Yousaf, A. Jalilian, K. M. Muttaqi, D. Sutanto, "An Adaptive Overcurrent Protection Scheme
5   for Dual-Setting Directional Recloser and Fuse Coordination in Unbalanced Distribution Networks
6   With Distributed Generation", IEEE Trans. Ind. Appl., 58(2), 1831-1842, 2022.
7
8   Runs either inside PowerFactory (ComPython) or from a normal Python 3.9 shell (engine mode).
9   """
10
11  import os
12  import sys
13
14  PF_DIR = r"C:\Program Files\DigSILENT\PowerFactory 2021 SP2"
15  PROJECT = "IEEE13 DSDR Fuse Coordination"
16  PFD_FILE = os.path.join(os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
17                          "IEEE 13 Node Feeder.pfd")
18  # The IEEE short-circuit benchmark (Kersting & Shirek 2012, used in ref. [8]) includes the
19  # 115/4.16 kV 5 MVA substation transformer; this study case reproduces it within ~2 %.
20  STUDY_CASE = "Study with Substation Transformer"
21  # Protection devices are built in the base network (this case has no active variation), so they
22  # exist in every study case; the substation transformer case then adds only its variation.
23  BUILD_CASE = "Study Detailed Network Model"
24
25  HERE = os.path.dirname(os.path.abspath(__file__))
26  RESULTS = os.path.join(HERE, "results")
27  os.makedirs(RESULTS, exist_ok=True)
28
29  _app = None
30
31
32  def get_app():
33      global _app
34      if _app is not None:
35          return _app
36      try:
37          import powerfactory as pf                # inside PowerFactory
38      except ImportError:
39          os.environ["PATH"] = PF_DIR + ";" + os.environ["PATH"]
40          sys.path.append(os.path.join(PF_DIR, "Python", "%d.%d" % sys.version_info[:2]))
41          import powerfactory as pf
42          _app = pf.GetApplication()
43          if _app is None:
44              raise RuntimeError("Could not start PowerFactory")
45          return _app
46
47
48  def attr(obj, name, default=None):
49      try:
50          v = obj.GetAttribute(name)
51          return default if v is None else v
52      except Exception:
53          return default
54
55
56  def out(msg=""):
57      print(msg)
58      try:
59          get_app().PrintPlain(msg)
60      except Exception:
61          pass

```

```

62
63
64 def activate(build=False):
65     """Activate the replication project (importing the .pfd the first time) and the study case
66     (BUILD_CASE for model changes, STUDY_CASE for calculations)."""
67     app = get_app()
68     user = app.GetCurrentUser()
69     prj = user.GetContents(PROJECT + ".IntPrj")
70     if not prj:
71         before = {p.loc_name for p in user.GetContents("*.IntPrj")}
72         imp = user.CreateObject("CompfdImport", "import")
73         imp.SetAttribute("e:g_file", PFD_FILE)
74         imp.SetAttribute("g_target", user)
75         imp.Execute()
76         imp.Delete()
77         new = [p for p in user.GetContents("*.IntPrj") if p.loc_name not in before]
78         new[0].SetAttribute("loc_name", PROJECT)
79         prj = [new[0]]
80     prj[0].Activate()
81     clean_variation()
82     name = BUILD_CASE if build else STUDY_CASE
83     case = app.GetProjectFolder("study").GetContents(name + ".IntCase", 1)[0]
84     case.Activate()
85     ldf = app.GetFromStudyCase("ComLdf")
86     ldf.SetAttribute("iopt_net", 1) # unbalanced 3-phase load flow
87     return app
88
89
90 # The "Substation Transformer" variation as delivered in the .pfd: 13 objects. Anything else in its
91 # stage was recorded by a script (or a click) that changed the network while STUDY_CASE was active -
92 # DG in / out, relay or fuse settings, a breaker closed - and overrides the base network from then
93 # on.
94 _ORIGINAL_STAGE = {"Substation Transformer": {
95     "Terminal", "2-Winding Transformer", "Cub_1", "Switch", "Cub_5", "Switch(1)", "External Grid(1)"
96     , "Cub_2",
97     "Switch(2)", "Cub_4", "External Grid", "GND611(1)", "Cub_6"}}
98
99 def clean_variation():
100     """Restore the variation stage to its original objects, so that the base network applies."""
101     app = get_app()
102     n = 0
103     for st in app.GetActiveProject().GetContents("*.IntSstage", 1):
104         keep = _ORIGINAL_STAGE.get(st.loc_name)
105         if keep is None:
106             continue
107         for o in st.GetContents():
108             if o.loc_name not in keep:
109                 o.Delete()
110                 n += 1
111     return n
112
113 def obj(name, cls):
114     hits = get_app().GetCalcRelevantObjects("%s.%s" % (name, cls))
115     if not hits:
116         raise KeyError("%s.%s not found" % (name, cls))
117     return hits[0]

```

```

118
119
120 def lib_type(pattern, must_contain=""):
121     hits = [h for h in get_app().GetGlobalLibrary().GetContents(pattern, 1)
122              if must_contain in h.GetFullName() and "\\Arch\\" not in h.GetFullName()
123              and "\\v001\\" not in h.GetFullName()]
124     if not hits:
125         raise KeyError("library type %s (%s) not found" % (pattern, must_contain))
126     return hits[0]
127
128
129 def dg_units():
130     return get_app().GetCalcRelevantObjects("*.ElmSym")
131
132
133 def save_table1():
134     """Read the DG and its transformer back from the model -> results/Table_I_model.json (Table I)."""
135     import json
136     sym = dg_units()[0]
137     typ = sym.typ_id
138     tr = obj("2-Winding Transformer", "ElmTr2").typ_id
139     keys = ("sgn", "ugn", "iturbo", "xl", "rstr", "xd", "xq", "xds", "xqs", "xdss", "xqss", "tds0",
140            "tdss0", "tqs0",
141            "tqss0", "h")
142     data = {k: round(float(attr(typ, k, float("nan"))), 5) for k in keys}
143     data.update(tr_strn=round(attr(tr, "strn"), 4), tr_hv=round(attr(tr, "utrn_h"), 4), tr_lv=round(
144         attr(tr, "utrn_l"), 4),
145         tr_uk=round(attr(tr, "uktr"), 4), node=sym.bus1.ctrm.loc_name, type=typ.loc_name)
146     with open(os.path.join(RESULTS, "Table_I_model.json"), "w") as fh:
147         json.dump(data, fh, indent=1)
148     return data
149
150 def set_dg(in_service):
151     for g in dg_units():
152         g.SetAttribute("outserv", 0 if in_service else 1)

```

C.3 protection_data.py

Feeder, device and fault data.

```

1 """
2 Protection data of the IEEE 13-node feeder (Fig. 6 of Yousaf et al. 2022) and the paper's
3 published values, used by every other script.
4
5 Sources for device data the 2022 paper does not print:
6 [8] M. Yousaf, K. M. Muttaqi, D. Sutanto, "Overcurrent protection scheme for the IEEE 13-node
7 benchmark test feeder with improved selectivity", IEEE PES GM 2020, Table V
8 (Gould-Shawmut A055C fuses, GE IAC recloser, GE/Alstom CDG relay, GL-800 LV fuse).
9 Figs. 8, 9, 11, 12, 13, 15, 16 of the 2022 paper: fuse bands digitised and matched to the
10 A055C library curves (melting current at 100 s / 1000 s, within 2 %); used for every
11 fuse that a figure shows (F632, F633, F646, F671-1, F671-2, F684, F692, F692-R).
12 """
13

```

```

14 # -----
15 # Device locations. ("line", name, side) -> cubicle of branch 'name' at 'side' (bus1/bus2/buslv)
16 # ("load", name) -> cubicle of load 'name'
17 # -----
18 # Fuse sizes: the size drawn in the 2022 paper's own figures where a figure shows the fuse (bands
19 # digitised and matched to the A055C library curves), otherwise ref. [8] Table V. The figure sizes
20 # take precedence because they are what the 2022 results were produced with: with [8]'s smaller
21 # F671-2 (200E) the fuse melts in the second fast shot of Fig. 10 (0.62 s) instead of after both
22 # shots (paper 1.21 s; 300E gives 1.25 s). The two papers name some devices differently
23 # ([8] Fig. 1 against Fig. 6 of the 2022 paper):
24 # [8] REC1, REC2 = reclosers R1, R2 of the 2022 paper
25 # [8] R1, R2, R5 = overcurrent relays at 632 and 671 - not used in the 2022 scheme, neglected
26 # [8] F671 = fuse at 671 towards 684 -> F671-2 here
27 # [8] F671-R = fuse of the load L671 -> F671 here
28 # F632 (632-645) and F671-1 (671-692) replace [8]'s relays R1 and R5; they are not in [8], so
29 # their sizes come from the fuse bands of the 2022 paper's figures.
30 # [8] also fuses the capacitors (F-C611 50E, F-C675 125E); Fig. 6 of the 2022 paper has no
31 # capacitor fuses, so they are not modelled.
32 FUSES = {
33     # name location library type source of the size
34     "F632": (("line", "LOHL632-645", "bus1"), "A055C400E", "2022 Figs. 12, 15, 16 (not in
35     [8]: relay R1 there)"),
36     "F633": (("line", "LOHL632-633", "bus1"), "A055C250E", "[8] Table V"),
37     "F634": (("line", "XFM-1", "buslv"), "gL-800A", "[8] Table V (IEC 60269 gL 800 A)"),
38     "F645": (("load", "L645-YcPQ"), "A055C250E", "[8] Table V"),
39     "F646": (("line", "LOHL645-646", "bus1"), "A055C300E", "2022 Figs. 11, 15, 16 ([8]: 200E)"),
40     "F-DL": (("line", "LDL", "bus1"), "A055C250E", "[8] Table V"),
41     "F671": (("load", "L571-DcPQ"), "A055C250E", "[8] Table V (F671-R there)"),
42     "F671-1": (("line", "Switch", "bus1"), "A055C300E", "2022 Fig. 9 (not in [8]: relay R5
43     there)"),
44     "F692": (("load", "L692-DcI"), "A055C200E", "[8] Table V"),
45     "F692-R": (("line", "LC692-675", "bus1"), "A055C250E", "2022 Fig. 9 ([8]: 300E)"),
46     "F675": (("load", "L675-YcPQ"), "A055C200E", "[8] Table V"),
47     "F671-2": (("line", "LOHL671-684", "bus1"), "A055C300E", "2022 Fig. 8 ([8]: 200E, F671
48     there)"),
49     "F684": (("line", "LOHL684-611", "bus1"), "A055C200E", "2022 Fig. 8 ([8]: 150E)"),
50     "F611": (("load", "L611-YcI"), "A055C125E", "[8] Table V"),
51     "F652": (("line", "LC684-652A", "bus1"), "A055C150E", "[8] Table V"),
52 }
53
54 # Reclosers. R1: feeder recloser (GE IAC77B801A, extremely inverse, TDS 0.5 / 10 - paper).
55 # R2: mid-line recloser on 632-671 at the 671 end (GE/Alstom CDG34, extremely inverse).
56 RECLOSERS = {
57     "R1": dict(loc=("line", "LOHL650-632", "bus1"), relay="IAC77B801A", ct=900.0),
58     "R2": dict(loc=("line", "LOHL632-671end", "bus2"), relay="CDG34", ct=500.0),
59 }
60
61 R2_UNITS = ["R2 Fast", "R2 Delayed", "R2 Rev Fast", "R2 Rev Delayed"]
62
63 # -----
64 # Fault locations of Figs. 14 / 17 and Table IV. Nodes upstream of R2 are in R1's zone.
65 # phases: conductors present at the node. primary: fuse feeding the node (Table IV "Fuse").
66 # path: fuses in series from the fault towards the source (primary first).

```

```

67 # -----
68 NODES = {
69     "632": dict(phases="ABC", zone="R1", path=[]),
70     "633": dict(phases="ABC", zone="R1", path=["F633"]),
71     "634": dict(phases="ABC", zone="R1", path=["F634", "F633"], lv=True),
72     "645": dict(phases="BC", zone="R1", path=["F632"]),
73     "646": dict(phases="BC", zone="R1", path=["F646", "F632"]),
74     "DL": dict(phases="ABC", zone="R1", path=["F-DL"]),
75     "671": dict(phases="ABC", zone="R2", path=[]),
76     "692": dict(phases="ABC", zone="R2", path=["F671-1"]),
77     "675": dict(phases="ABC", zone="R2", path=["F692-R", "F671-1"]),
78     "680": dict(phases="ABC", zone="R2", path=[]),
79     "684": dict(phases="AC", zone="R2", path=["F671-2"]),
80     "652": dict(phases="A", zone="R2", path=["F652", "F671-2"]),
81     "611": dict(phases="C", zone="R2", path=["F684", "F671-2"]),
82 }
83 NODE_ORDER = ["632", "633", "634", "645", "646", "671", "692", "675", "680", "684", "652", "611", "
    DL"]
84 FAULT_TYPES = ["LG", "LL", "LLG", "LLL"]
85 PF_FAULT = {"LG": "spgf", "LL": "2psc", "LLG": "2pgf", "LLL": "3rst"}
86
87 # Branches of Table II: label -> (branch, side of the measuring cubicle, downstream nodes)
88 TABLE2_BRANCHES = [
89     ("RG60", "632", "LOHL650-632", "bus1", ["632", "633", "645", "646", "DL", "671", "692", "
    675", "680", "684", "652", "611"]),
90     ("632", "633", "LOHL632-633", "bus1", ["633"]),
91     ("632", "645", "LOHL632-645", "bus1", ["645", "646"]),
92     ("632", "671", "LOHL632-671end", "bus2", ["671", "692", "675", "680", "684", "652", "611"
    ]),
93     ("XFM1-HV", "side", "XFM-1", "bushy", ["634"]),
94     ("XFM1-LV", "side", "XFM-1", "busly", ["634"]),
95     ("645", "646", "LOHL645-646", "bus1", ["646"]),
96     ("671", "692", "Switch", "bus1", ["692", "675"]),
97     ("671", "684", "LOHL671-684", "bus1", ["684", "652", "611"]),
98     ("671", "680", "LOHL671-680", "bus1", ["680"]),
99     ("692", "675", "LC692-675", "bus1", ["675"]),
100     ("684", "611", "LOHL684-611", "bus1", ["611"]),
101     ("684", "652", "LC684-652A", "bus1", ["652"]),
102 ]
103
104 # -----
105 # Published values (for comparison only)
106 # -----
107 PAPER_TABLE2 = { # (from, to): (Inom A, If,min kA, If,max kA)
108     ("RG60", "632"): (587.1, 1.24, 5.41), ("632", "633"): (81.1, 0.72, 6.73),
109     ("632", "645"): (143.3, 0.93, 4.25), ("632", "671"): (478.2, 1.08, 4.52),
110     ("XFM1-HV", "side"): (81.1, 0.73, 6.48), ("XFM1-LV", "side"): (704.7, 0.59, 18.76),
111     ("645", "646"): (64.9, 0.84, 3.64), ("671", "692"): (229.3, 0.78, 3.69),
112     ("671", "684"): (70.9, 0.69, 3.47), ("671", "680"): (0.0, 0.59, 6.25),
113     ("692", "675"): (205.9, 0.71, 7.83), ("684", "611"): (70.8, 0.63, 1.93),
114     ("684", "652"): (63.1, 0.66, 2.67),
115 }
116 PAPER_TABLE3 = {"F632": 6.38, "F633": 6.83, "F634": 8.32, "F645": 6.65, "F646": 6.67, "F-DL": 6.54,
117     "F611": 6.19, "F692": 6.13, "F671": 6.08, "F671-1": 5.85, "F671-2": 5.87,
118     "F652": 6.17, "F684": 6.15, "F675": 6.19, "F692-R": 6.14}
119 PAPER_TABLE4 = { # node: (R1 fast, R1 delayed, R2 fast, R2 delayed, fuse MMT)
120     "632": (0.070, 1.398, 0.045, 0.514, None), "633": (0.094, 1.880, 0.052, 0.706, 0.163),
121     "645": (0.105, 2.095, 0.087, 1.659, 0.276), "646": (0.128, 2.555, 0.103, 2.046, 0.181),

```

```

122 "DL": (0.087, 1.733, 0.076, 1.402, 0.248), "671": (0.105, 2.091, 0.068, 1.113, None),
123 "692": (0.115, 2.307, 0.072, 1.258, 0.175), "675": (0.121, 2.414, 0.074, 1.332, 0.185),
124 "684": (0.125, 2.507, 0.075, 1.394, 0.205), "680": (0.137, 2.747, 0.079, 1.555, None),
125 "611": (0.326, 6.523, 0.146, 4.385, 0.417), "652": (0.676, 13.525, 0.143, 9.370, 0.233),
126 }
127 # Fig. 14 (without DSDR): cells marked "doesn't exist"; Fig. 17 (with DSDR): none.
128 PAPER_FIG14_LOST = {("633", "LLG"), ("633", "LLL"), ("645", "LG"), ("645", "LL"), ("645", "LLG"),
129                     ("646", "LL"), ("646", "LLG"), ("675", "LLG"), ("675", "LLL")}
130
131 OLF = 1.25          # overload factor, eqs. (3) and (12)
132 A_FUSE = -1.8       # fuse slope a_i, eq. (6)
133 T_RECL_MIN = 10 / 60 # minimum time delay between the reclosers' delayed curves (10 cycles)

```

C.4 curves.py

Relay and fuse time–current curves.

```

1  """
2  Operating-time curves, identical to the PowerFactory library types (data from studies.json):
3
4  R1  GE IAC77B801A   IAC extremely inverse GES7005B (formula), curve from 1.5 x Ip to 40 x Ip
5  R2  GE/Alstom CDG34 CDG14 extremely inverse 398.S23.37 (table of t vs M for TMS 0.1-1.0),
6       curve from 2 x Is to 40 x Is, t >= 0.031 s at any TMS
7  fuses             minimum melting (MMT) and total clearing (TCT) tables of the library type
8  Times are in s, currents in primary A. A current below a curve's start returns inf (no trip).
9  """
10
11 import math
12
13 INF = float("inf")
14
15
16 def _loglog(x, pts):
17     """Log-log interpolation in a sorted list of (x, y); None outside the range."""
18     if x < pts[0][0] or x > pts[-1][0]:
19         return None
20     for (x1, y1), (x2, y2) in zip(pts, pts[1:]):
21         if x1 <= x <= x2:
22             if x2 == x1:
23                 return y1
24             f = (math.log(x) - math.log(x1)) / (math.log(x2) - math.log(x1))
25             return math.exp(math.log(y1) + f * (math.log(y2) - math.log(y1)))
26     return pts[-1][1]
27
28
29 class IAC:
30     """IAC extremely inverse:  $t = TDS (0.004 + 0.6379/(M-0.62) + 1.7872/(M-0.62)^2 + 0.2461/(M-0.62)^3)$ ."""
31
32     def __init__(self, data):
33         self.imin, self.imax = data["imin"], data["imax"]
34
35     def t(self, i, ip, tds):
36         m = i / ip
37         if m < self.imin:

```

```

38         return INF
39         x = min(m, self.imax) - 0.62
40         return tds * (0.004 + 0.6379 / x + 1.7872 / x ** 2 + 0.2461 / x ** 3)
41
42
43 class CDG14:
44     """CDG14 extremely-inverse table, log-log in M, linear in TMS (extrapolated below 0.1)."""
45
46     def __init__(self, data):
47         rows = data["vmat"]
48         self.tms = rows[0][1:]
49         self.rows = rows[1:]
50         self.imin, self.imax, self.tmin = data["imin"], data["imax"], data["tmin"]
51
52     def t(self, i, ip, tms):
53         m = i / ip
54         if m < self.imin:
55             return INF
56         m = min(m, self.imax)
57         col = [_loglog(m, [(r[0], r[j + 1]) for r in self.rows]) for j in range(len(self.tms))]
58         k = max(0, min(len(self.tms) - 2, next((j for j in range(len(self.tms) - 1)
59             if tms <= self.tms[j + 1]), len(self.tms) - 2)))
60         f = (tms - self.tms[k]) / (self.tms[k + 1] - self.tms[k])
61         return max(self.tmin, col[k] + f * (col[k + 1] - col[k])) # tmin is absolute (as in PF)
62
63
64 class Fuse:
65     def __init__(self, data):
66         m = data["vmat"]
67         self.irat = data["irat"]
68         self.melt = sorted((r[0], r[1]) for r in m if r[0] > 0)
69         self.clear = sorted((r[2], r[3]) for r in m if r[2] > 0)
70
71     @staticmethod
72     def _t(pts, i):
73         if i < pts[0][0]:
74             return INF
75         if i > pts[-1][0]:
76             return pts[-1][1]
77         return _loglog(i, pts)
78
79     def mmt(self, i):
80         return self._t(self.melt, i)
81
82     def tct(self, i):
83         return self._t(self.clear, i)
84
85     def i_at_mmt(self, t):
86         """Current that melts the fuse in t seconds."""
87         pts = sorted((b, a) for a, b in self.melt)
88         return _loglog(t, pts)

```

C.5 step1_build_model.py

Builds the protection devices and the DG in the model.

```
1 """
2 Step 1 - build the protection system of Fig. 6 in a clean copy of the IEEE 13-node feeder.
3
4 * removes every relay, fuse and CT left in the project (fresh start)
5 * adds the lateral nDL - DL for the distributed-load fuse F-DL (fault location "DL")
6 * sets the synchronous DG to the data of Table I
7 * places the 15 fuses (Gould-Shawmut A055C / gL, see protection_data.FUSES)
8 * places R1 (2 x GE IAC77B801A: fast, delayed) and R2 (4 x GE/Alstom CDG34: forward fast,
9   forward delayed, reverse fast, reverse delayed) with their CTs.
10 Settings are written afterwards by step4_apply_settings.py from results/settings.json.
11 """
12
13 from pf_setup import activate, attr, out, obj, lib_type, save_table1
14 from protection_data import FUSES, RECLOSERS, R2_UNITS, DL_LUMP
15
16 app = activate(build=True)
17 grid = obj("632", "ElmTerm").GetParent()
18
19
20 def cubicle(loc):
21     if loc[0] == "load":
22         return obj(loc[1], "ElmLod").GetAttribute("bus1")
23     for cls in ("ElmLne", "ElmTr2", "ElmCoup"):
24         hits = app.GetCalcRelevantObjects("%s.%s" % (loc[1], cls))
25         if hits:
26             return hits[0].GetAttribute(loc[2])
27     raise KeyError(loc)
28
29
30 # ---- 1. fresh start -----
31 n = 0
32 for cls in ("ElmRelay", "RelFuse", "StaCt"):
33     for o in grid.GetContents(".*" + cls, 1):
34         o.Delete()
35     n += 1
36 for f in app.GetProjectFolder("equip").GetContents("Fuse Types Yousaf 2022*"):
37     f.Delete()
38 out("Removed %d old protection objects" % n)
39
40 # ---- 2. distributed-load lateral nDL - DL -----
41 if not app.GetCalcRelevantObjects("DL.ElmTerm"):
42     ndl = obj("nDL", "ElmTerm")
43     dl = grid.AddCopy(ndl, "DL")
44     for c in dl.GetContents(".*StaCubic"):
45         c.Delete()
46     proto = obj("LOHL632-671start", "ElmLne")
47     line = grid.AddCopy(proto, "LDL")
48     line.SetAttribute("bus1", ndl.CreateObject("StaCubic", "Cub_LDL"))
49     line.SetAttribute("bus2", dl.CreateObject("StaCubic", "Cub_LDL"))
50     line.SetAttribute("dline", 10.0) # 10 ft (project length unit)
51     load = obj(DL_LUMP, "ElmLod")
52     old = load.GetAttribute("bus1")
53     load.SetAttribute("bus1", dl.CreateObject("StaCubic", "Cub_Load"))
```

```

54     old.Delete()
55     out("Added terminal DL and 10 ft lateral LDL (config 601); %s moved to DL" % DL_LUMP)
56
57 # ---- 3. DG of Table I -----
58 sym = obj("Synchronous Machine", "ElmSym")
59 typ = sym.typ_id
60 table1 = dict(xl=0.05, rstr=0.0014, xd=1.4, xq=1.372, xds=0.231, xqs=0.8, xdss=0.118, xqss=0.118,
61             tds0=5.5, tdss0=0.05, tq0=1.25, tqss0=0.19, h=0.75)
62 # The machine type takes short-circuit time constants; convert Table I's open-circuit ones.
63 sc_tc = dict(tds=5.5 * 0.231 / 1.4, tdss=0.05 * 0.118 / 0.231, tq0=1.25 * 0.8 / 1.372,
64            tqss=0.19 * 0.118 / 0.8)
65 # Table I gives X'q and T'q0, i.e. a round-rotor machine; with the salient-pole model PowerFactory
66 # ignores them and reads T''q as T''q0 * X''q / Xq, which turns T''q0 = 0.19 s into 0.326 s.
67 for k, v in [("iturbo", 1)] + list(table1.items()) + list(sc_tc.items()):
68     try:
69         typ.SetAttribute(k, v)
70     except Exception:
71         pass
72 out("DG (%s): %s" % (typ.loc_name, ".join("%s=%s" % (k, round(attr(typ, k, float("nan")), 4))
73     for k in table1)))
74 save_table1()
75
76 # ---- 4. fuses -----
77 for name, (loc, tname, src) in FUSES.items():
78     cub = cubicle(loc)
79     f = cub.CreateObject("RelFuse", name)
80     f.typ_id = lib_type(tname + ".TypFuse", "ProtFuse")
81     out("  %-7s %-10s at %-6s (%s)" % (name, tname, cub.GetAttribute("cterm").loc_name, src))
82
83
84 # ---- 5. reclosers -----
85 def ct_type(prim):
86     equip = app.GetProjectFolder("equip")
87     name = "CT %.0f-5A" % prim
88     hit = equip.GetContents(name + ".TypCt")
89     t = hit[0] if hit else equip.CreateObject("TypCt", name)
90     t.SetAttribute("prmtaps", [prim])
91     t.SetAttribute("sectaps", [5.0])
92     return t
93
94
95 def add_relay(cub, name, rtype):
96     r = cub.CreateObject("ElmRelay", name)
97     r.typ_id = rtype
98     r.SlotUpdate()
99     for e in r.GetContents("*.RelIoc") + r.GetContents("*.Earth*.RelToc"):
100         e.SetAttribute("outserv", 1) # phase time-overcurrent only
101     return r
102
103
104 iac = lib_type("IAC77B801A.TypRelay", "ProtRelay\\GE\\IAC\\60Hz")
105 cdg = lib_type("CDG34-5A (1-4).TypRelay", "CDG34 A & B & C\\Preferred Taps")
106 for rname, cfg in RECLOSERS.items():
107     cub = cubicle(cfg["loc"])
108     ct = cub.CreateObject("StaCt", "CT " + rname)
109     ct.typ_id = ct_type(cfg["ct"])
110     ct.SetAttribute("ptapset", cfg["ct"])
111     ct.SetAttribute("stapset", 5.0)

```

```

112     units = ["R1 Fast", "R1 Delayed"] if rname == "R1" else R2_UNITS
113     for u in units:
114         add_relay(cub, u, iac if rname == "R1" else cdg)
115     out("  %s: %s at %s, CT %.0f/5, units %s" % (rname, cfg["relay"], cub.GetAttribute("cterm").
        loc_name,
116                                           cfg["ct"], ", ".join(units)))
117
118 ldf = app.GetFromStudyCase("ComLdf")
119 out("Load flow after build: %s" % ("OK" if ldf.Execute() == 0 else "FAILED"))

```

C.6 step2_studies.py

Unbalanced load flow and short-circuit studies.

```

1  """
2  Step 2 - load flows and short circuits in PowerFactory (complete method, unbalanced).
3
4  Saves results/studies.json with, for every case, the phase currents (A) through R1, R2, every
5  fuse and every Table II branch. The currents do not depend on the protection settings, so the
6  design (step 3) can be iterated in Python without re-running PowerFactory.
7
8  Cases
9      loadflow    DG out / DG in                                -> Inom (Table II), reverse current at R2
10     max         all nodes x LG/LL/LLG/LLL x phases, Rf = 0, DG out and DG in (100 %)
11     min         LG through 3 ohm at all nodes, DG out         -> If,min (Table II)
12     pen         LLL at 633 and 671, DG penetration 0-100 %    (kept in studies.json; Fig. 7 is not
        produced)
13     figures     the faults of Figs. 8-13, 15, 16 and Fig. 10 (DG in)
14  """
15
16 import json
17 import os
18
19 from pf_setup import activate, attr, out, obj, lib_type, RESULTS, dg_units, clean_variation,
        save_table1
20 from protection_data import FUSES, RECLOSERS, NODES, NODE_ORDER, PF_FAULT, TABLE2_BRANCHES
21
22 app = activate()
23 ldf = app.GetFromStudyCase("ComLdf")
24 shc = app.GetFromStudyCase("ComShc")
25 shc.SetAttribute("iopt_mde", 3) # complete method (prefault load flow)
26 shc.SetAttribute("iopt_allbus", 0)
27
28 # ---- measuring points -----
29 POINTS = {}
30 for name, cfg in RECLOSERS.items():
31     POINTS[name] = cfg["loc"]
32 for name, (loc, _, _) in FUSES.items():
33     POINTS[name] = loc
34 for frm, to, br, side, _ in TABLE2_BRANCHES:
35     POINTS["%s-%s" % (frm, to)] = ("line", br, side)
36
37
38 def element(loc):
39     if loc[0] == "load":

```

```

40         return obj(loc[1], "ElmLod"), "bus1"
41     for cls in ("ElmLne", "ElmTr2", "ElmCoup"):
42         hits = app.GetCalcRelevantObjects("%s.%s" % (loc[1], cls))
43         if hits:
44             return hits[0], loc[2]
45     raise KeyError(loc)
46
47
48 ELEMENTS = {k: element(v) for k, v in POINTS.items()}
49 TERMS = {t.loc_name: t for t in app.GetCalcRelevantObjects("*.ElmTerm")}
50
51
52 def currents(var, scale=1000.0):
53     """Phase currents in A (short-circuit results are in kA, load-flow results in A)."""
54     res = {}
55     for k, (e, side) in ELEMENTS.items():
56         res[k] = [round(scale * abs(attr(e, "%s:%s:%s" % (var, side, p), 0.0)), 2) for p in "ABC"]
57     return res
58
59
60 # ---- DG penetration -----
61 sym = dg_units()[0]
62 dg_tr = obj("2-Winding Transformer", "ElmTr2")
63 BASE = dict(sgn=attr(sym.typ_id, "sgn"), strn=attr(dg_tr.typ_id, "strn"),
64             pgini=attr(sym, "pgini"), qgini=attr(sym, "qgini"))
65 save_table1() # Table I as it is in the model (results/Table_I_model.json)
66
67
68 def set_penetration(p):
69     """p = DG rating / 4.05 MVA (4.05 MVA = total feeder load, so 1.0 = 100 %)."""
70     if p <= 0:
71         sym.SetAttribute("outserv", 1)
72         return
73     sym.SetAttribute("outserv", 0)
74     sym.typ_id.SetAttribute("sgn", BASE["sgn"] * p)
75     dg_tr.typ_id.SetAttribute("strn", BASE["strn"] * p)
76     sym.SetAttribute("pgini", BASE["pgini"] * p)
77     sym.SetAttribute("qgini", BASE["qgini"] * p)
78
79
80 # ---- short circuits -----
81 PH_ATTR = {"spg": "i_pspgf", "2psc": "i_p2psc", "2pgf": "i_p2pgf"}
82
83
84 def run_fault(where, ftype, index=0, rf=0.0, xf=0.0, dist=None):
85     """where: terminal name, or line name with dist = % from bus1. Returns dict or None."""
86     code = PF_FAULT[ftype]
87     if dist is None:
88         shc.SetAttribute("shcobj", TERMS[where])
89     else:
90         shc.SetAttribute("shcobj", obj(where, "ElmLne"))
91         shc.SetAttribute("ppro", float(dist))
92     shc.SetAttribute("iopt_shc", code)
93     if code in PH_ATTR:
94         shc.SetAttribute(PH_ATTR[code], index)
95     shc.SetAttribute("Rf", rf)
96     shc.SetAttribute("Xf", xf)

```

```

97     if shc.Execute() != 0:
98         return None
99     if dist is None:
100         t = TERMS[where]
101         ifault = [round(1000.0 * abs(attr(t, "m:Ikss:" + p, 0.0)), 2) for p in "ABC"]
102     else:
103         ifault = None
104     faulted = None
105     if ifault:
106         big = max(ifault)
107         faulted = "".join(p for p, i in zip("ABC", ifault) if big > 0 and i > 0.2 * big)
108     return dict(where=where, type=ftype, index=index, rf=rf, dist=dist, phases=faulted,
109               ifault=ifault, I=currents("m:Ikss"))
110
111
112 def sweep(dg, rf=0.0, types=("LG", "LL", "LLG", "LLL"), nodes=NODE_ORDER, tag="max"):
113     res = []
114     for node in nodes:
115         for ft in types:
116             seen = set()
117             for idx in ([0] if ft == "LLL" else [0, 1, 2]):
118                 r = run_fault(node, ft, idx, rf)
119                 if r is None or not r["phases"]:
120                     continue
121                 key = r["phases"]
122                 if ft != "LLL" and (key in seen or len(key) != (1 if ft == "LG" else 2)):
123                     continue
124                 if not set(key) <= set(NODES[node]["phases"]):
125                     continue
126                 seen.add(key)
127                 r.update(case=tag, dg=dg)
128                 res.append(r)
129     return res
130
131
132 def loadflow(dg_on):
133     set_penetration(1.0 if dg_on else 0.0)
134     if ldf.Execute() != 0:
135         raise RuntimeError("load flow failed")
136     res = currents("m:I", 1.0)
137     e, side = ELEMENTS["R2"]
138     res["R2_P_kW"] = round(attr(e, "m:Psum:%s" % side, 0.0), 1)
139     res["DG_P_MW"] = round(attr(sym, "m:Psum:bus1", 0.0) / 1000.0, 3) if dg_on else 0.0
140     return res
141
142
143 data = dict(points={k: list(v) for k, v in POINTS.items()})
144 data["loadflow"] = {"dg_out": loadflow(False), "dg_in": loadflow(True)}
145 out("Load flow: R2 %.0f kW (DG out) / %.0f kW (DG in)" % (
146     data["loadflow"]["dg_out"]["R2_P_kW"], data["loadflow"]["dg_in"]["R2_P_kW"]))
147
148 faults = []
149 set_penetration(0.0)
150 faults += sweep(0.0, tag="max")
151 faults += sweep(0.0, rf=3.0, types=("LG",), tag="min")
152 set_penetration(1.0)
153 faults += sweep(1.0, tag="max")
154 faults += sweep(1.0, rf=3.0, types=("LG",), tag="min")

```

```

155 out("Node sweeps: %d faults" % len(faults))
156
157 for p in (0.0, 0.10, 0.25, 0.37, 0.50, 0.75, 1.0):
158     set_penetration(p)
159     for r in sweep(p, types=("LLL",), nodes=["633", "671"], tag="pen"):
160         faults.append(r)
161
162 # Figure faults (DG in service). Fig. 9: LLG at the middle of 692-675 through 1 ohm;
163 # Fig. 13: 3-phase at 10 % of 632-633; Fig. 10: LL (a-c) at 684 through 0.2 ohm.
164 set_penetration(1.0)
165 FIG = [("Fig8", "611", "LG", None, 0.0, None), ("Fig9", "LC692-675", "LLG", None, 1.0, 50.0),
166        ("Fig10", "684", "LL", "AC", 0.2, None), ("Fig11", "646", "LL", "BC", 1.0, None),
167        ("Fig12", "645", "LL", "BC", 1.5, None), ("Fig13", "LOHL632-633", "LLL", None, 0.0, 10.0),
168        ("Fig15", "646", "LL", "BC", 0.0, None)]
169 for fig, where, ft, want, rf, dist in FIG:
170     for idx in ([0] if ft in ("LLL",) else [0, 1, 2]):
171         r = run_fault(where, ft, idx, rf, dist=dist)
172         if r is None:
173             continue
174         if want and r["phases"] != want:
175             continue
176         if dist is not None and ft == "LLG" and idx != 1:      # b-c on the 3-phase cable
177             continue
178         r.update(case=fig, dg=1.0)
179         faults.append(r)
180         out(" %s: %s %s at %s%s, Rf %.1f -> R1 %.0f A, R2 %.0f A" % (
181             fig, ft, r["phases"] or "", where, "" if dist is None else " (%.0f %%) " % dist, rf,
182             max(r["I"]["R1"]), max(r["I"]["R2"])))
183         break
184
185 data["faults"] = faults
186
187 # ---- curve data used by the Python analysis (identical to the PowerFactory types) -----
188 curves = {"fuses": {}}
189 for name, (loc, tname, _) in FUSES.items():
190     t = obj(name, "RelFuse").typ_id      # keyed by the installed type: step 4 may have revised
191                                         the size
192     curves["fuses"][t.loc_name] = dict(irat=attr(t, "irat"), vmat=[list(r) for r in t.GetAttribute("
193         vmat")])
194     a055c = lib_type("A055C200E.TypFuse", "ProtFuse").GetParent()
195     for t in a055c.GetContents("A055C*.TypFuse"):      # every size, for fuse revision
196         curves["fuses"].setdefault(t.loc_name, dict(irat=attr(t, "irat"),
197             vmat=[list(r) for r in t.GetAttribute("vmat")]))
198     cdg = obj("R2 Fast", "ElmRelay").GetContents("*.RelToc")[0]
199     ch = attr(cdg, "pcharac")
200     curves["cdg"] = dict(name=ch.loc_name, imin=attr(ch, "imin"), imax=attr(ch, "imax"),
201         tmin=attr(ch, "tmin"), vmat=[list(r) for r in ch.GetAttribute("vmat")])
202     iac = attr(obj("R1 Fast", "ElmRelay").GetContents("Toc.RelToc")[0], "pcharac")
203     curves["iac"] = dict(name=iac.loc_name, expr=attr(iac, "expr"), imin=attr(iac, "imin"),
204         imax=attr(iac, "imax"))
205 data["curves"] = curves
206 set_penetration(1.0)
207 ldf.Execute()
208 with open(os.path.join(RESULTS, "studies.json"), "w") as f:
209     json.dump(data, f, indent=1)
210 clean_variation()      # DG switching above was recorded in the study case's
211                         variation
212 out("Saved %d fault cases to results/studies.json" % len(faults))

```

C.7 step3_design_and_evaluate.py

Settings, fuse coefficients and classification of the coordination.

```
1 """
2 Step 3 - protection design by the paper's method (Section II, Fig. 5) and coordination analysis.
3 Pure Python on results/studies.json (no PowerFactory needed).
4
5 The paper's workflow, in three stages:
6
7 A Design without DG (conventional fuse-saving scheme)
8     step 1 load flow -> Inom (Table II)
9     step 3 pickups by eq. (3),  $I_p = OLF \times I_{nom}$ , realised with the relay's CT and taps
10         R1 IAC77B801A, TDS 0.5 (fast) / 10 (delayed) - paper
11         R2 CDG34 (CDG14 table, curve from 2 x plug). Fast plug =  $I_p/2$  (curve starts at  $I_p$ ,
12             as in Figs. 8 and 13), delayed plug =  $I_p$ . Fast TMS = lowest dial 0.1; delayed
13             TMS = highest dial that stays 10 cycles below R1's delayed curve.
14     step 4 fuse coefficients  $b_i$ , eqs. (6)-(9) -> Table III ( $i = 1$  closest to the fault, [25])
15     step 5 fault study; steps 8-10 until every fault is coordinated without DG
16 B DG added (4.05 MVA at 692), settings unchanged -> Fig. 14, Figs. 8-13
17 C R2 as DSDR: reverse pickup eq. (12), TMS revised by  $I_{f,Rec} / I_{f,Fuse}$  (step 8);
18     fuse revision (step 9) where a dial limit is reached -> Fig. 17, Table IV, Figs. 15-16
19
20 Coordination of one fault ("held"):
21     * every recloser that interrupts a source feeding the fault (R1 for the grid in R1's zone,
22         R2 for the grid in R2's zone, R2 in reverse for the DG in R1's zone) picks up, trips on its
23         fast curve before the primary fuse melts (MMT), and on its delayed curve after the fuse clears
24         (TCT);
25     * series fuses:  $TCT(primary) < 0.75 MMT(backup)$ , eq. (8);
26     * R2-zone faults: R2 fast before R1 fast and R2 delayed  $\geq 10$  cycles before R1 delayed.
27 A cell of Figs. 14 / 17 is held only if every phase combination of that fault type holds.
28 """
29
30 import copy
31 import csv
32 import json
33 import math
34 import os
35
36 import matplotlib
37 matplotlib.use("Agg")
38 import matplotlib.pyplot as plt
39 from matplotlib.lines import Line2D
40
41 from curves import IAC, CDG, Fuse, INF, _loglog
42 from protection_data import (FUSES, NODES, NODE_ORDER, FAULT_TYPES, TABLE2_BRANCHES, PAPER_TABLE2,
43                             PAPER_TABLE3, PAPER_TABLE4, PAPER_FIG14_LOST, OLF, A_FUSE, T_RECL_MIN)
44
45 HERE = os.path.dirname(os.path.abspath(__file__))
46 RES = os.path.join(HERE, "results")
47 FIG = os.path.join(RES, "figures")
48 os.makedirs(FIG, exist_ok=True)
49
50 D = json.load(open(os.path.join(RES, "studies.json")))
51 IACC, CDGC = IAC(D["curves"]["iac"], CDG(D["curves"]["cdg"]))
52 FTYPE = {t: Fuse(v) for t, v in D["curves"]["fuses"].items()}
53 AO55C = sorted([t for t in FTYPE if t.startswith("A055C")], key=lambda t: FTYPE[t].irat)
```

```

54 FAULTS = D["faults"]
55 LF = D["loadflow"]
56 REPORT = []
57
58 CDG_TAPS = [1.0, 1.2, 1.5, 2.0, 2.4, 3.0, 4.0] # CDG34-5A (1-4), secondary A
59 IAC_TAPS = [0.5, 0.6, 0.7, 0.8, 1.0, 1.2, 1.5, 2.0, 2.5, 3.0, 4.0] # IAC77B801A, secondary A
60 CT_R1, CT_R2, CT_R2RV = 900.0, 1000.0, 500.0
61 TMS_MIN, TMS_MAX = 0.1, 1.0 # CDG34 dial range
62 # The DSDR keeps the CDG34 extremely-inverse curve (paper, Section IV-A). That curve is a table
63 # for TMS 0.1-1.0 whose times are not proportional to TMS, so the dial is not extended below 0.1
64 # (PowerFactory's linear extrapolation below 0.1 even gives negative times at low multiples).
65 DSDR_TMS_MIN = TMS_MIN
66
67
68 def say(s=""):
69     print(s)
70     REPORT.append(s)
71
72
73 def imax(rec, point):
74     return max(rec["I"][point])
75
76
77 def fmt(t, nd=3):
78     return "no trip" if t == INF else ("%.*f" % (nd, t))
79
80
81 def save_png(fig, fname):
82     """Save a figure; if the file is open in a viewer (locked on Windows) write <name>_new.png."""
83     path = os.path.join(FIG, fname)
84     alt = path[:-4] + "_new.png"
85     try:
86         fig.savefig(path, dpi=160)
87         if os.path.isfile(alt):
88             os.remove(alt)
89     except OSError:
90         fig.savefig(alt, dpi=160)
91         print("NOTE: %s is open in another program; the new figure is %s" % (fname, os.path.basename(alt)))
92
93
94 CMP = os.path.join(RES, "comparison") # values of the reference paper, for the
    comparison only
95 os.makedirs(CMP, exist_ok=True)
96
97
98 def write_csv(name, header, rows):
99     path = os.path.join(RES, name)
100     try:
101         f = open(path, "w", newline="")
102     except PermissionError: # open in Excel: the old file stays, the new one
        goes beside it
103         f = open(path[:-4] + "_new.csv", "w", newline="")
104         print("NOTE: %s is open in another program; the new table is %s_new.csv" % (name, name[:-4])
105             )
106     with f:
107         w = csv.writer(f)
108         w.writerow(header)

```

```

108         w.writerows(rows)
109
110
111 def faults(case="max", dg=None, node=None, ftype=None, zone=None):
112     return [r for r in FAULTS if r["case"] == case and (dg is None or r["dg"] == dg)
113             and (node is None or r["where"] == node) and (ftype is None or r["type"] == ftype)
114             and (zone is None or (r["where"] in NODES and NODES[r["where"]]["zone"] == zone
115                                 and not NODES[r["where"]].get("lv")))]
116
117
118 # =====
119 # Operating times
120 # =====
121 def t_r1(i, s, mode):
122     return IACC.t(i, s["R1"]["ip"], s["R1"]["tds_" + mode])
123
124
125 def t_r2(i, s, unit, mode):
126     u = s[unit]
127     return CDGC.t(i, u["is_" + mode], u["tms_" + mode])
128
129
130 def fuse(s, name):
131     return FTYPE[s["fuses"][name]]
132
133
134 # =====
135 # Coordination of one fault
136 # =====
137 def evaluate(r, s, dsdr):
138     node = r["where"]
139     info = NODES[node]
140     i1, i2 = imax(r, "R1"), imax(r, "R2")
141     reasons, reclosers = [], [] # reclosers: (name, unit, I, t_fast, t_delayed)
142
143     if info["zone"] == "R2":
144         reclosers.append(("R2", "R2fw", i2, t_r2(i2, s, "R2fw", "f"), t_r2(i2, s, "R2fw", "d")))
145         t1f, t1d = t_r1(i1, s, "f"), t_r1(i1, s, "d")
146         if reclosers[0][3] < INF and not reclosers[0][3] < t1f:
147             reasons.append("R2 fast (%.3f s) not before R1 fast (%s s)" % (reclosers[0][3], fmt(t1f)))
148
149         if reclosers[0][4] < INF and t1d < INF and not t1d - reclosers[0][4] >= T_RECL_MIN:
150             reasons.append("R2/R1 delayed margin %.3f s < 10 cycles" % (t1d - reclosers[0][4]))
151     else:
152         reclosers.append(("R1", "R1", i1, t_r1(i1, s, "f"), t_r1(i1, s, "d")))
153         if r["dg"] > 0 and i2 >= s["dg_infeed"]: # DG infeed above the eq. (12) pickup
154             unit = "R2rv" if dsdr else "R2fw"
155             reclosers.append(("R2 rev" if dsdr else "R2", unit, i2, t_r2(i2, s, unit, "f"),
156                             t_r2(i2, s, unit, "d")))
157     res = dict(node=node, type=r["type"], phases=r["phases"], dg=r["dg"], reclosers=reclosers,
158               fuse=None, cti_f=None, cti_d=None, flags=set())
159     for name, unit, i, tf, td in reclosers:
160         if tf == INF:
161             reasons.append("%s does not pick up (%.0f A)" % (name, i))
162             res["flags"].add(("nopickup", unit))
163     path = info["path"]
164     if path:
165         p = path[0]

```

```

164         ip_ = imax(r, p)
165         f = fuse(s, p)
166         mmt, tct = f.mmt(ip_), f.tct(ip_)
167         tf = max(x[3] for x in reclosers)
168         td = min(x[4] for x in reclosers)
169         res.update(fuse=p, i_fuse=ip_, mmt=mmt, tct=tct, cti_f=mmt - tf, cti_d=td - tct)
170         for name, unit, i, tfk, tdk in reclosers:
171             if tfk < INF and not tfk < mmt:
172                 reasons.append("%s melts (%.3f s) before %s fast (%.3f s)" % (p, mmt, name, tfk))
173                 res["flags"].add(("fast", unit))
174             if tdk < INF and not tct < tdk:
175                 reasons.append("%s delayed (%.3f s) before %s clears (%.3f s)" % (name, tdk, p, tct))
176         )
177         res["flags"].add(("delayed", unit))
178         for b in path[1:]:
179             mb = fuse(s, b).mmt(imax(r, b))
180             if not tct < 0.75 * mb:
181                 reasons.append("series %s/%s: TCT %.3f > 0.75 x MMT %.3f" % (p, b, tct, 0.75 * mb))
182                 res["flags"].add(("series", b))
183         res["held"] = not reasons
184         res["reason"] = "; ".join(reasons)
185         return res
186
187 def classify(s, dsdr, dg):
188     grid, detail = {}, []
189     for node in NODE_ORDER:
190         for ft in FAULT_TYPES:
191             recs = faults("max", dg, node, ft)
192             if NODES[node].get("lv") or not recs:
193                 grid[(node, ft)] = "n/a"
194                 continue
195             ev = [evaluate(r, s, dsdr) for r in recs]
196             detail += ev
197             grid[(node, ft)] = "held" if all(e["held"] for e in ev) else "lost"
198     return grid, detail
199
200
201 def count(grid):
202     return sum(v == "held" for v in grid.values()), sum(v != "n/a" for v in grid.values())
203
204
205 # =====
206 # Steps 8-10 - revise the recloser dials, then the fuse sizes
207 # =====
208 def step8(s, dsdr, dg, units, log):
209     """Revise TMS of the given R2 units by If,Rec / If,Fuse until held or at a limit."""
210     for _ in range(40):
211         changed = False
212         ev = [e for e in classify(s, dsdr, dg)[1] if not e["held"]]
213         for unit in units:
214             fast = [e for e in ev if ("fast", unit) in e["flags"]]
215             slow = [e for e in ev if ("delayed", unit) in e["flags"]]
216             if fast and s[unit]["tms_f"] > s[unit]["tms_min"]:
217                 k = min(next(x[2] for x in e["reclosers"] if x[1] == unit) / e["i_fuse"] for e in
218 fast)
219                 new = max(s[unit]["tms_min"], round(s[unit]["tms_f"] * k, 3))

```

```

219         log.append("step 8: %s fast TMS %.3f x If,Rec/If,Fuse (%.3f) -> %.3f" % (unit, s[
unit]["tms_f"], k, new))
220         s[unit]["tms_f"], changed = new, True
221         if slow and s[unit]["tms_d"] < s[unit]["tms_d_max"]:
222             k = max(e["i_fuse"] / next(x[2] for x in e["reclosers"] if x[1] == unit) for e in
slow)
223             new = min(s[unit]["tms_d_max"], round(s[unit]["tms_d"] * max(k, 1.05), 3))
224             log.append("step 8: %s delayed TMS %.3f x If,Fuse/If,Rec (%.3f) -> %.3f" % (unit, s[
unit]["tms_d"], k, new))
225             s[unit]["tms_d"], changed = new, True
226             if not changed:
227                 return
228
229
230 def resize(s, name, step, log, why):
231     t = s["fuses"][name]
232     if t not in A055C:
233         return False
234     k = A055C.index(t) + step
235     if not 0 <= k < len(A055C) or FTYPE[A055C[k]].irat < 100:
236         return False
237     s["fuses"][name] = A055C[k]
238     log.append("step 9/10: %s %s -> %s (%s)" % (name, t, A055C[k], why))
239     return True
240
241
242 def n_lost(s, dsdr, dgs):
243     return sum(not e["held"] for dg in dgs for e in classify(s, dsdr, dg)[1])
244
245
246 def step9(s, dsdr, dgs, log):
247     """Fuse revision (step 9) and series-fuse revision (step 10). Candidate moves from the
248     failures: primary fuse one A055C size up (melts before a recloser's fast curve at its dial
249     limit) or down (clears after the delayed curve), or for eq. (8) the backup one size up or the
250     primary one size down. The move that removes most lost cases is applied; repeat."""
251     for _ in range(30):
252         base = n_lost(s, dsdr, dgs)
253         if base == 0:
254             return
255         moves = set()
256         for dg in dgs:
257             for e in classify(s, dsdr, dg)[1]:
258                 path = NODES[e["node"]]["path"]
259                 for kind, what in e["flags"]:
260                     where = "%s %s" % (e["node"], e["type"])
261                     for k in (1, 2, 3):
262                         if kind == "fast":
263                             moves.add(((e["fuse"], k),))
264                             for k2 in (1, 2, 3):
265                                 moves.add(((e["fuse"], k),) + tuple((b, k2) for b in path[1:]))
266                         elif kind == "delayed":
267                             moves.add(((e["fuse"], -k),))
268                         elif kind == "series":
269                             moves.add(((what, k),))
270                             moves.add(((e["fuse"], -k),))
271         best = None
272         for mv in sorted(moves):
273             trial = copy.deepcopy(s)

```

```

274         if not all(resize(trial, name, k, [], "") for name, k in mv):
275             continue
276         n = n_lost(trial, dsdr, dgs)
277         size = sum(abs(k) for _, k in mv)
278         if n < base and (best is None or (n, size) < (best[0], best[1])):
279             best = (n, size, mv)
280     if best is None:
281         return
282     for name, k in best[2]:
283         resize(s, name, k, log, "lost cases %d -> %d" % (base, best[0]))
284
285
286
287 # =====
288 # Stage A - settings without DG
289 # =====
290 def nearest_tap(target, taps, ratio):
291     return min(taps, key=lambda t: abs(t * ratio - target))
292
293
294 def table2():
295     rows = []
296     for frm, to, br, side, nodes in TABLE2_BRANCHES:
297         key = "%s-%s" % (frm, to)
298         inom = max(LF["dg_out"][key])
299         fmax = [imax(r, key) for r in faults("max", 0.0) if r["where"] in nodes]
300         fmin = [imax(r, key) for r in faults("min", 0.0) if r["where"] in nodes]
301         rows.append([frm, to, round(inom, 1), round(min(fmin) / 1e3, 2), round(max(fmax) / 1e3, 2)])
302     write_csv("Table_II.csv", ["From", "To", "Inom (A)", "If,min (kA)", "If,max (kA)"], rows)
303     write_csv(os.path.join("comparison", "Table_II_vs_paper.csv"),
304               ["From", "To", "Inom (A)", "If,min (kA)", "If,max (kA)", "paper Inom (A)", "paper If,
305               min (kA)",
306               "paper If,max (kA)", [r + list(PAPER_TABLE2[(r[0], r[1])]) for r in rows]])
307     say("\nTABLE II - rated (load flow), minimum (LG through 3 ohm, farthest node) and maximum "
308         "(bolted, nearest node) branch current, DG out")
309     say(" %-8s %-5s %8s | %8s | %8s" % ("from", "to", "Inom A", "Ifmin kA", "Ifmax kA"))
310     for r in rows:
311         say(" %-8s %-5s %8.1f | %8.2f | %8.2f" % (r[0], r[1], r[2], r[3], r[4]))
312     return {"%s-%s" % (r[0], r[1]): r for r in rows}
313
314 def design_reclosers(t2):
315     s = {"fuses": {n: t for n, (_, t, _) in FUSES.items()}}
316     inom1 = t2["RG60-632"][2]
317     tap = nearest_tap(OLF * inom1, IAC_TAPS, CT_R1 / 5)
318     s["R1"] = dict(ct=CT_R1, tap=tap, ip=tap * CT_R1 / 5, ip_eq3=OLF * inom1, tds_f=0.5, tds_d=10.0)
319     inom2 = t2["632-671"][2]
320     ip = OLF * inom2
321     tf, td = nearest_tap(ip / 2, CDG_TAPS, CT_R2 / 5), nearest_tap(ip, CDG_TAPS, CT_R2 / 5)
322     fw = dict(ct=CT_R2, ip_eq3=ip, tap_f=tf, tap_d=td, is_f=tf * CT_R2 / 5, is_d=td * CT_R2 / 5,
323              tms_f=TMS_MIN, tms_d=None, tms_min=TMS_MIN)
324     for k in range(int(TMS_MAX * 100), 4, -1): # highest delayed dial below R1 - 10
325         cycles
326         fw["tms_d"] = k / 100.0
327         if all(t_r2(imax(r, "R2"), {"R2fw": fw}, "R2fw", "d") == INF or
328              t_r1(imax(r, "R1"), s, "d") - t_r2(imax(r, "R2"), {"R2fw": fw}, "R2fw", "d") >=
329              T_RECL_MIN
330              for dg in (0.0, 1.0) for r in faults("max", dg, zone="R2")):

```

```

329         break
330     fw["tms_d_max"] = fw["tms_d"]
331     s["R2fw"] = fw
332     # a DG infeed through R2 counts as a fault contribution above the eq. (12) pickup
333     s["dg_infeed"] = OLF * max(LF["dg_in"]["R2"])
334     say("\nSTAGE A - SETTINGS WITHOUT DG (step 3)")
335     say(" R1 GE IAC77B801A: Inom %.1f A -> Ip = 1.25 Inom = %.1f A -> CT 900/5, tap %.1f A = %.0f
A; "
336         "TDS 0.5 (fast) / 10 (delayed)" % (inom1, OLF * inom1, s["R1"]["tap"], s["R1"]["ip"]))
337     say(" R2 GE/Alstom CDG34 (forward): Inom %.1f A -> Ip = %.1f A -> CT 1000/5; fast plug %.1f A
= %.0f A "
338         "(curve from %.0f A), delayed plug %.1f A = %.0f A (curve from %.0f A); TMS %.2f / %.2f" % (
339         inom2, ip, tf, fw["is_f"], 2 * fw["is_f"], td, fw["is_d"], 2 * fw["is_d"], fw["tms_f"],
fw["tms_d"]))
340     say("         delayed TMS %.2f = highest dial keeping R2 >= 10 cycles below R1 for all R2-zone
faults" % fw["tms_d"])
341     return s
342
343
344 # =====
345 # Table III
346 # =====
347 FUSE_PATHS = { # longest series path through each fuse (fault first), node below the fuse
348     "F632": ("645", ["F646", "F632"]), "F633": ("633", ["F633"]), "F634": ("634", ["F634", "F633"]),
349     "F645": ("645", ["F645", "F632"]), "F646": ("646", ["F646", "F632"]), "F-DL": ("DL", ["F-DL"]),
350     "F671": ("671", ["F671"]), "F671-1": ("692", ["F675", "F692-R", "F671-1"]),
351     "F692": ("692", ["F692", "F671-1"]), "F692-R": ("675", ["F675", "F692-R", "F671-1"]),
352     "F675": ("675", ["F675", "F692-R", "F671-1"]), "F671-2": ("684", ["F611", "F684", "F671-2"]),
353     "F684": ("611", ["F611", "F684", "F671-2"]), "F611": ("611", ["F611", "F684", "F671-2"]),
354     "F652": ("652", ["F652", "F671-2"]),
355 }
356
357
358 def table3(s):
359     rows = []
360     say("\nTABLE III - fuse coefficient b_i, eq. (9), a_i = -1.8, at the maximum fault current below
the fuse (DG out)")
361     say(" %-7s %7s %4s %7s %7s %8s | %6s | %s" % ("fuse", "If A", "i/z", "t_F s", "t_D s", "t_fuse"
,
362         "b_i", "b_i of the installed fuse"))
363     cmp_rows = []
364     for f, (node, path) in FUSE_PATHS.items():
365         recs = faults("max", 0.0, node)
366         i = max(max(r["ifault"]) for r in recs)
367         i_rel = i * (0.48 / 4.16) if NODES[node].get("lv") else i # F634: recloser sees HV
current
368         if NODES[node]["zone"] == "R1":
369             tf, td = t_r1(i_rel, s, "f"), t_r1(i_rel, s, "d")
370         else:
371             tf, td = t_r2(i_rel, s, "R2fw", "f"), t_r2(i_rel, s, "R2fw", "d")
372         z, idx = len(path), path.index(f) + 1
373         tgt = tf + idx / (z + 1.0) * (td - tf)
374         tgt_rev = tf + (z + 1 - idx) / (z + 1.0) * (td - tf)
375         b = math.log10(tgt) - A_FUSE * math.log10(i)
376         b_rev = math.log10(tgt_rev) - A_FUSE * math.log10(i)
377         mmt = fuse(s, f).mmt(i)
378         b_fit = math.log10(mmt) - A_FUSE * math.log10(i) if 0 < mmt < INF else float("nan")

```

```

379     base = [f, round(i), "%d/%d" % (idx, z), round(tf, 3), round(td, 3), round(tgt, 3), round(b,
380         2)]
381     rows.append(base + [round(b_fit, 2), s["fuses"][f], fmt(mmt), fmt(fuse(s, f).tct(i))])
382     cmp_rows.append(base + [round(b_rev, 2), PAPER_TABLE3[f]] + rows[-1][7:])
383     say("%-7s %7.0f %4s %7.3f %7.3f %8.3f | %6.2f | %.2f (%s)" % (*rows[-1][:8], s["fuses"][f
384         ]))
385     write_csv("Table_III.csv", ["Fuse", "If (A)", "i/z", "t_fast (s)", "t_delayed (s)", "t_fuse eq
386         .(9) (s)",
387         "b_i (eq. 9, i=1 closest to fault)", "b_i of installed fuse at If",
388         "installed fuse",
389         "t_MMT of installed fuse at If (s)", "t_TCT of installed fuse at If
390         (s)"], rows)
391     write_csv(os.path.join("comparison", "Table_III_vs_paper.csv"),
392         ["Fuse", "If (A)", "i/z", "t_fast (s)", "t_delayed (s)", "t_fuse eq.(9) (s)",
393         "b_i (i=1 closest to fault, ref. [25])", "b_i (i counted from source)", "paper b_i",
394         "b_i of installed fuse at If", "installed fuse", "t_MMT of installed fuse at If (s)",
395         "t_TCT of installed fuse at If (s)"], cmp_rows)
396     return rows
397
398 # =====
399 # Figures
400 # =====
401 PAL = dict(blue="#2a78d6", orange="#eb6834", aqua="#1baf7a", violet="#4a3aa7", magenta="#e87ba4",
402     green="#008300", good="#0ca30c", critical="#d03b3b", ink="#0b0b0b", ink2="#52514e",
403     grid="#d9d8d4", surface="#fcfcfb")
404 plt.rcParams.update({"font.size": 9, "axes.edgecolor": PAL["ink2"], "axes.labelcolor": PAL["ink"],
405     "text.color": PAL["ink"], "xtick.color": PAL["ink2"], "ytick.color": PAL["ink2"]
406     },
407     {"figure.facecolor": PAL["surface"], "axes.facecolor": PAL["surface"],
408     "savefig.facecolor": PAL["surface"], "font.family": "DejaVu Sans"})
409
410 def coord_grid(grid, title, fname):
411     fig, ax = plt.subplots(figsize=(8.6, 3.1))
412     for x, node in enumerate(NODE_ORDER):
413         for y, ft in enumerate(FAULT_TYPES[::-1]):
414             st = grid[(node, ft)]
415             if st == "n/a":
416                 ax.text(x, y, "-", ha="center", va="center", fontsize=13, color=PAL["ink2"])
417             else:
418                 c = PAL["good"] if st == "held" else PAL["critical"]
419                 ax.add_patch(plt.Circle((x, y), 0.36, facecolor="none", edgecolor=c, lw=1.6))
420                 ax.text(x, y, "held" if st == "held" else "lost", ha="center", va="center", fontsize
421                     =12,
422                     color=c, fontweight="bold")
423     ax.set_xticks(range(len(NODE_ORDER)))
424     ax.set_xticklabels(NODE_ORDER)
425     ax.set_yticks(range(4))
426     ax.set_yticklabels(FAULT_TYPES[::-1])
427     ax.set_xlim(-0.6, len(NODE_ORDER) - 0.4)
428     ax.set_ylim(-0.6, 3.6)
429     ax.set_aspect("equal")
430     for sp in ax.spines.values():
431         sp.set_visible(False)
432     ax.tick_params(length=0)
433     ax.set_xlabel("Faulted node")
434     ax.set_title(title, fontsize=10, loc="left")

```

```

430 handles = [Line2D([], [], marker="$held$", color=PAL["good"], ls="", ms=9, label="coordination
431             held"),
432             Line2D([], [], marker="$lost$", color=PAL["critical"], ls="", ms=9, label="
coordination lost"),
433             Line2D([], [], marker="$-$", color=PAL["ink2"], ls="", ms=9, label="not applicable")]
434 ax.legend(handles=handles, loc="upper center", bbox_to_anchor=(0.5, -0.3), ncol=3, frameon=False
435 )
436 fig.tight_layout()
437 save_png(fig, fname)
438 plt.close(fig)
439
440 def annotate_times(ax, opts, x_left=100.0):
441     """Operating times on a TCC: a dotted line from each operating point to the time axis with its
442     value,
443     and the coordination margins as arrows at the primary fuse current:
444     CTI = t_MMT(primary fuse) - t_fast(recloser)          (fast trip before the fuse melts)
445     t_delayed(recloser) - t_TCT(primary fuse)             (fuse clears before the delayed trip)
446     opts: [(device, current, [t1, t2], colour)] - for a recloser t1/t2 = fast/delayed, for a fuse
MMT/TCT."""
447     import math
448     labels = []
449     for dev, i_dev, ops, c in opts:
450         names = ("melts", "clears") if dev in FUSES else ("fast", "delayed")
451         short = dev.replace("R2rv", "R2 rev").replace("R2fw", "R2")
452         for t, nm in zip(ops, names):
453             if t < INF and 0.01 < t < 1000:
454                 ax.plot([x_left, i_dev], [t, t], color=c, lw=0.7, ls=":", alpha=0.9, zorder=2)
455                 labels.append([t, "%s %s %.3f s" % (short, nm, t), c])
456     # labels at the left edge, spread so that they do not overlap (log axis)
457     labels.sort(key=lambda x: x[0])
458     ys = []
459     for t, _, _ in labels:
460         y = math.log10(t)
461         if ys and y - ys[-1] < 0.13:
462             y = ys[-1] + 0.13
463         ys.append(y)
464     for (t, txt, c), y in zip(labels, ys):
465         ax.text(x_left * 1.08, 10 ** y, txt, fontsize=6.4, color=c, va="bottom", ha="left",
466                bbox=dict(boxstyle="square,pad=0.08", fc="white", ec="none", alpha=0.75), zorder=7)
467     recs = [o for o in opts if o[0] not in FUSES]
468     fuses = [o for o in opts if o[0] in FUSES]
469     if not recs or not fuses:
470         return
471     # a recloser that carries current but does not pick up
472     lo_y, hi_y = ax.get_ylim()
473     for dev, i_dev, (tf, _), c in recs:
474         if tf == INF and i_dev > 1.0:
475             ax.text(i_dev * 1.05, (lo_y * hi_y) ** 0.5 / 3, "%s: %.0f A, does not trip" % (
476                 dev.replace("R2rv", "R2 rev").replace("R2fw", "R2"), i_dev), rotation=90, fontsize
477                 =7.4,
478                 color="#c62828", fontweight="bold", va="center", ha="left", zorder=8,
479                 bbox=dict(boxstyle="round,pad=0.15", fc="white", ec="#c62828", lw=0.6, alpha=0.9))
480     # the recloser that decides: the one with the latest fast trip (it must still beat the fuse)
481     live = [r for r in recs if r[2][0] < INF]
482     if not live:
483         return
484     _, i_rec, (tf, td), _ = max(live, key=lambda r: r[2][0])

```

```

482     _, i_fu, (mmt, tct), _ = fuses[0]
483     x = i_fu * 1.35
484     for lo, hi, name in ((tf, mmt, "CTI"), (tct, td, "t$_D$ - t$_{TCT}$")):
485         if not (lo < INF and hi < INF and 0.01 < min(lo, hi) and max(lo, hi) < 1000):
486             continue
487         ok = hi > lo
488         col = "#2e7d32" if ok else "#c62828"
489         d = hi - lo
490         val = ("%+.0f ms" % (1000 * d)) if abs(d) < 1 else ("%+.2f s" % d)
491         ax.annotate("", xy=(x, hi), xytext=(x, lo), arrowprops=dict(arrowstyle="<->", color=col, lw
=1.3), zorder=8)
492         right = x * 4.5 < ax.get_xlim()[1] # room for the label right of the
arrow?
493         ax.text(x * 1.06 if right else x / 1.06, (lo * hi) ** 0.5, "%s = %s" % (name, val.replace("-", "-")),
494                 fontsize=7.2, color=col, fontweight="bold", va="center", ha="left" if right else "
right", zorder=8,
495                 bbox=dict(boxstyle="round,pad=0.15", fc="white", ec=col, lw=0.6, alpha=0.9))
496
497
498 def tcc(fig_id, rec, s, devices, title, fname, dsdr):
499     fig, ax = plt.subplots(figsize=(6.6, 4.8))
500     ii = [10 ** (2 + k / 200.0) for k in range(601)]
501     colors = [PAL["blue"], PAL["orange"], PAL["aqua"], PAL["violet"]]
502     handles = []
503     opts = []
504     for n, dev in enumerate(devices):
505         c = colors[n]
506         if dev in FUSES:
507             f = fuse(s, dev)
508             i_dev = imax(rec, dev)
509             lo = [(i, f.mmt(i)) for i in ii if f.mmt(i) < 1000]
510             hi = [(i, f.tct(i)) for i in ii if f.tct(i) < 1000]
511             ax.plot(*zip(*lo), color=c, lw=1.3)
512             ax.plot(*zip(*hi), color=c, lw=1.3)
513             ax.fill_betweenx([p[1] for p in lo], [p[0] for p in lo],
514                             [_loglog(p[1], sorted((b, a) for a, b in f.clear)) or p[0] for p in lo
],
515                             color=c, alpha=0.2, lw=0)
516             ops = [f.mmt(i_dev), f.tct(i_dev)]
517             label = ("%s %s %.0f A: melts %.3f s, clears %.3f s" % (dev, s["fuses"][dev], i_dev,
ops[0], ops[1])
518                     if ops[1] < INF else "%s %s %.0f A: does not melt" % (dev, s["fuses"][dev],
i_dev))
519             else:
520                 i_dev = imax(rec, "R1" if dev == "R1" else "R2")
521                 if dev == "R1":
522                     fun = lambda i, m: t_r1(i, s, m)
523                     name = "R1"
524                 else:
525                     unit = "R2rv" if dev == "R2rv" else "R2fw"
526                     fun = lambda i, m, u=unit: t_r2(i, s, u, m)
527                     name = "R2 reverse (DSDR)" if dev == "R2rv" else "R2"
528                 for m, ls in (("f", "-"), ("d", "--")):
529                     pts = [(i, fun(i, m)) for i in ii if fun(i, m) < 1000]
530                     ax.plot(*zip(*pts), color=c, lw=1.7, ls=ls)
531                 ops = [fun(i_dev, "f"), fun(i_dev, "d")]

```

```

532         label = "%s %.0f A: fast %s, delayed %s" % (name, i_dev, *(("%0.3f s" % x) if x < INF
else "no trip"
533
534         ax.axvline(i_dev, color=c, lw=0.8, alpha=0.7)
535         for t in ops:
536             if t < INF:
537                 ax.plot([i_dev], [t], marker="o", ms=6, color=c, mec=PAL["surface"], mew=1.5, zorder
=5)
538                 handles.append(Line2D([], [], color=c, lw=1.7, label=label))
539                 opts.append((dev, i_dev, ops, c))
540 handles.append(Line2D([], [], color=PAL["ink2"], lw=1.2, ls="--", label="dashed: recloser
delayed curve"))
541 ax.set_xscale("log")
542 ax.set_yscale("log")
543 ax.set_xlim(100, 1e5)
544 ax.set_ylim(0.01, 1000)
545 annotate_times(ax, opts)
546 ax.grid(True, which="major", color=PAL["grid"], lw=0.8)
547 ax.grid(True, which="minor", color=PAL["grid"], lw=0.3, alpha=0.6)
548 ax.set_xlabel("Current (A at 4.16 kV)")
549 ax.set_ylabel("Time (s)")
550 status = evaluate(rec, s, dsdr) if rec["where"] in NODES else None
551 extra = "" if status is None else (" - coordination held" if status["held"] else " -
coordination LOST")
552 ax.set_title("%s%s" % (title, extra), fontsize=9.5, loc="left") # the figure number is in
the caption
553 ax.legend(handles=handles, loc="upper right", fontsize=7.3, frameon=True, framealpha=0.92)
554 fig.tight_layout()
555 save_png(fig, fname)
556 plt.close(fig)
557
558
559 # =====
560 # main
561 # =====
562 def main():
563     t2 = table2()
564     s = design_reclosers(t2)
565
566     # ----- stage A: conventional scheme without DG -----
567     g, _ = classify(s, False, 0.0)
568     say("\n published fuse sizes, no DG: %d of %d cells held" % count(g))
569     for e in classify(s, False, 0.0)[1]:
570         if not e["held"]:
571             say("    lost %-4s %-3s %-3s %s" % (e["node"], e["type"], e["phases"], e["reason"]))
572     log_a = []
573     step8(s, False, 0.0, ["R2fw"], log_a)
574     step9(s, False, [0.0], log_a)
575     for l in log_a:
576         say(" " + l)
577     gA, dA = classify(s, False, 0.0)
578     say(" after steps 8-10, no DG: %d of %d cells held" % count(gA))
579     for e in dA:
580         if not e["held"]:
581             say("    lost %-4s %-3s %-3s %s" % (e["node"], e["type"], e["phases"], e["reason"]))
582     t3 = table3(s)
583     s_conv = copy.deepcopy(s)
584

```

```

585 # ----- stage B: DG added, conventional R2 -----
586 g14, d14 = classify(s_conv, False, 1.0)
587 say("\nSTAGE B - DG 4.05 MVA at 692, conventional (non-directional) R2 -> Fig. 14: %d of %d
    cells held" % count(g14))
588 for e in d14:
589     if not e["held"]:
590         say("    lost %-4s %-3s %-3s %s" % (e["node"], e["type"], e["phases"], e["reason"]))
591 coord_grid(g14, "Coordination without DSDR, DG 4.05 MVA at 692", "Fig14_without_DSDR.png")
592
593 # ----- stage C: R2 as DSDR -----
594 i_rv = max(LF["dg_in"]["R2"])
595 ip_rv = OLF * i_rv
596 tf, td = nearest_tap(ip_rv / 2, CDG_TAPS, CT_R2RV / 5), nearest_tap(ip_rv, CDG_TAPS, CT_R2RV /
    5)
597 s["R2rv"] = dict(ct=CT_R2RV, ip_eq12=ip_rv, i_nom_rv=i_rv, tap_f=tf, tap_d=td, is_f=tf * CT_R2RV
    / 5,
598                 is_d=td * CT_R2RV / 5, tms_f=s["R2fw"]["tms_f"], tms_d=s["R2fw"]["tms_d"],
599                 tms_d_max=TMS_MAX, tms_min=DSDR_TMS_MIN)
600 s["R2fw"]["tms_min"] = DSDR_TMS_MIN
601 say("\nSTAGE C - R2 AS DUAL-SETTING DIRECTIONAL RECLOSER")
602 say(" reverse load current with DG %.1f A (%.0f kW towards 632) -> Ip,rv = 1.25 x %.1f = %.1f A
    (eq. 12)" % (
603     i_rv, LF["dg_in"]["R2_P_kW"], i_rv, ip_rv))
604 say(" DSDR: CDG34 extremely-inverse curve in both directions, TMS %.1f-%.1f; reverse setting
    group on CT 500/5" % (
605     DSDR_TMS_MIN, TMS_MAX))
606 say(" reverse fast plug %.1f A = %.0f A (curve from %.0f A), delayed plug %.1f A = %.0f A (
    curve from %.0f A)" % (
607     tf, s["R2rv"]["is_f"], 2 * s["R2rv"]["is_f"], td, s["R2rv"]["is_d"], 2 * s["R2rv"]["is_d"]))
608 log_c = []
609 step8(s, True, 1.0, ["R2rv", "R2fw"], log_c)
610 step9(s, True, [1.0, 0.0], log_c)
611 step8(s, True, 1.0, ["R2rv", "R2fw"], log_c)
612 for l in log_c:
613     say(" " + l)
614 say(" final: reverse TMS %.3f (fast) / %.3f (delayed); forward TMS %.3f / %.3f" % (
615     s["R2rv"]["tms_f"], s["R2rv"]["tms_d"], s["R2fw"]["tms_f"], s["R2fw"]["tms_d"]))
616 g17, d17 = classify(s, True, 1.0)
617 g17_0, _ = classify(s, True, 0.0)
618 say(" Fig. 17: %d of %d cells held with DG; %d of %d without DG" % (count(g17) + count(g17_0)
    ))
619 for e in d17:
620     if not e["held"]:
621         say("    lost %-4s %-3s %-3s %s" % (e["node"], e["type"], e["phases"], e["reason"]))
622 coord_grid(g17, "Coordination with DSDR, DG 4.05 MVA at 692", "Fig17_with_DSDR.png")
623
624 # the same fuse sizes with the conventional R2 (to separate the DSDR's effect from step 9)
625 s_mix = copy.deepcopy(s)
626 s_mix["R2fw"] = copy.deepcopy(s_conv["R2fw"])
627 g14b, _ = classify(s_mix, False, 1.0)
628 say(" check: final fuse sizes but conventional R2 -> %d of %d cells held" % count(g14b))
629
630 rows = [[n, ft, g14[(n, ft)], g17[(n, ft)], g14b[(n, ft)]] for n in NODE_ORDER for ft in
    FAULT_TYPES]
631 write_csv("Fig14_Fig17_classification.csv", ["node", "fault", "without DSDR (Fig14)", "with DSDR
    (Fig17)",
632                                             "revised fuses, conventional R2"], rows)
633 write_csv(os.path.join("comparison", "Fig14_Fig17_vs_paper.csv"),

```

```

634         ["node", "fault", "Fig14 model", "Fig14 paper", "Fig17 model", "final fuses,
conventional R2"],
635         [[r[0], r[1], r[2], "n/a" if r[2] == "n/a" else ("lost" if (r[0], r[1]) in
PAPER_FIG14_LOST else "held"),
636          r[3], r[4]] for r in rows])
637 comp = [r for r in rows if r[2] != "n/a"]
638 say(" Fig. 14: %d of %d cells held; Fig. 17: %d of %d" % (
639     sum(r[2] == "held" for r in comp), len(comp), sum(r[3] == "held" for r in comp), len(comp)))
640
641 det = []
642 for tag, dd in (("conventional (Fig. 14)", d14), ("DSDR (Fig. 17)", d17)):
643     for e in dd:
644         det.append([tag, e["node"], e["type"], e["phases"], e["fuse"] or "", round(e.get("i_fuse", 0)),
645                     fmt(e["mmt"]) if e["fuse"] else "", fmt(e["tct"]) if e["fuse"] else "",
646                     " | ".join("%s %.0f A: %s / %s s" % (x[0], x[2], fmt(x[3]), fmt(x[4])) for x
in e["reclosers"]),
647                     "held" if e["held"] else "lost", e["reason"]])
648 write_csv("Coordination_detail.csv", ["scheme", "node", "fault", "phases", "primary fuse", "I
fuse (A)", "MMT (s)",
649                                         "TCT (s)", "reclosers: fast / delayed", "status", "reason"
], det)
650
651 # sympathetic operation of F671-1 (DG current flows up through it for faults outside its zone)
652 say("\nSYMPATHETIC CHECK - F671-1 carries the DG current for faults outside its zone (DSDR, DG
in)")
653 for node in ("632", "633", "645", "646", "DL", "671", "680", "684", "611", "652"):
654     worst = None
655     for r in faults("max", 1.0, node):
656         e = evaluate(r, s, True)
657         t_clear = max((x[3] for x in e["reclosers"] if x[0].startswith("R2")), default=None)
658         m = fuse(s, "F671-1").mmt(imax(r, "F671-1"))
659         ref = t_clear if t_clear is not None else None
660         if worst is None or m < worst[0]:
661             worst = (m, r["type"], r["phases"], imax(r, "F671-1"), ref)
662     m, ft, ph, i, ref = worst
663     say(" fault %-4s worst %-3s %-3s: F671-1 %5.0f A melts %s s; %s" % (
664         node, ft, ph, i, fmt(m), ("R2 reverse fast %s s -> %s" % (fmt(ref), "ok" if ref < m else
"F671-1 melts first"))
665         if ref is not None else "no recloser between the DG and the fault"))
666
667 # ----- Table IV -----
668 rep = {"632": "LLL", "633": "LLL", "645": "LL", "646": "LL", "DL": "LLL", "671": "LLL", "692": "
LLL",
669         "675": "LLL", "684": "LL", "680": "LLL", "611": "LG", "652": "LG"}
670 say("\nTABLE IV - operating times with the DSDR, DG in; bolted LLL (LL at 2-phase nodes, LG at
1-phase nodes)")
671 say(" %-4s %-3s | %-17s | %-22s | %-7s %-15s" % ("node", "flt", "R1 fast / delayed", "R2 unit
fast / delayed",
672                                         "fuse", "MMT / TCT"))
673 rows4 = []
674 for node, ft in rep.items():
675     r = max(faults("max", 1.0, node, ft), key=lambda x: imax(x, "R1"))
676     e = evaluate(r, s, True)
677     i1, i2 = imax(r, "R1"), imax(r, "R2")
678     unit = "R2fw" if NODES[node]["zone"] == "R2" else "R2rv"
679     r1 = (t_r1(i1, s, "f"), t_r1(i1, s, "d"))
680     r2 = (t_r2(i2, s, unit, "f"), t_r2(i2, s, unit, "d"))

```

```

681     rows4.append([node, ft, r["phases"], round(i1), fmt(r1[0]), fmt(r1[1]), round(i2), "fwd" if
unit == "R2fw" else "rev",
682                 fmt(r2[0]), fmt(r2[1]), e["fuse"] or "---", round(e.get("i_fuse", 0)) if e["
fuse"] else "",
683                 fmt(e["mmt"]) if e["fuse"] else "---", fmt(e["tct"]) if e["fuse"] else "---",
684                 "held" if e["held"] else "lost"])
685     say(" %-4s %-3s | %7s / %-7s | %s %7s / %-7s | %-7s %6s / %-6s%s" % (
686         node, ft, fmt(r1[0]), fmt(r1[1]), rows4[-1][7], fmt(r2[0]), fmt(r2[1]), e["fuse"] or "
---",
687         rows4[-1][12], rows4[-1][13], "" if e["held"] else "   LOST: " + e["reason"]))
688     head4 = ["node", "fault", "phases", "I R1 (A)", "R1 fast (s)", "R1 delayed (s)", "I R2 (A)", "R2
unit", "R2 fast (s)",
689             "R2 delayed (s)", "fuse", "I fuse (A)", "fuse MMT (s)", "fuse TCT (s)", "status"]
690     write_csv("Table_IV.csv", head4, rows4)
691     write_csv(os.path.join("comparison", "Table_IV_vs_paper.csv"),
692             head4 + ["paper R1 fast", "paper R1 delayed", "paper R2 fast", "paper R2 delayed", "
paper fuse MMT"],
693             [r + [x if x is not None else "---" for x in PAPER_TABLE4[r[0]]] for r in rows4])
694
695     # ----- time-current figures -----
696     fr = {c: [r for r in FAULTS if r["case"] == c][0] for c in ("Fig8", "Fig9", "Fig11", "Fig12", "
Fig13", "Fig15")}
697     tcc("Fig. 8", fr["Fig8"], s_conv, ["R2", "F684", "F671-2"], "LG fault at 611", "Fig08_LG_611.png
", False)
698     tcc("Fig. 9", fr["Fig9"], s_conv, ["R2", "F692-R", "F671-1"], "LLG fault mid 692-675, 1 ohm",
699         "Fig09_LLG_692-675.png", False)
700     tcc("Fig. 11", fr["Fig11"], s_conv, ["R1", "F646", "F632"], "LL fault at 646, 1 ohm", "
Fig11_LL_646_R1.png", False)
701     tcc("Fig. 12", fr["Fig12"], s_conv, ["R2", "F646", "F632"], "LL at 645, 1.5 ohm, conventional R2
",
702         "Fig12_LL_645_conventional_R2.png", False)
703     tcc("Fig. 13", fr["Fig13"], s_conv, ["R2", "F633"], "3-ph fault at 10 % of 632-633, conventional
R2",
704         "Fig13_LLL_632-633_conventional_R2.png", False)
705     tcc("Fig. 15", fr["Fig15"], s, ["R2rv", "F646", "F632"], "solid LL at 646, R2 as DSDR",
706         "Fig15_LL_646_DSDR.png", True)
707     tcc("Fig. 16", fr["Fig15"], s, ["R1", "F646", "F632"], "solid LL at 646, R1", "Fig16_LL_646_R1.
png", True)
708
709     out = {"conventional": s_conv, "dsdr": s}
710     json.dump(out, open(os.path.join(RES, "settings.json"), "w"), indent=1)
711     with open(os.path.join(RES, "summary.txt"), "w", encoding="utf-8") as f:
712         f.write("\n".join(REPORT))
713
714
715 if __name__ == "__main__":
716     main()

```

C.8 step4_apply_settings.py

Writes the settings into the relay and fuse elements of the model.

```

1  """
2  Step 4 - write the designed settings (results/settings.json) into PowerFactory and check that
3  PowerFactory's own relay and fuse models give the operating times used in step 3.

```

```

4
5 python step4_apply_settings.py          # DSDR scheme (Fig. 17) - left in the model
6 python step4_apply_settings.py conventional # conventional R2 (Fig. 14)
7
8 R2 units in the model: "R2 Fast" / "R2 Delayed"          forward setting group (CT 1000/5)
9                      "R2 Rev Fast" / "R2 Rev Delayed" reverse setting group (CT 500/5),
10                                     out of service in the conventional scheme
11 PowerFactory's library CDG34 is non-directional; the direction of each setting group is applied
12 in the coordination analysis (faults upstream of R2 = reverse, downstream = forward).
13 The check compares c:Ttrip of every unit and of the primary fuse with the Python curves.
14 """
15
16 import csv
17 import json
18 import os
19 import sys
20
21 from pf_setup import activate, attr, out, obj, lib_type, RESULTS, clean_variation
22 from protection_data import FUSES, NODES
23 import step3_design_and_evaluate as S3
24
25 scheme = sys.argv[1] if len(sys.argv) > 1 else "dsdr"
26 settings = json.load(open(os.path.join(RESULTS, "settings.json")))[scheme]
27
28 app = activate(build=True)          # settings belong to the base network
29 out("Applying the %s settings" % scheme)
30
31 for name, tname in settings["fuses"].items():
32     f = obj(name, "RelFuse")
33     f.typ_id = lib_type(tname + ".TypFuse", "ProtFuse")
34
35
36 def ct(cub, name, prim):
37     hit = cub.GetContents(name + ".StaCt")
38     c = hit[0] if hit else cub.CreateObject("StaCt", name)
39     equip = app.GetProjectFolder("equip")
40     tn = "CT %.0f-5A" % prim
41     t = equip.GetContents(tn + ".TypCt")
42     t = t[0] if t else equip.CreateObject("TypCt", tn)
43     t.SetAttribute("prmtaps", [prim])
44     t.SetAttribute("sectaps", [5.0])
45     c.typ_id = t
46     c.SetAttribute("ptapset", prim)
47     c.SetAttribute("stapset", 5.0)
48     return c
49
50
51 def set_unit(name, tap, dial, ct_obj=None, in_service=True):
52     r = obj(name, "ElmRelay")
53     r.SetAttribute("outserv", 0 if in_service else 1)
54     if ct_obj is not None:
55         sl = list(r.GetAttribute("pdiselm"))
56         sl[0] = ct_obj
57         r.SetAttribute("pdiselm", sl)
58     for toc in r.GetContents("*.RelToc"):
59         if "Earth" in toc.loc_name:
60             continue
61         toc.SetAttribute("Ipsetr", tap)

```

```

62         toc.SetAttribute("Tpset", dial)
63     return r
64
65
66 r1 = settings["R1"]
67 set_unit("R1 Fast", r1["tap"], r1["tds_f"])
68 set_unit("R1 Delayed", r1["tap"], r1["tds_d"])
69 cub2 = obj("R2 Fast", "ElmRelay").GetParent()
70 fw = settings["R2fw"]
71 ct_fw = ct(cub2, "CT R2", fw["ct"])
72 set_unit("R2 Fast", fw["tap_f"], fw["tms_f"], ct_fw)
73 set_unit("R2 Delayed", fw["tap_d"], fw["tms_d"], ct_fw)
74 if "R2rv" in settings:
75     rv = settings["R2rv"]
76     ct_rv = ct(cub2, "CT R2 rev", rv["ct"])
77     set_unit("R2 Rev Fast", rv["tap_f"], rv["tms_f"], ct_rv)
78     set_unit("R2 Rev Delayed", rv["tap_d"], rv["tms_d"], ct_rv)
79 else:
80     for u in ("R2 Rev Fast", "R2 Rev Delayed"):
81         set_unit(u, 1.0, 0.1, None, in_service=False)
82 out(" R1 tap %.1f A (CT 900/5) TDS %.2f / %.2f" % (r1["tap"], r1["tds_f"], r1["tds_d"]))
83 out(" R2 forward plug %.1f / %.1f A (CT %.0f/5) TMS %.3f / %.3f" % (fw["tap_f"], fw["tap_d"], fw["ct"], fw["tms_f"], fw["tms_d"]))
84 if "R2rv" in settings:
85     out(" R2 reverse plug %.1f / %.1f A (CT %.0f/5) TMS %.3f / %.3f" % (rv["tap_f"], rv["tap_d"], rv["ct"], rv["tms_f"], rv["tms_d"]))
86 out(" fuses: " + ", ".join("%s %s" % kv for kv in settings["fuses"].items()))
87
88 # ---- check PowerFactory's operating times against the Python curves -----
89 app = activate() # study case with the substation transformer
90 shc = app.GetFromStudyCase("ComShc")
91 shc.SetAttribute("iopt_mde", 3)
92 terms = {t.loc_name: t for t in app.GetCalcRelevantObjects("*ElmTerm")}
93 sym = app.GetCalcRelevantObjects("*ElmSym")[0]
94 PH = {"spg": "i_pspg", "2psc": "i_p2psc", "2pgf": "i_p2pgf"}
95 UNITS = [("R1 Fast", "R1", "f"), ("R1 Delayed", "R1", "d"), ("R2 Fast", "R2fw", "f"), ("R2 Delayed", "R2fw", "d")]
96 if "R2rv" in settings:
97     UNITS += [("R2 Rev Fast", "R2rv", "f"), ("R2 Rev Delayed", "R2rv", "d")]
98 s3 = dict(settings)
99
100
101 def pf_time(o):
102     v = attr(o, "c:Ttrip")
103     if v is None:
104         return float("inf")
105     v = v if isinstance(v, list) else [v]
106     v = [x for x in v if x is not None and 0 < x < 9999]
107     return min(v) if v else float("inf")
108
109
110 rows, worst = [], 0.0
111 for rec in [r for r in S3.FAULTS if r["case"] == "max" and r["dg"] in (0.0, 1.0) and r["where"] in
    NODES
    and not NODES[r["where"]].get("lv")]:
112     sym.SetAttribute("outserv", 0 if rec["dg"] else 1)
113     code = S3.D["faults"] and {"LG": "spg", "LL": "2psc", "LLG": "2pgf", "LLL": "3rst"}[rec["type"]]
114     ]]

```

```

115 shc.SetAttribute("shcobj", terms[rec["where"]])
116 shc.SetAttribute("iopt_shc", code)
117 if code in PH:
118     shc.SetAttribute(PH[code], rec["index"])
119 shc.SetAttribute("Rf", 0.0)
120 shc.SetAttribute("Xf", 0.0)
121 if shc.Execute() != 0:
122     continue
123 checks = []
124 for unit, key, mode in UNITS:
125     r = obj(unit, "ElmRelay")
126     t_pf = min(pf_time(t) for t in r.GetContents("*.RelToc") if not attr(t, "outserv"))
127     i = S3.imax(rec, "R1" if key == "R1" else "R2")
128     t_py = S3.t_r1(i, s3, mode) if key == "R1" else S3.t_r2(i, s3, key, mode)
129     checks.append((unit, t_pf, t_py))
130 path = NODES[rec["where"]]["path"]
131 if path:
132     f = obj(path[0], "RelFuse")
133     checks.append((path[0] + " MMT", pf_time(f), S3.FTYPE[settings["fuses"][path[0]]].mmt(S3.
134         imax(rec, path[0])))
135 for unit, t_pf, t_py in checks:
136     dev = 0.0 if t_pf == t_py else (abs(t_pf / t_py - 1) if t_py not in (0, float("inf")) else
137         1.0)
138     if t_py < 100:
139         worst = max(worst, dev)
140     rows.append([rec["where"], rec["type"], rec["phases"], rec["dg"], unit, S3.fmt(t_pf, 4), S3.
141         fmt(t_py, 4),
142         "%.2f" % (100 * dev) if dev < 1 else "-"])
143 sym.SetAttribute("outserv", 0)
144 clean_variation()
145 with open(os.path.join(RESULTS, "PF_vs_Python_times_%s.csv" % scheme), "w", newline="") as f:
146     w = csv.writer(f)
147     w.writerow(["node", "fault", "phases", "DG", "device", "PowerFactory t (s)", "Python t (s)", "
148         deviation %"])
149     w.writerows(rows)
150 devs = sorted(float(r[7]) for r in rows if r[7] != "-")
151 out("Checked %d operating times: PowerFactory vs Python median deviation %.2f %, max %.2f %" % (
152     len(rows), devs[len(devs) // 2], max(devs)))
153 bad = [r for r in rows if r[7] == "-" and r[5] != r[6]]
154 for r in bad[:10]:
155     out(" trip / no-trip mismatch: %s" % r)

```

C.9 step5_fig10_emt.py

EMT simulation of the reclosing sequence.

```

1 """
2 Step 5 - Fig. 10: time-domain fault current during the recloser's fast shots and the fuse operation.
3
4 Fault (paper): line-line a-c at 684 through 0.2 ohm, conventional settings (R2 forward).
5 Run twice in PowerFactory EMT: DG out (the paper's recloser-fuse sequence) and DG in.
6
7 Every switching time comes from the step-3 curves applied to the simulated currents
8 (one-cycle RMS of the EMT waveforms), in passes:
9     pass 1 fault only                -> R2 current -> R2 fast trip time

```

```

10     pass 2 R2 fast, reclose, fast, reclose (dead time 0.3 s)
11         -> F671-2 heating from its simulated current, including the DG current that keeps
12             flowing while R2 is open: melts when  $\text{sum}(\text{dt}/\text{MMT}(\text{I})) = 1$ , clears when  $\text{sum}(\text{dt}/\text{TCT}(\text{I})) = 1$ 
13         -> R2 delayed trip (timer from the last reclose)
14     pass 3 the whole sequence with the fuse opening (or R2 locking out if it is first)
15 PowerFactory's own relay and fuse elements are out of service during the runs (restored after).
16 """
17
18 import csv
19 import json
20 import math
21 import os
22
23 import matplotlib
24 matplotlib.use("Agg")
25 import matplotlib.pyplot as plt
26
27 from pf_setup import activate, out, obj, RESULTS, clean_variation
28 import step3_design_and_evaluate as S3
29
30 TO, T_END, DT = 0.30, 2.00, 1e-4 # fault at 0.30 s, as in the paper's figure
31 T_BREAKER = 3 / 60.0 # recloser interrupting time
32 T_DEAD = 0.20 # reclosing dead time read from the paper's figure (0.46 -> 0.65 s
33     , 0.80 -> 1.00 s)
34 FAST_SHOTS = 2
35 CYCLE = 1 / 60.0
36
37 s = json.load(open(os.path.join(RESULTS, "settings.json"))["conventional"])
38 FIG10 = [r for r in S3.FAULTS if r["case"] == "Fig10"][0]
39 FUSE = S3.FTYPE[s["fuses"]]["F671-2"]
40
41 app = activate()
42 case = app.GetActiveStudyCase()
43 evtfold = case.GetContents("*.IntEvt")[0]
44 sym = app.GetCalcRelevantObjects("*.ElmSym")[0]
45 brk = obj("LOHL632-671end", "ElmLne").GetAttribute("bus2").GetContents("*.StaSwitch")[0]
46 fsw = obj("LOHL671-684", "ElmLne").GetAttribute("bus1").GetContents("*.StaSwitch")[0]
47 res = case.GetContents("Fig10 EMT.ElmRes")
48 res = res[0] if res else case.CreateObject("ElmRes", "Fig10 EMT")
49 for c in res.GetContents():
50     c.Delete()
51
52 MON = [("I632in", obj("LOHL650-632", "ElmLne"), "bus2"), # feeder current arriving at node
53         ("I632", obj("LOHL632-671start", "ElmLne"), "bus1"), # current leaving 632 towards 671
54         ("IR2", obj("LOHL632-671end", "ElmLne"), "bus2"),
55         ("IF", obj("LOHL671-684", "ElmLne"), "bus1")] # through F671-2
56
57 for _, el, side in MON:
58     for p in ("A", "C"):
59         res.AddVariable(el, "m:I:%s:%s" % (side, p))
60
61 def simulate(events, tstop, csv_name):
62     app.ResetCalculation() # events cannot be edited while a simulation is initialised
63     for e in evtfold.GetContents():
64         e.Delete()
65         sc = evtfold.CreateObject("EvtShc", "LL a-c at 684")
66         sc.SetAttribute("p_target", obj("684", "ElmTerm"))
67         sc.SetAttribute("time", TO)

```

```

66     sc.SetAttribute("i_shc", 1) # 2-phase
67     sc.SetAttribute("i_p2psc", FIG10["index"]) # a-c (index found by the step-2 short circuit)
68     sc.SetAttribute("R_f", 0.2)
69     sc.SetAttribute("X_f", 0.0)
70     for k, (tt, kind, label) in enumerate(events):
71         ev = evtfold.CreateObject("EvtSwitch", "%02d %s" % (k, label))
72         ev.SetAttribute("p_target", fsw if kind == "fuse" else brk)
73         ev.SetAttribute("time", tt)
74         ev.SetAttribute("i_switch", 1 if kind == "close" else 0)
75         ev.SetAttribute("i_allph", 1)
76     inc = app.GetFromStudyCase("ComInc")
77     inc.SetAttribute("iopt_sim", "ins")
78     inc.SetAttribute("dtemt", DT)
79     inc.SetAttribute("p_resvar", res)
80     sim = app.GetFromStudyCase("ComSim")
81     sim.SetAttribute("tstop", tstop)
82     if inc.Execute() or sim.Execute():
83         raise RuntimeError("EMT simulation failed")
84     path = os.path.join(RESULTS, csv_name)
85     exp = case.CreateObject("ComRes", "export")
86     exp.SetAttribute("pResult", res)
87     exp.SetAttribute("iopt_exp", 6)
88     exp.SetAttribute("f_name", path)
89     exp.SetAttribute("iopt_sep", 0)
90     exp.SetAttribute("col_Sep", ",")
91     exp.SetAttribute("dec_Sep", ".")
92     exp.Execute()
93     exp.Delete()
94     app.ResetCalculation()
95     for e in evtfold.GetContents():
96         e.Delete()
97     with open(path) as f:
98         rows = list(csv.reader(f))
99     data = [[float(x) for x in r] for r in rows[2:] if r and r[0].strip()]
100    t = [r[0] for r in data]
101    wave = {}
102    for key, el, _ in MON: # PowerFactory sorts the columns: map by name
103        cols = [k for k in range(1, len(rows[0])) if rows[0][k] == el.loc_name]
104        ca = [k for k in cols if "Phase Current A" in rows[1][k][0]]
105        cc = [k for k in cols if "Phase Current C" in rows[1][k][0]]
106        wave[key] = ([r[ca] for r in data], [r[cc] for r in data])
107    return t, wave
108
109
110 def rms(t, x):
111     """one-cycle moving RMS"""
112     n = max(1, int(round(CYCLE / (t[1] - t[0]))))
113     out_, acc = [], 0.0
114     for k, v in enumerate(x):
115         acc += v * v
116         if k >= n:
117             acc -= x[k - n] * x[k - n]
118             out_.append(math.sqrt(max(acc, 0.0) / min(k + 1, n)))
119     return out_
120
121
122 def i_max_rms(t, wave, key):
123     a, c = rms(t, wave[key][0]), rms(t, wave[key][1])

```

```

124     return [max(p, q) for p, q in zip(a, c)]
125
126
127 SUMMARY = {}
128
129
130 def run(dg_on):
131     tag = "DG in" if dg_on else "DG out"
132     sym.SetAttribute("outserv", 0 if dg_on else 1)
133     # pass 1: R2 current -> fast trip time
134     t, w = simulate([], T0 + 0.15, "Fig10_pass.csv")
135     ir2 = i_max_rms(t, w, "IR2")
136     i_r2 = max(v for tt, v in zip(t, ir2) if T0 + 0.05 < tt < T0 + 0.15)
137     t_fast = S3.t_r2(i_r2, s, "R2fw", "f") + T_BREAKER
138     events, tt = [], T0
139     for shot in range(FAST_SHOTS):
140         tt += t_fast
141         events.append((tt, "open", "R2 fast trip %d" % (shot + 1)))
142         tt += T_DEAD
143         events.append((tt, "close", "R2 reclose %d" % (shot + 1)))
144     t_last = tt
145     # pass 2: fuse heating from the simulated fuse current
146     t, w = simulate(events, T_END, "Fig10_pass.csv")
147     iF = i_max_rms(t, w, "IF")
148     ir2 = i_max_rms(t, w, "IR2")
149     i_fault = max(v for tt_, v in zip(t, iF) if T0 < tt_ < T0 + t_fast) # fuse current during
150                                     the fault
151     melt = clear = 0.0
152     t_melt = t_clear = None
153     how = "clears"
154     t_delayed = None
155     acc_d = 0.0
156     for k in range(1, len(t)):
157         dt = t[k] - t[k - 1]
158         if t[k] < T0:
159             continue
160         i = iF[k]
161         if i > 0:
162             melt += dt / FUSE.mmt(i)
163             clear += dt / FUSE.tct(i)
164             if t_melt is None and melt >= 1:
165                 t_melt = t[k]
166             if t_clear is None and clear >= 1:
167                 t_clear = t[k]
168             # a fuse that has melted is finished as soon as a recloser interrupts its current
169             if t_clear is None and t_melt is not None and t[k] > t_melt + CYCLE and i < 0.05 * i_fault:
170                 t_clear, how = t[k], "is blown (melted, current interrupted by R2)"
171             if t[k] > t_last + 2 * CYCLE and t_delayed is None: # R2 delayed timer after last
172                 reclose
173                 acc_d += dt / S3.t_r2(max(ir2[k], 1.0), s, "R2fw", "d")
174                 if acc_d >= 1:
175                     t_delayed = t[k] + T_BREAKER
176     heat_fast = None
177     for k in range(len(t)):
178         if t[k] >= t_last:
179             break
180     melt_at_last = sum((t[j] - t[j - 1]) / FUSE.mmt(iF[j]) for j in range(1, len(t))
181                        if T0 <= t[j] <= t_last and iF[j] > 0)

```

```

180 if t_clear is not None and (t_delayed is None or t_clear < t_delayed):
181     # the fault is gone after the fuse: no further recloser trips, only a pending reclose
182     kept = [e for e in events if e[0] < t_clear]
183     if kept and kept[-1][1] == "open":
184         kept.append(events[len(kept)])
185     events = sorted(kept + [(t_clear, "fuse", "F671-2 " + how.split(" ")[0])])
186     final = "F671-2 %s at %.3f s" % (how, t_clear)
187 elif t_delayed is not None:
188     events.append((t_delayed, "open", "R2 lockout"))
189     final = "R2 locks out at %.3f s" % t_delayed
190 else:
191     final = "no clearing within %.1f s" % T_END
192 t, w = simulate(events, T_END, "Fig10_EMT_%s.csv" % tag.replace(" ", "_"))
193 out("Fig. 10 %s: R2 %.0f A -> fast shot %.3f s incl. breaker; fuse heat at the end of the fast
shots "
194     "%.0f %% of melting; melts %s, %s" % (tag, i_r2, t_fast, 100 * melt_at_last,
195     "at %.3f s" % t_melt if t_melt else "-", final))
196 for e in events:
197     out("    t = %.3f s %s" % (e[0], e[2]))
198 ra, rc = rms(t, w["I632in"][0]), rms(t, w["I632in"][1])
199
200 def at(x, when):
201     return x[min(range(len(t)), key=lambda j: abs(t[j] - when))]
202
203 def peak(x, t1, t2):
204     return max(abs(v) for tt_, v in zip(t, x) if t1 <= tt_ <= t2)
205 node632 = dict(prefault_rms_a=at(ra, T0 - 0.002), prefault_rms_c=at(rc, T0 - 0.002),
206     fault_rms_a=at(ra, T0 + t_fast - 0.005), fault_rms_c=at(rc, T0 + t_fast - 0.005),
207     first_peak_a=peak(w["I632in"][0], T0, T0 + 0.03), first_peak_c=peak(w["I632in"
208     ] [1], T0, T0 + 0.03),
209     steady_peak_a=peak(w["I632in"][0], T0 + 0.06, T0 + t_fast - 0.01),
210     steady_peak_c=peak(w["I632in"][1], T0 + 0.06, T0 + t_fast - 0.01),
211     dead_time_rms_a=at(ra, events[0][0] + T_DEAD - 0.02), dead_time_rms_c=at(rc,
212     events[0][0] + T_DEAD - 0.02))
213 out("    current into node 632 (A): prefault rms a/c %.0f / %.0f; fault rms a/c %.0f / %.0f, peak
a/c %.0f / %.0f "
214     "(first peak %.0f / %.0f); R2 open rms a/c %.0f / %.0f" % (
215     node632["prefault_rms_a"], node632["prefault_rms_c"], node632["fault_rms_a"], node632["
216     fault_rms_c"],
217     node632["steady_peak_a"], node632["steady_peak_c"], node632["first_peak_a"], node632["
218     first_peak_c"],
219     node632["dead_time_rms_a"], node632["dead_time_rms_c"]))
220 SUMMARY[tag] = dict(node632=node632, t_fault=T0, t_dead=T_DEAD, i_r2=i_r2, i_fuse=i_fault,
221     t_fast=t_fast, heat_after_fast_shots=melt_at_last, t_melt=t_melt,
222     t_clear=t_clear, final=final, events=[(round(e[0], 3), e[2]) for e in events
223     ],
224     fuse=s["fuses"]["F671-2"], fast_shots_before_melt=sum(
225     1 for e in events if e[1] == "open" and (t_melt is None or e[0] <=
226     t_melt)))
227 return tag, t, w, events, melt_at_last, t_melt
228
229 # PowerFactory's own relay and fuse models would also act in the EMT run; the sequence is
230 # scripted from the curves instead, so they are taken out of service for the simulation.
231 PROT = [(o, o.GetAttribute("outserv")) for o in app.GetCalcRelevantObjects("*.ElmRelay") +
232     app.GetCalcRelevantObjects("*.RelFuse")]
233 try:
234     for o, _ in PROT:

```

```

229         o.SetAttribute("outserv", 1)
230     cases = [run(False), run(True)]
231 finally:
232     app.ResetCalculation()
233     for o, st in PROT:
234         o.SetAttribute("outserv", st)
235     sym.SetAttribute("outserv", 0)
236     clean_variation() # the switching above was recorded in the variation stage
237 os.remove(os.path.join(RESULTS, "Fig10_pass.csv"))
238
239 fig, axes = plt.subplots(2, 1, figsize=(7.4, 6.2), sharex=True)
240 for ax, (tag, t, w, events, heat, t_melt) in zip(axes, cases):
241     ax.plot(t, w["I632in"][1], lw=0.7, color="#d03b3b", label="phase c, feeder current at node 632")
242     ax.plot(t, w["I632in"][0], lw=0.7, color="#2a78d6", label="phase a, feeder current at node 632")
243     ax.plot(t, w["IF"][0], lw=0.6, color="#8a8983", alpha=0.75, label="phase a through fuse F671-2")
244     top = max(max(map(abs, w["IF"][0])), max(map(abs, w["I632in"][0])), max(map(abs, w["I632in"][1])
245     ))
246     for tt, kind, label in events:
247         ax.axvline(tt, color="#52514e", lw=0.7, ls=":")
248         ax.text(tt, top * 1.02, label, rotation=90, va="top", ha="right", fontsize=6.8, color="
249             #52514e")
250     ax.axvline(T0, color="#52514e", lw=0.8, ls="--")
251     ax.text(T0, top * 1.02, "fault initiated", rotation=90, va="top", ha="right", fontsize=6.8,
252         color="#52514e")
253     if t_melt:
254         ax.axvline(t_melt, color="#eb6834", lw=1.0, ls="--")
255         ax.text(t_melt, -top * 1.02, "F671-2 melts", rotation=90, va="bottom", ha="right", fontsize
256             =6.8,
257             color="#eb6834")
258     ax.set_ylim(-1.15 * top, 1.15 * top)
259     ax.set_ylabel("Current (A)")
260     ax.set_title("%s (fuse F671-2 %s)" % ("With DG" if tag == "DG in" else "Without DG", s["fuses"][
261         "F671-2"].replace("A055C", "")), fontsize=9, loc="left")
262     ax.grid(True, color="#d9d8d4", lw=0.5)
263     for sp in ("top", "right"):
264         ax.spines[sp].set_visible(False)
265     axes[0].legend(frameon=False, fontsize=7.2, loc="lower left", ncol=3)
266     axes[1].set_xlabel("Time (s)")
267     fig.suptitle("Current at node 632, LL a-c fault at 684 through 0.2 ohm, PowerFactory EMT",
268         fontsize=9.5, x=0.01, ha="left")
269     fig.tight_layout()
270     try:
271         fig.savefig(os.path.join(RESULTS, "figures", "Fig10_EMT_LL_684.png"), dpi=160)
272     except OSError:
273         # open in a viewer
274         fig.savefig(os.path.join(RESULTS, "figures", "Fig10_EMT_LL_684_new.png"), dpi=160)
275         out("NOTE: Fig10_EMT_LL_684.png is open in another program; the new figure is
276             Fig10_EMT_LL_684_new.png")
277 json.dump(SUMMARY, open(os.path.join(RESULTS, "Fig10_summary.json"), "w"), indent=1)
278 out("Saved figures/Fig10_EMT_LL_684.png, results/Fig10_EMT_DG_*.csv and Fig10_summary.json")

```

APPENDIX D: COMPLETE COPY OF THE REFERENCE PAPER

The complete reference paper [1] is attached as a separate, unchanged PDF copy in accordance with the requirement of the department. It is not reproduced inside this document.